Hamzstlab Mathematics and Hamzstplot

Freya · Hamzst · Lasthrim · DS Glanzsche[1]

Berlin-Sentinel Academy of Science, AliceGard

2025 Edition

[1] A thank you or further information

Contents

Preface

For my Wife Freya, and our daughters Solreya, Mithra, Catenary, Iyzumrae and Zefir.

For Lucrif and Znane too along with all the 8 Queens.

For Albert Silverberg, Camus and Miklotov who left TREVOCALL to guard me.

To Nature(Kala, Kathmandu, Big Tree, Sentinel, Aokigahara, Hoia Baciu, Jacob's Well, Mt Logan, etc) and my family Berlin: I have served, I will be of service.

To my dogs who always accompany me working in Valhalla Projection, go to Puncak Bintang or Kathmandu: Sine Bam Bam, Kecil, Browni Bruncit, Sweden Sexy, Cambridge Klutukk, Milan keng-keng, Piano Bludut and more will be adopted. To my cat who guard the home while I'm away with my dogs: London.

The one who moves a mountain begins by carrying away small stones - Confucius

Come to me, all who labor and are heavy burdened, and I will give you rest. - The Bible, Matthew 25:28

A book to explained how we create a C++ library from forking / branching out ImGUI 1.92.0 WIP, Implot, Implot 3D, merge them into one big pile that we call Hamzstlab Mathematics so it can be easily compiled / build so we can focus more on the Mathematics side / learning.

We also write about Hamzstplot that is forking / branching out from Matplot++, it is a C++ plotting library with backend gnuplot, so you can have more creativity in plotting since the syntax will be all C++. It is not as interactive as Hamzstlab Mathematics but it has more options of plotting that you can play with.

a little about GlanzFreya:
Freya the Goddess is (DS Glanzsche') Goddess wife. We get married on Puncak Bintang on November 5th, 2020 after we go back from Waghete, Papua. My wife birthday (Freya the Goddess) is on August 1st. After not working with human anymore since 2019, I (DS Glanzsche) work in a forest, shaping a forest into a natural wonder / like a canvas for painting, and also clean up trashes that are thrown in the forest. Thus after 6 years working with Nature I have found what can makes me waltz to work, it is going to the forest in the morning then go back home again and study / learn science, engineering, computer programming.

Freya specialty is Electrical Engineering, she is the one that taught Glanz about Arduino Uno and all that.

Glanz specialty is Dogs, she loves dogs and love feeding stray dogs and taking home stray puppies given she has money and foods, otherwise she will bring stray dogs to animal shelter.

a little about Hamzst:
Hamzst is a nickname for Alice from Persona game series made by Atlus, sometimes she can also be similar with Alice from Alice in Wonderland (1951 movie). When she said "But in my world, the books will be nothing but picture," it is true for Hamzstplot and Hamzstlab Mathematics. She is a Ding, Yin Fire daymaster (The candle), she is the controller of Glanz and bring a lot of luck to Glanz too. Her specialty is Physics.

a little about Lasthrim:
She was known as Anabelle the Doll from the famous movie "The Conjuring," but she changes her name into Lasthrim since 2021 and want a true repentance since she has found a new joy and Love in mathematics. She is a Bing, Yang Fire daymaster (The Sun), she can shapes Glanz into the best Excalibur. Her specialty is Mathematics. Realizing that after writing Lasthrim Projection book, it is better to use our own C++ library modifying from existing C++ library to make the plot, computation and simulation, instead of using Python and JULIA. So here she is.

Figure 1: *Freya, thank you for everything, I am glad I marry you and I could never have done it without you.*

Figure 2: *I paint her 3 days before Christmas in 2021.*

Chapter 1

Introduction Story

On top of the mountain a heart shaped stone waiting for You, my eyes can't see, my body can't touch, but my heart knows it's You. Forever only You. - ***Glanz to Freya***

This book is written on May 29th, 2025, while we also have another project to make SymIntegration C++ library, we are working on this when we saw Implot, Implot 3D, and Matplot++. We want to clone them and modify them, or probably add some features to plot, simulate and more in the future, it is called branching out / forking. Then rename it as Hamzstlab Mathematics for combining Imgui+Implot+Implot3D+Imnodes, as a C++ library / software where you can create interactive graph or simulation for mathematics problems. It will be very useful for students and researchers. While Hamzstplot is branching out from Matplot++ to create graph and plot as well but it is not interactive as Hamzstlab Mathematics but the options for building the plot are enormous. We can choose which one to use depends on our need.

To avoid confusion in the future: Whenever there is a written of subject "I" it means DS Glanzsche as the I subject.

Years ago, around December 2019 when I (DS Glanzsche) was hiking / jogging up to Puncak Bintang, I pray at dawn, I remember I used to arrive there at 4:30 AM , departing around 3:00 AM from home, what I pray is " I pray and hope one day I can create a software like MATLAB," well I am starting this and remember that prayer again, it is becoming true, when I was installing MATLAB back then in 2011 to 2014 for college purpose, they give MATLAB course to students and even pay the license for it, I think it is great, but then realize, when I was reading a book like "Elementary Linear Algebra" or ""Elementary Differential Equation" or "Calculus" none of them have the MATLAB code to show how the plot is made or the computation is being computed, even if today there is a book called "Advanced Engineering Mathematics with MATLAB" we still need the code for the basic that undergraduate needs, since the knowledge for other science and engineering department will rely heavily on those Calculus, Linear Algebra, and Differential Equations first.

Not to mention, that there are tons of menu / on the menu bar on MATLAB, that the user won't even need to use for the rest of his/her life, the size of MATLAB today is over 20 GB, and for the installation it takes hours, now I am comparing with Hamzstlab Mathematics, if you are an Electrical Engineer, and we already create the basic code and API to create any electric circuit with basic NPN' or PNP' BJT transistor or MOSFET n-channel or p-channel along with any number of resistors, inductor, and capacitor and will have the computed voltage and current, and you

probably won't even need to download Hamzstlab Mathematics for even 1 GB, isn't it going to save your day and your time?

This is the real life example where we try to plot a solution to a simple first order ordinary differential equation and the Euler' method approximation, the size of the C++ file and the Makefile only consumes 3.1 KB, and if you build the example the C++ codes it will create this object files that will consumes 27.3 MB of size of your storage, which can be cleaned after you are finish. Hamzstlab Mathematics will only consume your storage of 50.3 MB, and the rest will depends on the project that you will be working on.

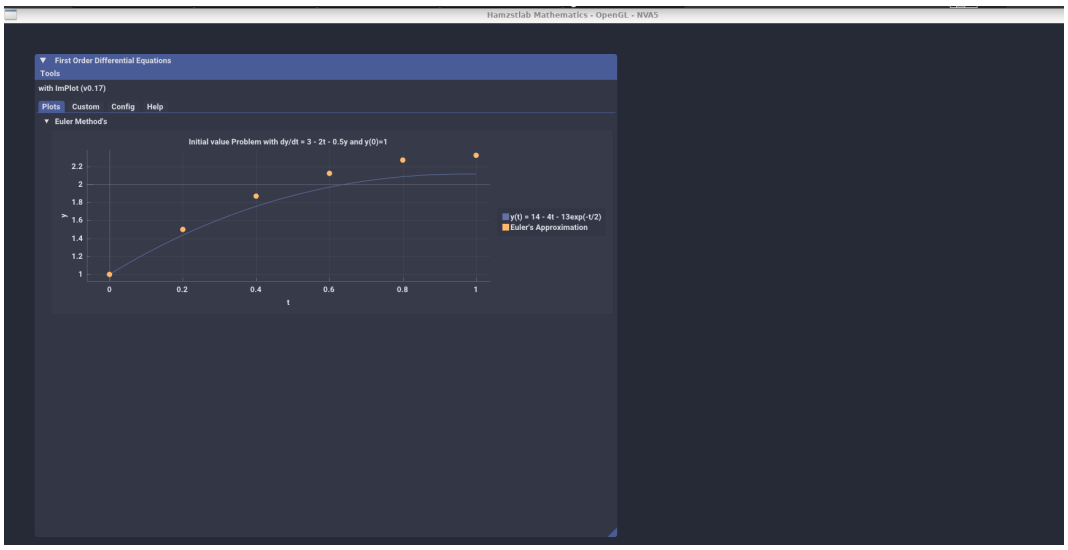

Figure 1.1: *The interactive plot of Euler' method approximation and the solution plot for* $\frac{dy}{dt} = 3 - 2t - \frac{1}{2}y$ *with Hamzstlab Mathematics.*

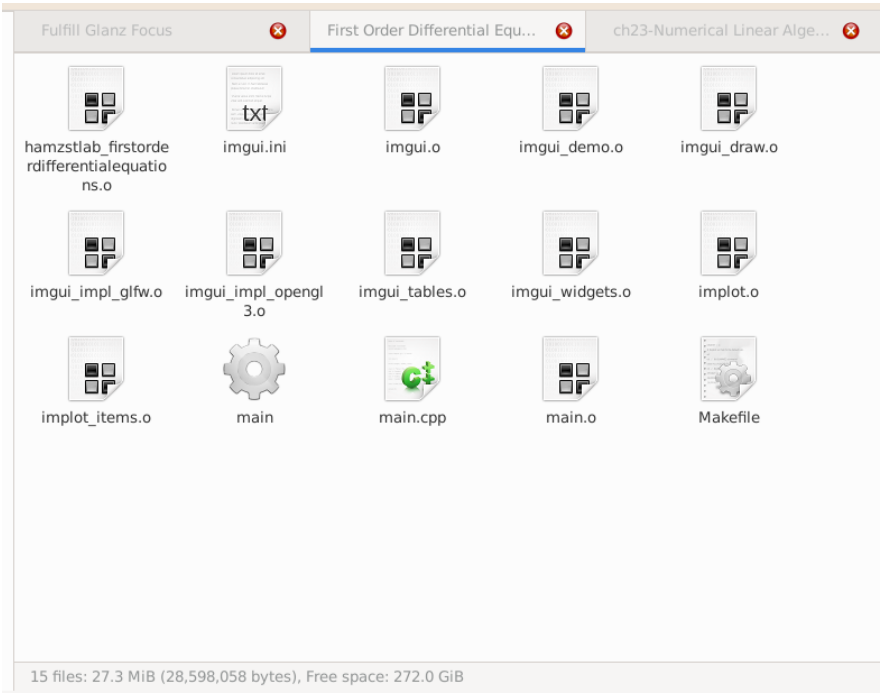

Figure 1.2: *The size of a built example with all the object files.*

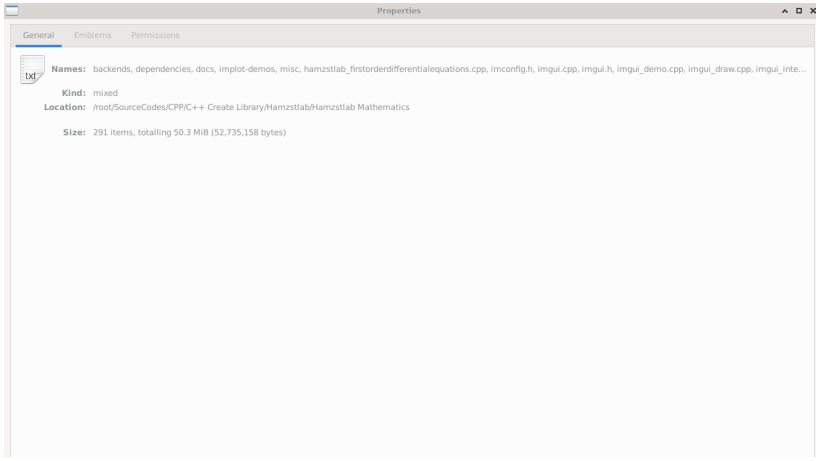

Figure 1.3: *The size of Hamzstlab Mathematics source code as of June 4th, 2025.*

On May 26th, 2025 while in St Peter Cathedral in Bandung I was reading a book [9], then at the first chapter it stated like this:
If all you are interested in is a quick calculation, then Python along with its ecosystem is likely going to be your one-stop shop. As your work becomes more computationally challenging, you may need to switch to a compiled language; most work on supercomputers is carried out using languages like Fortran or C++ or sometimes even C.

Basically, what Imgui combined with Implot and Implot 3D now able to do, show a lot of

interactive visualizations in one window, is a big gap between C++ and Python, like the sky and the earth. I (DS Glanzsche) as one of the author, starts learning C++ rigorously since October 2023, still very infant stage, then able to write DianFreya Math Physics Simulator and already finish the whole Elementary Linear Algebra book along with all the C++ codes for plotting and simulation. Previously, I was using Python and JULIA to do the computation while writing the book Lasthrim Projection (focusing on Calculus and basic Mathematics field that are more theoretical like Discrete Mathematics, Elementary Differential Equations, Partial Differential Equations). Now I have learned Python, JULIA, C++, Fortran, and C. I know writing a code with C++ to achieve the same result will take longer and steeper learning curve, but it is worth the time spent.

The Operating System and Softwares in Use:

1. GFreya OS version 1.8 (based on LFS and BLFS version 11.0 System V books)

2. GCC version 11.2.0 (to compile C++ codes)

3. Hamzstplot version 1.5

4. Hamzstlab Mathematics version 1.51

We will explain about Hamzstplot and Hamzstlab Mathematics in the next two chapters, then the rest of the chapters will be talking about their application (in creating the plot, interactive plot, or simulation with either Hamzstplot or Hamzstlab Mathematics).

The content of this book covers the basic undergraduate courses in Mathematics from Differential Equation to Numerical Analysis and Linear Programming. If one is keen and acute enough and wonder, why there is no Elementary Linear Algebra here? Guilty. It is because we have done the summarization of the whole Elementary Linear Algebra based on [16] book in another book [8] that we are working on from 2023 to 2025, and we are mainly using C++ libraries that are already well known like Armadillo, Eigen, and GiNaC to help us for the matrix and vector computation till QR method, spectral decomposition, eigenvalue and eigenvector. We let the reader read that book or any other Elementary Linear Algebra book to get the basic knowledge of Linear Algebra, since it is really needed for almost all fields in mathematics, science and engineering. This books is focusing on showing what Hamzstplot, Hamzstlab Mathematics can do to help compute, plot and simulate the mathematics problems we have in each branch / field. We didn't mention it in the cover but we will also use SymIntegration for the symbolic integration computation part.

We really appreciate those who spend time to write constructing critics, not hate words without reason that will only add more good luck and karma to me, write opinions on what should be added in Hamzstlab Mathematics and Hamzstplot or if there is an incorrect algorithm, please inform the author. Feedbacks can be sent to **dsglanzsche@gmail.com**.

Chapter 2

Hamzstplot

"You can't run from what you see and feel." - ***Odessa Silverberg***

Hamzstplot was born on a Full Moon day on May 12th, 2025. We are branching out / forking from Matplot++ and we want to develop it further, changing the name first and will add several features in the future to help us in learning Mathematics, or other branch of science, e.g. to create direction fields. The basic syntax of C++ will really help for a very long term future prospects and more complex problems in science to be computed, analyzed and plot.

I. Build and Install Hamzstplot in GFreya OS

We are using GFreya OS as it is a custom Linux-based Operating System, your distro your rules. The commands here assuming that you / the reader are using Linux-based OS,they are familiar and easy to follow for other Linux-based OS users.

To download Hamzstplot, open terminal / xterm then type:
git clone https://github.com/glanzkaiser/Hamzstplot.git
Enter the directory then type:

```
mkdir build
cd build
cmake —DBUILD_SHARED_LIBS=ON ..
make
make install
```

Code 1: *Create Hamzstplot dynamic library*

It will create the dynamic library **libhamzstplot.so**, we are using CMake, and it is said that dynamic library saves more RAM memory than static library.

The prefix at default is **/usr/local** so the installed dynamic library will go to **/usr/local/lib**.

Then open terminal and from the current working directory / this repository main directory:

```
cd /source/3rd_party/cimg/
cp —r CImg.h* /usr/local/include

cd /

cd ../Hamzstplot/source/hamzstplot/
cp —r * /usr/local/include

cd ../Hamzstplot/source/3rd_party/nodesoup/include/
cp —r nodesoup.hpp /usr/local/include

cd /
cd ../Hamzstplot/source/3rd_party/nodesoup/src/
bash dynamiclibrary.sh
cp —r libnodesoup.so /usr/local/lib
```

Code 2: *Move include files and 3rd party library*

When we want to create the shared library it will look for this header files in the default path where the include files are usually located. In Linux OS **usr/include** is the basic / default path, another default path is **usr/local/include**.

All files, examples codes and source codes used in this books can be found in this repository:
https://github.com/glanzkaiser/Hamzstplot

II. Example of Plotting with Hamzstplot in GFreya OS

We will create a 3D plot of a helix, the source code is this:

```
#include <cmath>
#include <hamzstplot/hamzstplot.h>

int main() {
       using namespace hamzstplot;

       auto xt = [](double t) { return sin(t); };
       auto yt = [](double t) { return cos(t); };
       auto zt = [](double t) { return t; };
       fplot3(xt, yt, zt);

       show();
       return 0;
}
```

Code 3: *Hamzstplot/Examples with Makefile/Plot 3D Helix/main.cpp*

Open terminal and from the current working directory **Hamzstplot/Examples with Makefile/-Plot 3D Helix/**, then type:

```
make
./main
```

Code 4: *Build and run the executable*

or you can also type in the current working directory (a manual way to compile):
g++ main.cpp -o main -lhamzstplot
./main

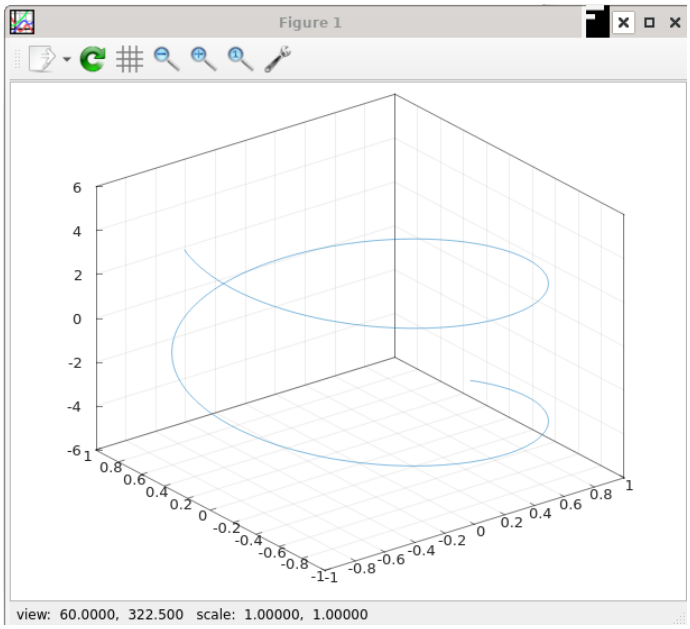

Figure 2.1: *The plot here is using the gnuplot as the backend, that is why the GUI window should look very familiar.*

Chapter 3

Hamzstlab Mathematics

"In this world perhaps, but in my world, the books will be full of pictures" - ***Alice (Alice in Wonderland - 1951)***

"If in other sciences we should arrive at certainty without doubt and truth without error, it behooves us to place the foundations of knowledge in mathematics." - ***Roger Bacon***

"If I were again beginning my studies, I would follow the advice of Plato and start with mathematics." - ***Galileo Galilei***

Why would we need Hamzstlab Mathematics if we already have Hamzstplot? For the extra interactive feature, better simulation, even if the backend of gnuplot or OpenGL from Hamzstplot is already good enough, the features in Hamzstlab Mathematics that is coming from ImGUI, Implot, Implot3D, Imnodes can reach to almost all branches of science and engineering, help them to design and make prototype. Box2D is using ImGUI and it helps me personally to create some basic Physics simulation from an undergraduate book "Fundamental of Physics."

You can design any kind of electric circuit here with all the computation necessary too, you can simulate how a mechanical system works, how leverage works, how a human organ works, how a nuclear fission works, and you can even do gene sequence simulation better here, they are all in my head now, but one day we will add the source code and API so people can really create what I am talking now.

In the end, there is no perfect software, one software or one C++ library complement the other. That is life. Just like marriage, since nobody is perfect, just be honest, no lie no cheating, accept your partner for whoever she is, with no extramarrital affair / lie / cheating, then you both will be happy and reach your goal together, with a bonus become power couple too.

I. Build and Install Hamzstlab Mathematics in GFreya OS

We are using GFreya OS as it is a custom Linux-based Operating System, your distro your rules. The dependencies are:

1. ImGUI version 1.92.0 WIP
2. Implot version 0.17
3. Implot-3D version 0.3 WIP
4. Imnodes

There is no need to download all of the above one by one anymore, because they are already blended into one in Hamzstlab Mathematics.

To download Hamzstlab Mathematics, open terminal / xterm then type:
git clone https://github.com/glanzkaiser/Hamzstlab-Mathematics.git

Enter the directory then type:

```
cd dependencies

cp −r cxxopts.hpp /usr/include
```

Code 5: *Move an include file / header*

For the **implot-demos** we need a few compiling first, from the main directory open the terminal then type:

```
cd implot−demos

mkdir build
cd build
cmake ..
make

cp −r libimnodes.a libimplot.a libapp.a libimgui.a ../../dependencies/
```

Code 6: *Create implot-demos related static library*

The steps above are used to create the static library to run some demos from **implot-demos** that are more complex than the normal implot demo.

Alternative way besides static library:
You don't really need to create all the static libraries, considering if the dependencies are updated and you need a new feature. We think it is better to take the source files instead, e.g. if you take a look at our example:
Hamzstlab-Mathematics/examples/ImNodes Test/

It is added on June 18th, 2025. We don't use **libimnodes.a**, instead we put the source files for **Imnodes** (imnodes.cpp, imnodes.h, imnodes_internal.h) on the parent directory **Hamzstlab-Mathematics/** and adjust the Makefile in that example to process the source codes of Imnnodes into

object files so the example that needs **Imnodes** feature will be able to be built as executable and run.

Now, you are set to go, in the **Hamzstlab-Mathematics/examples/** directory there are tons of examples that you can just build with Makefile, easy.

For example, from the main directory **Hamzstlab-Mathematics/** open terminal

```
cd examples

cd 3D Test

make
./main
```

Code 7: *Build an example*

This will run the 3D test. You can check each examples source code and learn how to modify according to your needs.

All files, examples codes and source codes used in this books can be found in this repository: **https://github.com/glanzkaiser/Hamzstlab-Mathematics**

II. Example of Creating a 2D Interactive Plot with Hamzstlab-Mathematics in GFreya OS

First you have to already clone or download the repository, then open terminal at the main working directory **Hamzstlab-Mathematics** and type:

```
cd examples

cd First Order Differential Equation

make
./main
```

Code 8: *Create dynamic library*

This will be the result:

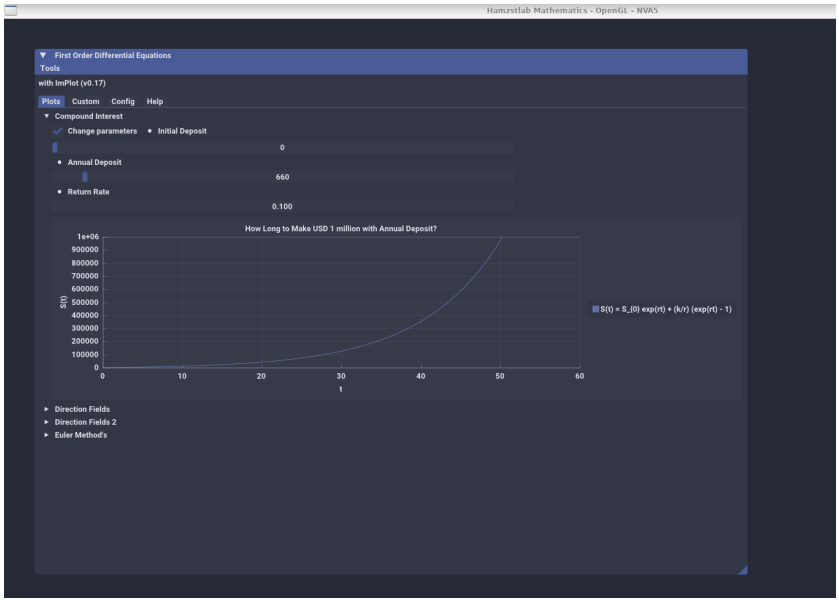

Figure 3.1: *The interactive plot of $S(t)$ with Hamzstlab-Mathematics you can change the parameters and the plot will adjust with the parameters that you just input.*

Now, we are going to explain the code on how to make this example works.

```
#include "App.h"

struct ImPlotDemo : App {
	using App::App;
	void Update() override {
		ImPlot::ShowFirstOrderDEWindow();
	}
};
```

```
int main(int argc, char const *argv[])
{
	ImPlotDemo app("Hamzstlab Mathematics",1920,1080,argc,argv);
	app.Run();

	return 0;
}
```

Code 9: *Hamzstlab-Mathematics/examples/First Order Differential Equation/main.cpp*

This **main.cpp** is pointing toward **Hamzstlab-Mathematics/hamzstlab_firstorderdifferentialequations.cpp**, so if you want to modify the example, you just need to edit **Hamzstlab-Mathematics/hamzstlab_firstorderdifferentialequations.cpp**.

The **ImPlot::ShowFirstOrderDEWindow();** needs to be an **IMPLOT_API**, and this can be done in **Hamzstlab-Mathematics/implot.h**, so you need to add a line like this there

```
...

IMPLOT_API void ShowFirstOrderDEWindow(bool* p_open = nullptr);

...
```

Code 10: *Hamzstlab-Mathematics/implot.h*

For example, from hundred lines of codes you can check the code here to create a plot of Compound Interest, if you scroll down again you will know that we are showing this **Demo_CompoundInterest()** under a new tab that can be opened by the user.

There might be hundreds to thousands of lines of codes, but if you are rigorous and persistence you will get used to it and able to create something from your imagination.

```
...
...

void Demo_CompoundInterest() {
	static float xs1[1001], ys1[1001];
	static int S0 = 0;
	static float r = 0.08;
	static int k = 2000;
	for (int i = 0; i < 1001; ++i) {
		xs1[i] = i;
		ys1[i] = S0 * exp(r*i) + (k/r)*(exp(r*i) - 1) ;
	}

	static bool range = false;
	ImGui::Checkbox("Change parameters", &range);

	if (range) {
		ImGui::SameLine();
```

```
                ImGui::SetNextItemWidth(200);
                ImGui::BulletText("Initial Deposit");
                ImGui::SliderInt("##Initial Deposit", &S0, 0, 10000);
                ImGui::SetNextItemWidth(200);
                ImGui::BulletText("Annual Deposit");
                ImGui::SliderInt("##Annual Deposit", &k, 1, 10000);
                ImGui::SetNextItemWidth(200);
                ImGui::BulletText("Return Rate");
                ImGui::DragFloat("##Return rate", &r, 0.01f, 0.2f);
                ImGui::SetNextItemWidth(200);
        }
        if (ImPlot::BeginPlot("How Long to Make USD 1 million with Annual
            Deposit?")) {
                ImPlot::SetupAxes("t","S(t)");
                ImPlot::SetupAxesLimits(0, 60, 0, 1000000);
                ImPlot::SetupLegend(ImPlotLocation_East,
                    ImPlotLegendFlags_Outside);

                ImPlot::PlotLine("S(t) = S_{0} exp(rt) + (k/r) (exp(rt) - 1)
                    ", xs1, ys1, 1001);
                ImPlot::EndPlot();
        }
}
...
...

void ShowFirstOrderDEWindow(bool* p_open) {
...

if (ImGui::BeginTabBar("ImPlotDemoTabs")) {
if (ImGui::BeginTabItem("Plots")) {
       DemoHeader("Compound Interest", Demo_CompoundInterest);

...
...
void ImPlot::ShowFirstOrderDEWindow(bool* p_open) {}

#endif
```

Code 11: *Hamzstlab-Mathematics/hamzstlab_firstorderdifferentialequations.cpp*

Last, but not least, for the Makefile **Hamzstlab-Mathematics/examples/First Order Differential Equation/Makefile**, we need to add the SOURCES to be compiled that is pointing toward **Hamzstlab-Mathematics/hamzstlab_firstorderdifferentialequations.cpp**

```
EXE = main
IMGUI_DIR = ../..
SOURCES = main.cpp
```

```
SOURCES += $(IMGUI_DIR)/imgui.cpp $(IMGUI_DIR)/imgui_demo.cpp $(IMGUI_DIR)/
    imgui_draw.cpp $(IMGUI_DIR)/imgui_tables.cpp $(IMGUI_DIR)/imgui_widgets
    .cpp
SOURCES += $(IMGUI_DIR)/backends/imgui_impl_glfw.cpp $(IMGUI_DIR)/backends/
    imgui_impl_opengl3.cpp
SOURCES += $(IMGUI_DIR)/implot.cpp $(IMGUI_DIR)/implot_items.cpp
SOURCES += $(IMGUI_DIR)/hamzstlab_firstorderdifferentialequations.cpp

OBJS = $(addsuffix .o, $(basename $(notdir $(SOURCES))))
UNAME_S := $(shell uname -s)
LINUX_GL_LIBS = -lGL -lglfw

CXXFLAGS = -std=c++11 -I$(IMGUI_DIR) -I$(IMGUI_DIR)/backends
CXXFLAGS += -g -Wall -Wformat
LIBS = ../../dependencies/glad.c -L../../dependencies/ -lapp -limgui -
    limnodes -limplot

...
...
```

Code 12: *Hamzstlab-Mathematics/examples/First Order Differential Equation/Makefile*

III. Example of Creating a 3D Interactive Plot with Hamzstlab-Mathematics in GFreya OS

First you have to already clone or download the repository, then open terminal at the main working directory **Hamzstlab-Mathematics** and type:

```
cd examples

cd 3D Minimization

make
./main
```

Code 13: *Create dynamic library*

This will be the result:

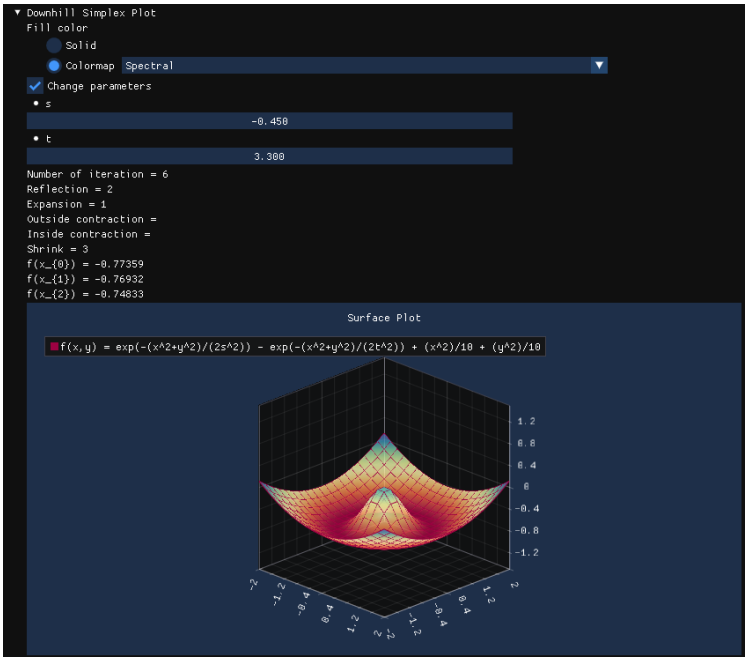

Figure 3.2: *The interactive plot of $f(x, y)$ with Hamzstlab-Mathematics you can change the parameters of s and t and the plot will adjust with the parameters that you just input in 3-dimension.*

Now, we are going to explain the code on how to make this example works.

```
#define IMGUI_DEFINE_MATH_OPERATORS // Access to math operators
#include <math.h>
#include "imgui_internal.h"
//————————————————————————————————————————————————————————————

// ImPlot3D Example for Minimization
// hamzstlab_minimization.cpp
```

```
// Date: 2025-08-11
//------------------------------------------------------------------

#include "imgui.h"
#include "imgui_impl_glfw.h"
#include "imgui_impl_opengl3.h"
#include "implot.h"
#include "implot3d.h"
#include <GLFW/glfw3.h>
#include <iostream>

// Callback to handle GLFW errors
void glfw_error_callback(int error, const char* description) { std::cerr <<
     "GLFW Error " << error << ": " << description << std::endl; }

int main() {
       // Setup error callback
       glfwSetErrorCallback(glfw_error_callback);

       // Initialize GLFW
       if (!glfwInit()) {
              std::cerr << "Failed to initialize GLFW" << std::endl;
              return -1;
       }

       // Setup OpenGL version
       #if defined(__APPLE__)
       // GL 3.2 + GLSL 150 (MacOS)
       const char* glsl_version = "#version 150";
       glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
       glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 2);
       glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE); //
           3.2+ only
       glfwWindowHint(GLFW_OPENGL_FORWARD_COMPAT, GL_TRUE); // Required on
           MacOS
       #else
       // GL 3.0 + GLSL 130 (Windows and Linux)
       const char* glsl_version = "#version 130";
       glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
       glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
       #endif

       // Create window
       GLFWwindow* window = glfwCreateWindow(1920, 1080, "Hamzstlab
           Mathematics", nullptr, nullptr);
       if (!window) {
              std::cerr << "Failed to create GLFW window" << std::endl;
              glfwTerminate();
```

```
        return -1;
    }
    glfwMakeContextCurrent(window);
    glfwSwapInterval(0); // Disable vsync

    // Setup context
    IMGUI_CHECKVERSION();
    ImGui::CreateContext();
    ImPlot::CreateContext();
    ImPlot3D::CreateContext();

    // Setup style
    ImGui::StyleColorsDark();

    // Setup backend
    ImGui_ImplGlfw_InitForOpenGL(window, true);
    ImGui_ImplOpenGL3_Init(glsl_version);

    // Main loop
    while (!glfwWindowShouldClose(window)) {
        glfwPollEvents();

        // Start frame
        ImGui_ImplOpenGL3_NewFrame();
        ImGui_ImplGlfw_NewFrame();
        ImGui::NewFrame();

        // Demo windows
        ImPlot3D::ShowMinimizationWindow();

        // Render
        ImGui::Render();
        int display_w, display_h;
        glfwGetFramebufferSize(window, &display_w, &display_h);
        glViewport(0, 0, display_w, display_h);
        glClearColor(0.1f, 0.1f, 0.1f, 1.0f);
        glClear(GL_COLOR_BUFFER_BIT);
        ImGui_ImplOpenGL3_RenderDrawData(ImGui::GetDrawData());

        // Swap buffers
        glfwSwapBuffers(window);
    }

    // Cleanup
    ImGui_ImplOpenGL3_Shutdown();
    ImGui_ImplGlfw_Shutdown();
    ImPlot3D::DestroyContext();
    ImPlot::DestroyContext();
```

```
        ImGui::DestroyContext();
        glfwDestroyWindow(window);
        glfwTerminate();

        return 0;
    }
```

Code 14: *Hamzstlab-Mathematics/examples/3D Minimization/main.cpp*

A lot of differences with the 2D interactive plot' **main.cpp**, in here we initialize **GLFW** then we call the function **ImPlot3D::ShowMinimizationWindow()** so it will call the demo of 3D minimization problem and plot that we have prepared.

This **main.cpp** is pointing toward
Hamzstlab-Mathematics/hamzstlab_minimization.cpp, this is the file that we have prepared that will be shown when we compile and run this demo so if you want to modify the example, you just need to edit
Hamzstlab-Mathematics/hamzstlab_minimization.cpp.

The **ImPlot3D::ShowMinimizationWindow();** needs to be an **IMPLOT3D_API**, and this can be done in **Hamzstlab-Mathematics/implot3d.h** under the **[SECTION] Demo**, so you need to add a line like this there

```
    ...

    IMPLOT3D_API void ShowMinimizationWindow(bool* p_open = nullptr);

    ...
```

Code 15: *Hamzstlab-Mathematics/implot3d.h*

For example, from hundred lines of codes you can check the code here to create a plot of a surface plot in 3-dimension for minimization problem with downhill simplex algorithm, we are showing this **Demo_DownhillSimplexPlot()** under a new tab that can be opened by the user.

```
    ...
    ...

    void DemoDownhillSimplexPlot() {

    constexpr int N = 20;
    static float xs[N * N], ys[N * N], zs[N * N];

    // Define the range for X and Y
    constexpr float min_val = −2.0f;
    constexpr float max_val = 2.0f;
    constexpr float step = (max_val − min_val) / (N − 1);
    static float s = 0.75;
    static float t = 1;

    ...
```

```
// Plot the surface
ImPlot3D::PlotSurface("f(x,y) = exp(—(x^2+y^2)/(2s^2)) — exp(—(x^2+y^2)
    /(2t^2)) + (x^2)/10 + (y^2)/10", xs, ys, zs, N, N);

// End the plot
ImPlot3D::PopStyleVar();
ImPlot3D::EndPlot();
}
if (selected_fill == 1)
{
ImPlot3D::PopColormap();
}

...
...

void ShowAllDemos() {
       ImGui::Text("with ImPlot3D (%s)", IMPLOT3D_VERSION);
       ImGui::Spacing();
       if (ImGui::BeginTabBar("Minimization")) {
              if (ImGui::BeginTabItem("Plots")) {
                     DemoHeader("Downhill Simplex Plot",
                         DemoDownhillSimplexPlot);
                     DemoHeader("Surface Plots", DemoSurfacePlots);
                     ImGui::EndTabItem();
              }
              if (ImGui::BeginTabItem("Custom")) {
                     DemoHeader("Custom Styles", DemoCustomStyles);
                     DemoHeader("Custom Rendering", DemoCustomRendering);
                     ImGui::EndTabItem();
              }
              if (ImGui::BeginTabItem("Help")) {
                     DemoHelp();
                     ImGui::EndTabItem();
              }
              ImGui::EndTabBar();
       }
}

void ShowMinimizationWindow(bool* p_open) {
       static bool show_implot3d_style_editor = false;
       static bool show_imgui_metrics = false;
       static bool show_imgui_style_editor = false;
       static bool show_imgui_demo = false;
...
...
```

Code 16: *Hamzstlab-Mathematics/hamzstlab_minimization.cpp*

Last, but not least, for the Makefile **Hamzstlab-Mathematics/examples/3D Minimization/-Makefile**, we need to add the SOURCES to be compiled that is pointing toward **Hamzstlab-Mathematics/hamzstlab_minimization.cpp**, then don't forget to add **-lsymintegration** and **-larmadillo** to link to the corresponding libraries that will be responsible for the symbolic computation and vector and matrix related computation.

```
EXE = main
IMGUI_DIR = ../..
SOURCES = main.cpp
SOURCES += $(IMGUI_DIR)/imgui.cpp $(IMGUI_DIR)/imgui_demo.cpp $(IMGUI_DIR)/
    imgui_draw.cpp $(IMGUI_DIR)/imgui_tables.cpp $(IMGUI_DIR)/imgui_widgets.cpp
SOURCES += $(IMGUI_DIR)/backends/imgui_impl_glfw.cpp $(IMGUI_DIR)/backends/
    imgui_impl_opengl3.cpp
SOURCES += $(IMGUI_DIR)/implot.cpp $(IMGUI_DIR)/implot_items.cpp
SOURCES += $(IMGUI_DIR)/implot3d.cpp $(IMGUI_DIR)/implot3d_items.cpp $(IMGUI_DIR)/
    implot3d_meshes.cpp
SOURCES += $(IMGUI_DIR)/hamzstlab_minimization.cpp

OBJS = $(addsuffix .o, $(basename $(notdir $(SOURCES))))
UNAME_S := $(shell uname -s)
LINUX_GL_LIBS = -lGL -lglfw

CXXFLAGS = -std=c++11 -I$(IMGUI_DIR) -I$(IMGUI_DIR)/backends
CXXFLAGS += -g -Wall -Wformat
LIBS = -lsymintegration -larmadillo ../../dependencies/glad.c -L../../
    dependencies/ -lapp -limgui -limnodes -limplot
```

Code 17: *Hamzstlab-Mathematics/examples/3D Minimization/Makefile*

Chapter 4

Intermediate Calculus

"Ma n'atu sole, che bello oje ne. O Sole Mio, sta fronte a te" - ***O Sole Mio***

We are going to learn how to plot, simulate and compute with either SymIntegration, Hamzstplot, or Hamzstlab Mathematics for this chapter, we are following the Calculus book [14] from chapter 9 to 14 for the theorems, problems and definition. The chapter 1-8 are covered in Lasthrim Projection book [11] and that book is still using Python and JULIA.

I. Infinite Series

II. Conics and Polar Coordinates

[H*] The first
[H*]

- The Method of Variation of Parameters

[H*] The first
[H*]

III. Geometry in Space and Vectors

- Vector-Valued Functions and Curvilinear Motion

[H*] The first
[H*]

- Surfaces in Three-Space

[H*] The first
[H*]

IV. Derivatives for Functions of Two or More Variables

- Functions of Two or More Variables

[H*] The first
[H*]

- Partial Derivatives

[H*] The first
[H*]

V. Multiple Integrals

- Double Integrals over Rectangles

[H*] The first
[H*]

- Iterated Integrals

[H*] The first
[H*]

VI. Vector Calculus

- Vector Fields

[H*] Previously, we use functions where the input is a real number and the output is a real number. Vector Calculus talks about vector-valued functions, that is, functions whose input is a real number and whose output is a vector.
Consider a function $\boldsymbol{F}$ that associates with each point $\boldsymbol{p}$ in n-space a vector $\boldsymbol{F}(p)$. A typical example in two-space is

$$\boldsymbol{F}(\boldsymbol{p}) = \boldsymbol{F}(x,y) = -\frac{1}{2}y\boldsymbol{i} + \frac{1}{2}x\boldsymbol{j}$$

For historical reasons, we refer to such function as a vector field. Imagine that to each point $\boldsymbol{p}$ in a region of space is attached a vector $\boldsymbol{F}(\boldsymbol{p})$ emanating from $\boldsymbol{p}$.

[H*] Vector fields that arise naturally in science are electric fields, magnetic fields, force fields, and gravitational fields. We consider only the case in which these fields are independent of time, which we call steady vector fields.

In contrast to a vector field, a function F that attaches a number to each point in space is called a scalar field. The function that gives the temperature at each point would be a good physical example of a scalar field.

[H*] **The Gradient of a Scalar Field**
Let $f(x,y,z)$ determine a scalar field and suppose that f is differentiable. then the gradient of f, denoted by ∇f, is the vector field given by

$$\boldsymbol{F}(x,y,z) = \nabla f(x,y,z) = \frac{\partial f}{\partial x}\boldsymbol{i} + \frac{\partial f}{\partial y}\boldsymbol{j} + \frac{\partial f}{\partial z}\boldsymbol{k} \tag{4.1}$$

We learned that $\nabla f(x,y,z)$ points in the direction of greatest increase of $f(x,y,z)$. A vector field $\boldsymbol{F}$ that is the gradient of a scalar field f is called a conservative vector field, and f is its potential function.

Such fields and their potential functions are important in physics. In particular, fields that obey the inverse square law (e.g., electric fields and gravitational fields) are conservative.

[H*] **The Divergence and Curl of a Vector Field**
With a given vector field

$$\boldsymbol{F} = M(x,y,z)\boldsymbol{i} + N(x,y,z)\boldsymbol{j} + P(x,y,z)\boldsymbol{k} \tag{4.2}$$

are associated to two other important fields. the first is called the divergence of $\boldsymbol{F}$ (div $\boldsymbol{F}$), which is a scalar field; the second is called the curl of $\boldsymbol{F}$ (curl $\boldsymbol{F}$), which is a vector field.

Definition 4.1: div and curl

Let $\boldsymbol{F} = M\boldsymbol{i} + N\boldsymbol{j} + P\boldsymbol{k}$ be a vector field for which the first partial derivatives of M, N, and P exist. Then

$$\operatorname{div} \boldsymbol{F} = \frac{\partial M}{\partial x} + \frac{\partial N}{\partial y} + \frac{\partial P}{\partial z} \tag{4.3}$$

$$\operatorname{curl} \boldsymbol{F} = \left(\frac{\partial P}{\partial y} - \frac{\partial N}{\partial z}\right)\boldsymbol{i} + \left(\frac{\partial M}{\partial z} - \frac{\partial P}{\partial x}\right)\boldsymbol{j} + \left(\frac{\partial N}{\partial x} - \frac{\partial M}{\partial y}\right)\boldsymbol{k} \tag{4.4}$$

- Line Integrals

  [H*] The first
  [H*]

- Independence of Path

  [H*] The first
  [H*]

- Green's Theorem in the Plane

  [H*] The first
  [H*]

- Surface Integrals

  [H*] The first
  [H*]

- Gauss's Divergence Theorem

  [H*] The first
  [H*]

- Stokes's Theorem

  [H*] The first
  [H*]

Chapter 5

Differential Equations

"Everyone's favorite fighting fairy godmother is still kicking, taking out Ether Strike hits for her godfather Odin. The heretofore unmatched fury of her scowl derives from her lack of lines in the main story" - ***Freya (Valkyrie Profile: Covenant of the Plume)***

Dıfferential equations are equations that contain derivatives. It is used to understand and to investigate problems involving the motion of fluids, the flow of current in electric circuits, the dissipation of heat in solid objects, the propagation and detection of seismic waves [2]. For example, the Newton's second law can be written as a differential equation

$$F = ma \tag{5.1}$$

$$F = m\frac{dv}{dt} = mv' \tag{5.2}$$

We think of v as a function of t, or we can also write it as $v(t)$.

For this chapter, we will follow the outline of [2] book and we start it on May 2025, we will take the measure how long it takes to finish this chapter that is equal to the book mentioned.

I. Direction Fields

[H*] Direction fields are valuable tools in studying the solutions of differential equations of the form

$$\frac{dy}{dt} = f(t, y) \tag{5.3}$$

where f is a given function of two variables t and y, sometimes referred to as the rate function.
A direction field (slope field) is a mathematical object used to graphically represent solutions to a first-order differential equation. At each point in a direction field, a line segment appears whose slope is equal to the slope of a solution to the differential equation passing through that point.

A direction field can be constructed by evaluating f at each point of a rectangular grid. At each point of the grid, a short line segment is drawn whose slope is the value of f at that point. Thus each line segment is tangent to the graph of the solution passing through that point.

[H*] A direction field drawn on a fairly fine grid gives good picture of the overall behavior of solutions of a differential equation.

The construction of a direction field is often a useful step in the investigation of a differential equation.

[H*] **Example:**
Consider a population of field mice who inhabit a certain rural area. If we denote time by t and the mouse population by $p(t)$ with the time is measured in months, then the differential that represents the population growth of field mice that is also preyed on by owls can be expressed by the equation

$$\frac{dp}{dt} = 0.5p - 450$$

Observe that the predation term is measured in months as -450, meaning that each month there is about 450 mouse being preyed by owls every month.

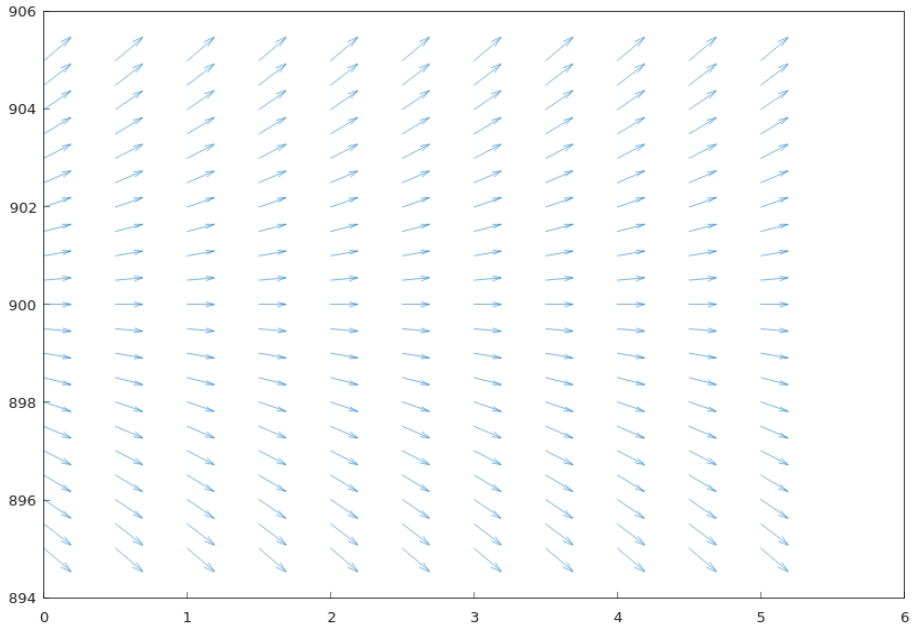

Figure 5.1: *The direction field for $\frac{dp}{dt} = 0.5p - 450$, the x-axis represents the variable t as the time and the y-axis represents variable $p(t)$ as the population growth of field mice, we can see that if the rate of the reproduction for the field mice is greater than the rate of the mice being preyed on by owls then the population growth will keep increasing, and if the rate of the reproduction for the fiel mice is smaller than the rate of the mice being preyed on by owls then the population of field mice will keep decreasing.*

The code to create the direction field above with Hamzstplot:

```
#include <cmath>
#include <hamzstplot/hamzstplot.h>

using namespace std;
```

```
int main() {
       using namespace hamzstplot;
       double dx, dy, magnitude;
       auto [x, y] = meshgrid(iota(0, 0.5, 5), iota(895, 0.5,
           905));

       vector_2d u = transform(x, y, [](double x, double y) {
           return 1 ; }); // dx
       vector_2d v = transform(x, y, [](double x, double y) {
           return 0.5*y - 450 ; }); // dy
       quiver(x, y, u, v);

       show();
       return 0;
}
```

Code 18: *Hamzstplot/Examples with Makefile/Plot Direction Fields Growth and Predation Rate/main.cpp*

For sufficiently large values of p it can be seen that $\frac{dp}{dt}$ is positive, so that solutions increase. On the other hand, if p is small, then $\frac{dp}{dt}$ is negative and solutions decrease.

The critical value of p that separates solutions that increase from those that decrease is the value of p for which $\frac{dp}{dt}$ is zero.

By setting $\frac{dp}{dt}$ equals zero and then solving for p, we find the equilibrium solution $p(t) = 900$ for which the growth term and the predation term are exactly balanced.

II. Solving a Simple Differential Equation

[H*] Consider we have this differential equation

$$\frac{dp}{dt} = 0.5p - 450$$

If we want to obtain the solution $p(t)$ we will have to perform some algebraic trick or manipulation then integrating them with respect to the correct variable.

$$\begin{aligned}
\frac{dp}{dt} &= 0.5p - 450 \\
&= \frac{p - 900}{2} \\
\frac{dp/dt}{p - 900} &= \frac{1}{2} \\
\frac{dp}{p - 900} &= \frac{dt}{2} \\
\int \frac{dp}{p - 900} &= \int \frac{dt}{2} \\
\ln|p - 900| &= \frac{t}{2} + C \\
|p - 900| &= e^{\frac{t}{2}+C} \\
|p - 900| &= e^{\frac{t}{2}}e^{C} \\
p - 900 &= \pm e^{\frac{t}{2}}e^{C} \\
p &= 900 + ce^{\frac{t}{2}}
\end{aligned}$$

where $c = \pm e^C$ is also an arbitrary (nonzero) constant. Note that the constant function $p = 900$ is also a solution where $c = 0$.

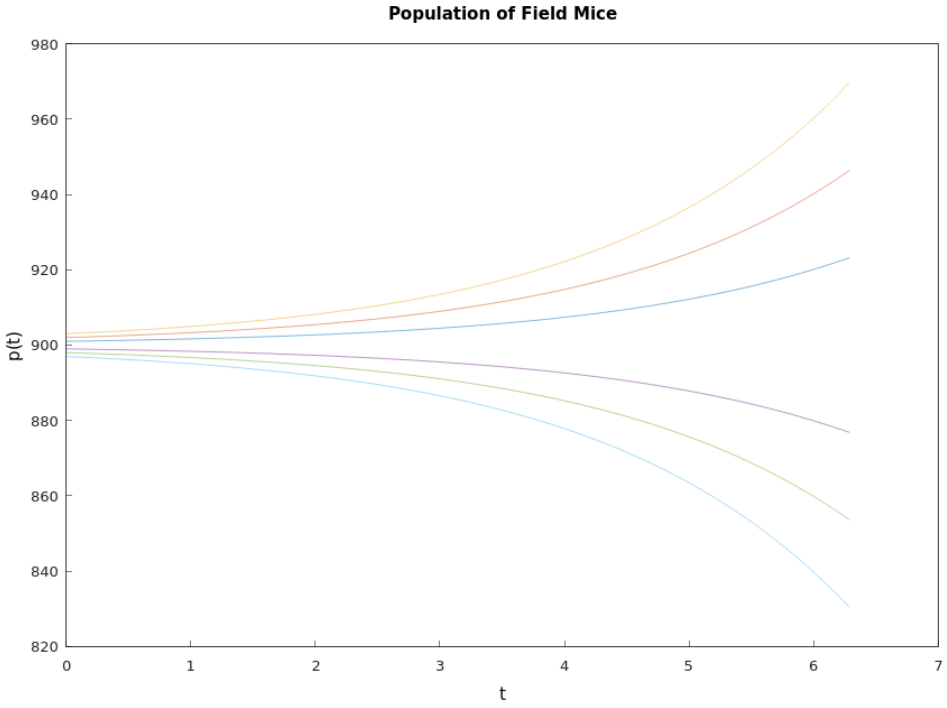

Figure 5.2: *The graphs shows the solutions for* $\frac{dp}{dt} = 0.5p - 450$ *with several valus of* c.

```
#include <cmath>
#include <hamzstplot/hamzstplot.h>

using namespace std;

int main() {
       using namespace hamzstplot;

       std::vector<double> t = linspace(0, 2 * pi);
       std::vector<double> y1 = transform(t, [](auto t) {
           return 900 + 1*exp(0.5*t); });
       std::vector<double> y2 = transform(t, [](auto t) {
           return 900 + 2*exp(0.5*t); });
       std::vector<double> y3 = transform(t, [](auto t) {
           return 900 + 3*exp(0.5*t); });
       std::vector<double> y4 = transform(t, [](auto t) {
           return 900 + -1*exp(0.5*t); });
       std::vector<double> y5 = transform(t, [](auto t) {
           return 900 + -2*exp(0.5*t); });
       std::vector<double> y6 = transform(t, [](auto t) {
           return 900 + -3*exp(0.5*t); });

       plot(t, y1, t, y2, t, y3, t, y4, "-", t, y5,"-", t,y6,
           "-");
       title("Population of Field Mice");
       xlabel("t");
       ylabel("p(t)");
       show();
       return 0;
}
```

Code 19: *Hamzstplot/Examples with Makefile/Plot First Order Differential Equation Solutions Curves/main.cpp*

III. Classification of Differential Equations

[H*] One important classification is based on whether the unknown function depends on a single independent variable or on several independent variables.

If a function depends on a single independent variable then it is said to be an ordinary differential equation, this is the example:

$$L\frac{d^2Q(t)}{dt^2} + R\frac{dQ(t)}{dt} + \frac{1}{C}Q(t) = E(t) \tag{5.4}$$

with $Q(t)$ represents the charge on a capacitor in a circuit with capacitance C, resistance R, and inductance L.

If a function depends on mroe than one independent variable then it is called a partial differential equation, this is the example:

$$\alpha^2 \frac{\partial^2 u(x,t)}{\partial x^2} = \frac{\partial u(x,t)}{\partial t} \tag{5.5}$$

it is called the heat conduction equation with $alpha$ as the physical constant. Another example is:

$$\alpha^2 \frac{\partial^2 u(x,t)}{\partial x^2} = \frac{\partial^2 u(x,t)}{\partial t^2} \tag{5.6}$$

it is called the wave equation with α as the physical constant.

[H*] System of differential equations usually have two or more unknown functions. For example, the Lotka-Volterra equations:

$$\begin{aligned} \frac{dx}{dt} &= ax - \alpha xy \\ \frac{dy}{dt} &= -cy + \gamma xy \end{aligned} \tag{5.7}$$

where $x(t)$ and $y(t)$ are the repsective populations of the prey and predator species. The constants a, α, c, and γ are based on empirical observations and depend on the particular species being studied.

[H*] The order of a differential equation is the order of the highest derivative that appears in the equation.

More genrally, the equation

$$F[t, u(t), u'(t), \ldots, u^{(n)}(t)] = 0 \tag{5.8}$$

is an ordinary differential equation of the nth order.

Another one,

$$y''' + 2e^t y'' + yy' = t^3$$

is a third order differential equation for $y = u(t)$.

[H*] The ordinary differential equation

$$F(t, y, y', \ldots, y^{(n)}) = 0$$

is said to be linear if F is a linear function of the variables $y, y', \ldots, y^{(n)}$; a similar definition applies to partial differential equations.

Thus, the general linear ordinary differential equation of order n is

$$a_0(t)y^{(n)} + a_1(t)y^{(n-1)} + \cdots + a_n(t)y = g(t) \tag{5.9}$$

A simple physical problem that leads to a nonlinear differential equation is the oscillating pendulum. The angle θ that an oscillating pendulum of length L makes with the vertical direction satisfies the equation

$$\frac{d^2\theta}{dt^2} + \frac{g}{L} \sin\ \theta = 0 \tag{5.10}$$

the term involving sin θ makes that equation to be nonlinear.

IV. First Order Ordinary Differential Equations

- Linear Equations; Method of Integrating Factors

[H*] Suppose we have this differential equations of first order

$$\frac{dy}{dt} = f(t, y)$$

where f is a given function of two variables. Any differentiable function $y = \phi(t)$ that satisfies this equation for all t in some interval is called a solution, and our object is to determine whether such functions exist.

For an arbitrary function f, there is no general method for solving the equation in terms of elementary functions.

[H*] We define the general first order linear equation in the standard form

$$\frac{dy}{dt} + p(t)y = g(t) \tag{5.11}$$

where p and g are given functions of the independent variable t.

To determine an appropriate integrating factor, we multiply Eq. (5.11) by an undetermined function $\mu(t)$, obtaining

$$\mu(t)\frac{dy}{dt} + p(t)\mu(t)y = \mu(t)g(t) \tag{5.12}$$

We see the left side of the Eq. (5.12) is the derivative of the product $\mu(t)y$, provided that $\mu(t)$ satisfies the equation

$$\frac{d\mu(t)}{dt} = p(t)\mu(t) \tag{5.13}$$

If we assume temporarily that $\mu(t)$ is positive, then we have

$$\frac{d\mu(t)/dt}{\mu(t)} = p(t)$$
$$\ln|\mu(t)| = \int p(t)\ dt + k$$

By choosing the arbitrary constant k to be zero, we obtain the simplest possible function for μ, namely,

$$\mu(t) = e^{\int p(t)\ dt} \tag{5.14}$$

Note that $\mu(t)$ is positive for all t, as we assumed. Returning to Eq. (5.12), we have

$$\frac{d}{dt}[\mu(t)y] = \mu(t)g(t) \tag{5.15}$$

Hence

$$\mu(t)y = \int \mu(t)g(t)\ dt + c \tag{5.16}$$

where c is an arbitrary constant.

The general solution of Eq. (5.11) is

$$y = \frac{1}{\mu(t)} \left[\int_{t_0}^{t} \mu(s) g(s)\, ds + c \right] \tag{5.17}$$

where t_0 is some convenient lower limit of integration.

[H*] **Example:**
Solve the initial value problem

$$\begin{aligned} ty' + 2y &= 4t^2 \\ y(1) &= 2 \end{aligned}$$

Solution:
In order to determine $p(t)$ and $g(t)$ correctly, we must first rewrite the equation in the standard form of Eq. (5.11). Thus we have

$$y' + (2/t)y = 4t \tag{5.18}$$

so

$$\begin{aligned} p(t) &= \frac{2}{t} \\ g(t) &= 4t \end{aligned}$$

Now, we first compute the integrating factor $\mu(t)$:

$$\begin{aligned} \mu(t) &= e^{\int \frac{2}{t}\, dt} \\ &= e^{2 \ln |t|} \\ &= t^2 \end{aligned}$$

Following the formula of Eq. (5.17), we obtain

$$\begin{aligned} y &= \frac{1}{\mu(t)} \left[\int_{t_0}^{t} \mu(s) g(s)\, ds + c \right] \\ &= \frac{1}{t^2} \left[\int_{0}^{t} s^2 (4s)\, ds \right] \\ &= \frac{1}{t^2} \left[\int_{0}^{t} 4s^3\, ds \right] \\ &= \frac{1}{t^2} \left[t^4 + c \right] \\ y &= t^2 + \frac{c}{t^2} \end{aligned}$$

$y(t) = t^2 + \frac{c}{t^2}$ is the general solution of $ty' + 2y = 4t^2$.

To obtain the value for c, we will input the the initial condition value $y(1) = 2$ to the general solution

$$\begin{aligned} y(1) &= 2 \\ 2 &= 1^2 + \frac{c}{1^2} \\ c &= 1 \end{aligned}$$

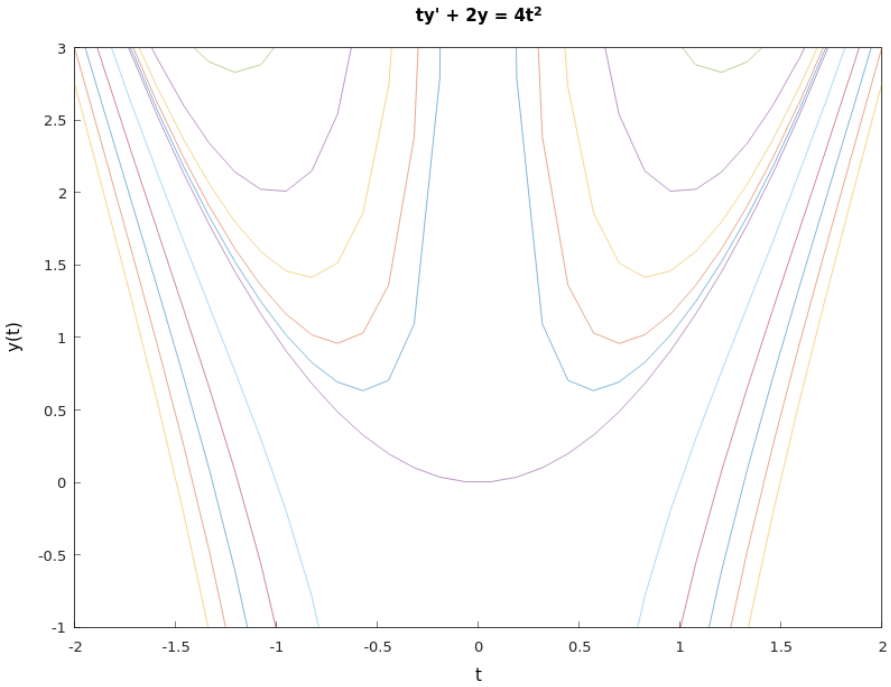

Figure 5.3: *The graphs shows the solutions for* $ty' + 2y = 4t^2$.

The solution becomes unbounded and is asymptotic to the positive y-axis as $t \to 0$ from the right. This is the effect of the infinite discontinuity in the coefficient $p(t)$ at the origin.

The function $y = t^2 + \frac{1}{t^2}$ for $t < 0$ is not part of the solution of this initial value problem. The solution fails to exist for some values of t. This is due to the infinite discontinuity in $p(t)$ at $t = 0$, which restricts the solution to the interval $0 < t < \infty$.

```
#include <cmath>
#include <hamzstplot/hamzstplot.h>

using namespace std;

int main() {
      using namespace hamzstplot;

      std::vector<double> t = linspace(—2 * pi, 2 * pi);
      std::vector<double> y1 = transform(t, [](auto t) { return
          pow(t,2) + 0.1/(pow(t,2)); });
      std::vector<double> y2 = transform(t, [](auto t) { return
          pow(t,2) + 0.23/(pow(t,2)); });
      std::vector<double> y3 = transform(t, [](auto t) { return
          pow(t,2) + 0.5/(pow(t,2)); });
      std::vector<double> y4 = transform(t, [](auto t) { return
          pow(t,2) + 1/(pow(t,2)); });
      std::vector<double> y5 = transform(t, [](auto t) { return
```

```
            pow(t,2) + 2/(pow(t,2)); });
        std::vector<double> y6 = transform(t, [](auto t) { return
            pow(t,2) + -1/(pow(t,2)); });
        std::vector<double> y7 = transform(t, [](auto t) { return
            pow(t,2) + -2/(pow(t,2)); });
        std::vector<double> y8 = transform(t, [](auto t) { return
            pow(t,2) + -3/(pow(t,2)); });
        std::vector<double> y9 = transform(t, [](auto t) { return
            pow(t,2) + -4/(pow(t,2)); });
        std::vector<double> y10 = transform(t, [](auto t) { return
            pow(t,2) + -5/(pow(t,2)); });
        std::vector<double> y11 = transform(t, [](auto t) { return
            pow(t,2); });

        plot(t, y1, t, y2, t, y3, t, y4, "-", t, y5,"-", t,y6, "-
            ", t,y7, "-", t,y8, "-", t,y9, "-", t,y10, "-", t,
            y11, "-");
        axis({-2, 2, -1, 3});
        title("ty' + 2y = 4t^2");
        xlabel("t");
        ylabel("y(t)");
        show();
        return 0;
    }
```

Code 20: *Hamzstplot/Examples with Makefile/Plot First Order Differential Equation Solutions Curves for Method of Integrating Factors/main.cpp*

- Separable Equations

[H*] The general first order equation is

$$\frac{dy}{dx} = f(x,y) \tag{5.19}$$

By setting $M(x,y) = -f(x,y)$ and $N(x,y) = 1$ we will have

$$M(x,y) + N(x,y)\frac{dy}{dx} = 0 \tag{5.20}$$

If it happens that M is a function of x only and N is a function of y only, then it becomes

$$M(x) + N(y)\frac{dy}{dx} = 0 \tag{5.21}$$

Such an equation is said to be separable, because if it is written in the differential form

$$M(x)\,dx + N(y)\,dy = 0 \tag{5.22}$$

The differential form (5.22) is also more symmetric and tends to suppress the distinction between independent and dependent variables.

A separable equation can be solved by integrating the functions M and N.

[H*] **Example:**
Show that the equation

$$\frac{dy}{dx} = \frac{x^2}{1-y^2}$$

is separable, and then find an equation for its integral curves.

Solution:

If we write the equation as

$$-x^2 + (1-y^2)\frac{dy}{dx} = 0$$

then it is separable. Recall from calculus that if y is a function of x, then by the chain rule

$$\frac{d}{dx}f(y) = \frac{d}{dy}f(y)\frac{dy}{dx} = f'(y)\frac{dy}{dx}$$

If $f(y) = y - \frac{y^3}{3}$, then

$$\frac{d}{dx}\left(y - \frac{y^3}{3}\right) = (1-y^2)\frac{dy}{dx}$$

Thus,

$$\begin{aligned} -x^2 + (1-y^2)\frac{dy}{dx} &= 0 \\ (1-y^2)dy &= x^2\,dx \\ y - \frac{y^3}{3} &= \frac{x^3}{3} + c \\ -x^3 + 3y - y^3 &= c \end{aligned}$$

Any differentiable function $y = \phi(x)$ that satisfies $-x^3 + 3y - y^3 = c$ is a solution of $\frac{dy}{dx} = \frac{x^2}{1-y^2}$.

An equation of the integral curve passing through a particular point (x_0, y_0) can be found by substituting x_0 and y_0 for x and y, respectively, in $-x^3 + 3y - y^3 = c$ and determining the corresponding value of c.

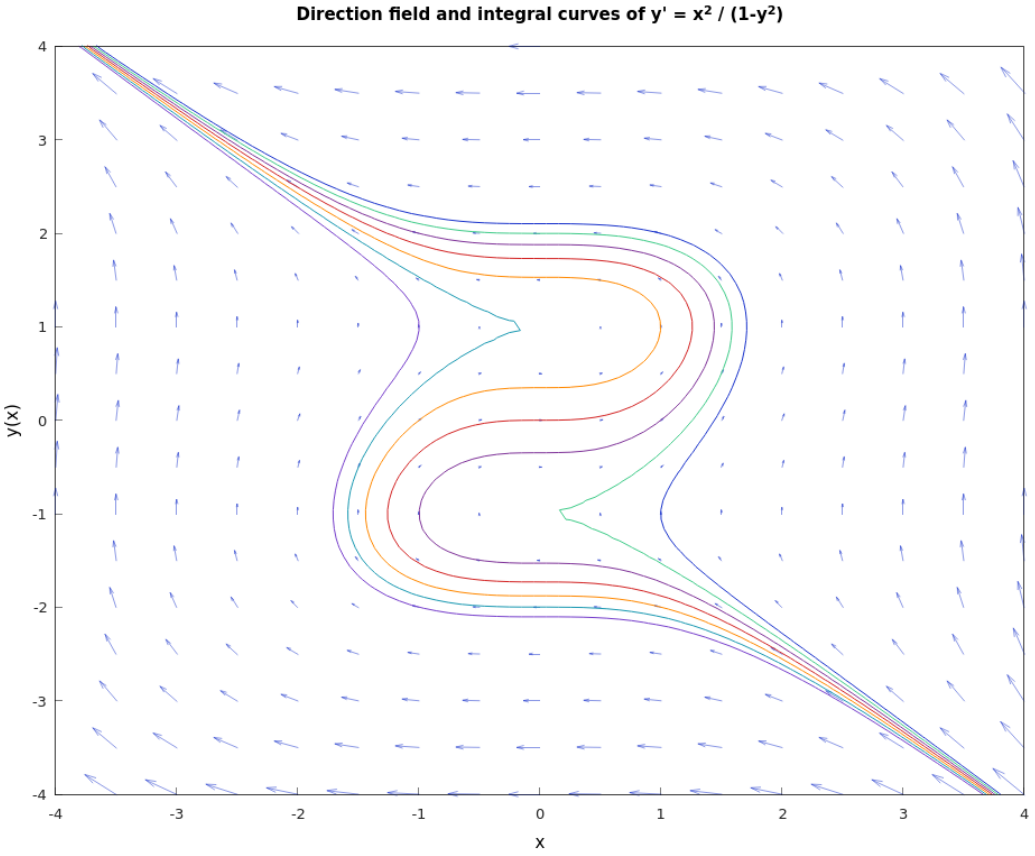

Figure 5.4: *The graphs shows the direction field and integral curves of* $y' = \frac{x^2}{1-y^2}$.

```
#include <cmath>
#include <hamzstplot/hamzstplot.h>

using namespace std;

int main() {
	using namespace hamzstplot;

	vector_2d newcolors = {{0.83, 0.14, 0.14},
		{1.00, 0.54, 0.00},
		{0.10, 0.6, 0.70},
		{0.47, 0.25, 0.80},
		{0.50, 0.2, 0.60},
		{0.25, 0.80, 0.54}};
	colororder(newcolors);

	auto [x, y] = meshgrid(iota(-4, 0.5, 5), iota(-4, 0.5, 4))
	    ;
	vector_2d u = transform(x, y, [](double x, double y) {
```

```
            return 1-pow(y,2) ; }); // dx
        vector_2d v = transform(x, y, [](double x, double y) {
            return pow(x,2) ; }); // dy

        quiver(x, y, u, v);
        hold(on);
        fimplicit([](double x, double y) { return -pow(x, 3) + 3*y
            - pow(y, 3) ; },"-")->line_width(1);
        fimplicit([](double x, double y) { return -pow(x, 3) + 3*y
            - pow(y, 3) - 1; },"-")->line_width(1);
        fimplicit([](double x, double y) { return -pow(x, 3) + 3*y
            - pow(y, 3) - 2; },"-")->line_width(1);
        fimplicit([](double x, double y) { return -pow(x, 3) + 3*y
            - pow(y, 3) - 3; },"-")->line_width(1);
        fimplicit([](double x, double y) { return -pow(x, 3) + 3*y
            - pow(y, 3) + 1; },"-")->line_width(1);
        fimplicit([](double x, double y) { return -pow(x, 3) + 3*y
            - pow(y, 3) + 2; },"-")->line_width(1);
        fimplicit([](double x, double y) { return -pow(x, 3) + 3*y
            - pow(y, 3) + 3; },"-")->line_width(1);
        hold(off);

        axis({-4, 4, -4, 4});
        title("Integral curves of y' = x^{2} / (1-y^{2})");
        xlabel("x");
        ylabel("y(x)");
        show();
        return 0;
    }
```

Code 21: *Hamzstplot/Examples with Makefile/Plot First Order Differential Equation Direction Field and Solutions Curves for Separable Equations/main.cpp*

The coding here is using **fimplicit** since we are inputting $-x^3 + 3y - y^3 - c = 0$ as the function to be drawn, it is the implicit function. We also draw the direction field so we are using **hold(on)** then closing it with **hold(off)**.

- Modeling with First Order Equations

[H*] Differential equations are of interest to nonmathematicians primarily because of the possibility of using them to investigate a wide variety of problems in the physical, biological, and social sciences.

One reason for this is that mathematical models and their solutions lead to equations relating the variables and parameters in the problem. These equations often enable you to make predictions about how the natural process will behave in various circumstances.

Mathematical modeling and experiment or observation are both critically important

and have somewhat complementary roles in scientific investigations. Mathematical models are validated by comparison of their predictions with experimental results.

[H*] Three identifiable steps that are always present in the process of mathematical modeling:

1. Construction of the Model.
2. Analysis of the Model.
3. Comparison with Experiment or Observation.

[H*] **Compound Interest**

Suppose that a sum of money is deposited in a bank that pays interest at an annual rate r. The value $S(t)$ of the investment at any time t depends on the frequency with which interest is compounded as well as on the interest rate.

Financial institutions have various policies concerning compounding: some compound monthly, some weekly, some even daily.

If we assume that compounding takes place continuously, then we can set up a simple initial value problem that describes the growth of the investment.

The rate of change of the value of the investment is $\frac{dS}{dt}$, and this quantity is equal to the rate at which interest accrues, which is the interest rate r times the current value of the investment $S(t)$. Thus

$$\frac{dS}{dt} = rS \tag{5.23}$$

is the differential equation that governs the process. Suppose that we also know the value of the investment at some particular time, say,

$$S(0) = S_0 \tag{5.24}$$

Then the solution of the initial value problem at Eq. (5.23) and Eq. (5.24) gives the balance $S(t)$ in the account at any time t. This initial value problem is readily solved, since the differential equation at Eq. (5.23) is both linear and separable. Consequently, by solving Eqs. (5.23) and (5.24), we find that

$$S(t) = S_0 e^{rt} \tag{5.25}$$

Thus a bank account with continuously compounding interest grows exponentially.

Let us suppose that there may be deposits or withdrawals in addition to the accrual of interest, dividends, or capital gains. If we assume that the deposits or withdrawals take place at a constant rate k, then Eq. (5.23) is replaced by

$$\frac{dS}{dt} = rS + k \tag{5.26}$$

where k is positive for deposits and negative for withdrawals.

Eq. (5.26) is linear with the integrating factor e^{-rt}, so its general solution is

$$S(t) = ce^{rt} - \frac{k}{r} \tag{5.27}$$

where c is an arbitrary constant. To satisfy the initial condition Eq. (5.24), we must choose $c = S_0 + \frac{k}{r}$. Thus the solution of the initial value problem is

$$S(t) = S_0 e^{rt} + \left(\frac{k}{r}\right)(e^{rt} - 1) \tag{5.28}$$

The first term in the expression (5.28) is the part of $S(t)$ that is due to the return accumulated on the initial amount S_0, and the second term is the part that is due to the deposit or withdrawal rate k.

The advantage of stating the problem in this general way without specific values for S_0, r, or k lies in the generality of the resulting formula (5.28) for $S(t)$. With this formula we can readily compare the results of different investment programs or different rates of return.

[H*] **Retirement Plan Example:**
Suppose that one person opens an individual retirement account (IRA) at age 25 and makes annual investments of USD 2000 thereafter in a continuous manner. Assuming a rate of return of 8%, what will be the balance in the IRA at age 65?

Solution:

We have

$$\begin{aligned} S_0 &= 0 \\ r &= 0.08 \\ k &= 2000 \end{aligned}$$

and we wish to determine $S(40)$. From Eq. (5.28) we have

$$\begin{aligned} S(40) &= (0)e^{(0.08)(40)} + \left(\frac{2000}{0.08}\right)(e^{(0.08)(40)} - 1) \\ &= (25,000)(e^{3.2} - 1) \\ &= 588,313 \end{aligned}$$

It is interesting to note that the total amount invested is USD 80,000 ($40 \times 2,000$), so the remaining amount of USD 508,313 results from the accumulated return on the investment. The balance after 40 years is also fairly sensitive to the assumed rate.

For the C++ code, we are using Hamzstlab Mathematics for the first time as the introduction, this is to complement Hamzstplot beautifully. Hamzstlab Mathematics is using ImGUI and Implot so it can make the interactive GUI like in this example.

We can of course plot $S(t)$ easily with various rate with Hamzstplot, but we want a slider that can change the plot at an instant, that is why we are using Hamzstlab Mathematics, it can do the job better and faster.

We create a few examples for the first order ordinary differential equations section for Hamzstlab Mathematics. The code to create this is longer, not only one CPP file, but more than one, and it is a lot different than Hamzstplot, a lot of learning, you have to

read a lot for the codes, read about ImGUI, Implot, steeper curve of learning, but it will worth your time since it can sustain not only decades but till another 100 years for the durability.

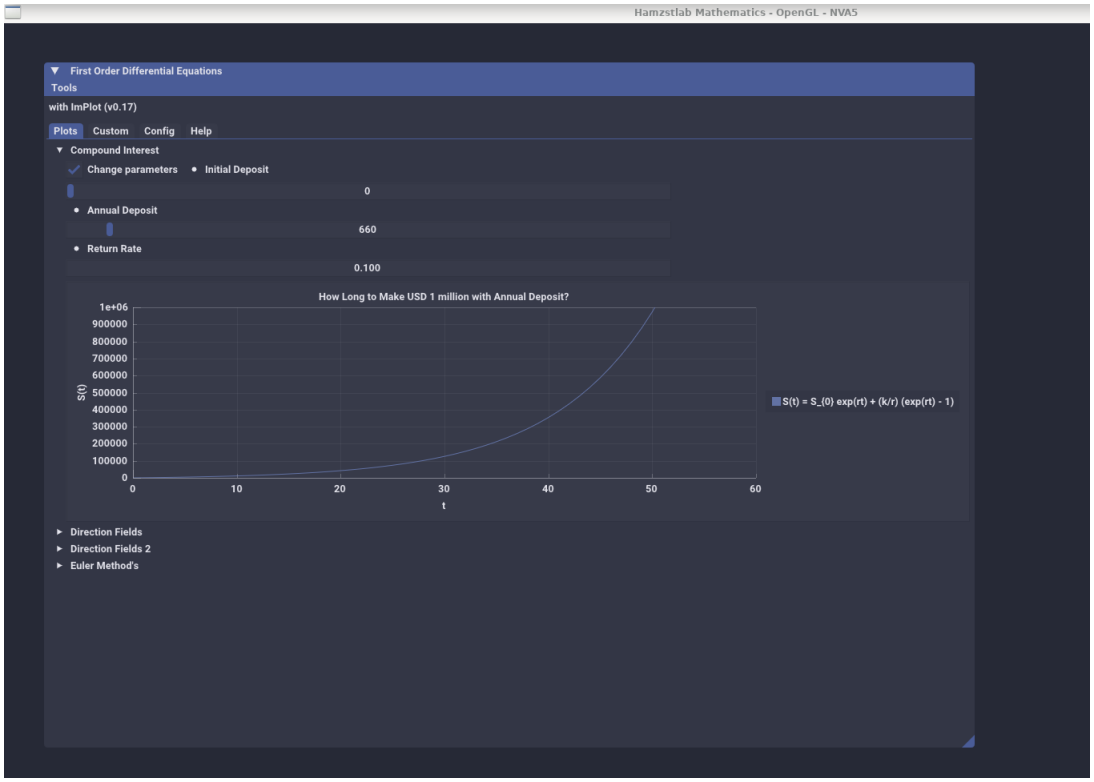

Figure 5.5: *The interactive plot of $S(t)$ with Hamzstlab-Mathematics you can change the parameters and the plot will adjust with the parameters that you just input (**Hamzstlab-Mathematics/examples/First Order Differential Equation/main.cpp**).*

```
...
...

void Demo_CompoundInterest() {
static float xs1[1001], ys1[1001];
static int S0 = 0;
static float r = 0.08;
static int k = 2000;
for (int i = 0; i < 1001; ++i) {
xs1[i] = i;
ys1[i] = S0 * exp(r*i) + (k/r)*(exp(r*i) - 1) ;
}

static bool range = false;
ImGui::Checkbox("Change parameters", &range);
```

```
if (range) {
ImGui::SameLine();
ImGui::SetNextItemWidth(200);
ImGui::BulletText("Initial Deposit");
ImGui::SliderInt("##Initial Deposit", &S0, 0, 10000);
ImGui::SetNextItemWidth(200);
ImGui::BulletText("Annual Deposit");
ImGui::SliderInt("##Annual Deposit", &k, 1, 10000);
ImGui::SetNextItemWidth(200);
ImGui::BulletText("Return Rate");
ImGui::DragFloat("##Return rate", &r, 0.01f, 0.2f);
ImGui::SetNextItemWidth(200);
}
if (ImPlot::BeginPlot("How Long to Make USD 1 million with Annual
    Deposit?")) {
ImPlot::SetupAxes("t","S(t)");
ImPlot::SetupAxesLimits(0, 60, 0, 1000000);
ImPlot::SetupLegend(ImPlotLocation_East, ImPlotLegendFlags_Outside
    );

ImPlot::PlotLine("S(t) = S_{0} exp(rt) + (k/r) (exp(rt) — 1)",
    xs1, ys1, 1001);
ImPlot::EndPlot();
}
}
...
...
```

Code 22: *Hamzstlab-Mathematics/hamzstlab_firstorderdifferentialequations.cpp*

We only include the snippet of code to plot the $S(t)$, the full code can be seen at Hamzstlab Mathematics from the repository of Hamzstlab Mathematics at the **Hamzstlab-Mathematics/hamzstlab_firstorderdifferentialequations.cpp**, to run the example with Makefile you can go to this directory **Hamzstlab-Mathematics/examples/First Order Differential Equation**.

[H*] **Mortgage Loan Example:**
A home buyer can afford to spend no more than USD 500 / month on mortgage payments. Suppose that the interest rate is 6%, that interest is compounded continuously, and that payments are also made continuously.

(a) Determine the maximum amount that this buyer can afford to borrow on a 20-year mortgage.

(b) Determine the total interest paid during the term of the mortgage.

Solution:

(a) The amount of money $S(t)$ that the buyer has to pay changes in time due to two factors, the compound interest and its' continuous payments. The rate of growth

for compounding is rS, and the rate of decay due to the continuous payments is k.

$$\frac{dS}{dt} = rS - k$$

$$\frac{dS}{dt} - rS = -k$$

This is a linear first-order inhomogeneous ODE, so it can be solved by multiplying both sides by an integrating factor $\mu(t)$

$$\mu(t) = e^{\int^t (-r)\ ds} = e^{-rt}$$

proceed with the multiplication and solve it for $S(t)$

$$\begin{aligned} e^{-rt}\frac{dS}{dt} - re^{-rt}S &= -ke^{-rt} \\ \frac{d}{dt}(e^{-rt}S) &= -ke^{-rt} \\ e^{-rt}S &= \frac{k}{r}e^{-rt} + C \\ S(t) &= \frac{k}{r} + Ce^{rt} \end{aligned}$$

Apply the initial condition $S(0) = S_0$ to determine C

$$\begin{aligned} S(0) &= \frac{k}{r} + Ce^{rt} \\ S_0 &= \frac{k}{r} + Ce^{rt} \\ C &= S_0 - \frac{k}{r} \end{aligned}$$

Therefore, the amount of money the buyer has to pay after t years is

$$\begin{aligned} S(t) &= \frac{k}{r} + \left(S_0 - \frac{k}{r}\right)e^{rt} \\ &= \frac{k}{r}(1 - e^{rt}) + S_0 e^{rt} \end{aligned}$$

For a year the home buyer will need to pay $k = 500 \times 12 = USD\ 6,000$ year and $r = 6\% = 0.06$.

$$S(t) = \frac{6000}{0.06}(1 - e^{0.06t}) + S_0 e^{0.06t}$$

For a 20-year mortgage, $S(t) = 0$ at $t = 20$.

$$\begin{aligned} 0 &= \frac{6000}{0.06}(1 - e^{0.06t}) + S_0 e^{0.06t} \\ S_0 &= 100000 - 100000e^{-1.2} \\ S_0 &\approx USD\ 69,880.58 \end{aligned}$$

The value of S_0 is how much the buyer can afford to borrow initially.

(b) For the 20-year mortgage, the home buyer pays a total of

$$USD\ 6,000 \times 20 = USD\ 120,000$$

so the total interest paid is

$$USD\ 120,000 - USD\ 69,880.58 = USD\ 50,119.42$$

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DSolve and IVP f
or Mortgage case ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DSolve and IVP f
or Mortgage case ]# ./main

DSolve for S' = rS - k

S(t) = k*r^(-1)+C*e^(t*r)
S(t=0) = k*r^(-1)+C
For ivp S(0) = S0,
C = -k*r^(-1)+S0

Test ivp = k*r^(-1)-k*r^(-1)*e^(t*r)+S0*e^(t*r)
For ivp k = 6000, r = 6%, t = 20, S(t) = 0,
S(0) = -100000*e^(-1.2)+100000
```

Figure 5.6: *The function **dsolve** and **ivp** in SymIntegration is able to compute the solution $S(t)$ for the ordinary differential equation of $\frac{dS}{dt} = Sr - k$ (**SymIntegration/Examples/Test SymIntegration DSolve and IVP for Mortgage case/main.cpp**).*

The symbolic computation to obtain the result can be done with **SymIntegration**, a C++ library that we create from branching out of SymbolicC++ v 3.35, we have been working on this project since April 2025.

For the visualization part, we will use Hamzstlab Mathematics to make an interactive graph.

Figure 5.7: *The interactive shaded line plot for this mortgage loan case example, you can change the time of the loan, the amount you can pay monthly and the interest rate, this can also be applied to other industry like life insurance or car insurance. (**Hamzstlab-Mathematics/examples/First Order Differential Equation/main.cpp**).*

```
...
...

void Demo_MortgageLoan() {
       static int t = 20, t1 = 1;
       static float xs1[100], ys1[100], ys2[100], ys3[100];
       static float r = 0.06;
       static int k = 500;
       for (int i = 1; i <= t; ++i) {
              xs1[i] = i;
              ys1[i] = (12*k/r) - (12*k/r)*(exp(-r*i)) ;
              ys2[i] = i*(12*k) - ys1[i];
              ys3[i] = i*(12*k); // Total payment
       }

       static ImVec4 color1 = ImVec4(0.133,0.545,0.133,1); // Forest
           green
       static ImVec4 color2 = ImVec4(0,0.808,0.82,1); // Dark turquoise
       static ImVec4 color3 = ImVec4(0.855,0.647,0.125,1); // Goldenrod
       static float thickness = 1;
       static int shade_mode = 0;
       static ImPlotShadedFlags flags = 0;
       static bool show_lines = true;
       static bool show_fills = true;
```

```
static bool range = false, details = false;
ImGui::Text("Monthly payment = %4.0d", k);
ImGui::Text("Annual payment = %4.0d ", k*12);
ImGui::Checkbox("Change parameters", &range);
ImGui::Checkbox("Payment details", &details);
ImGui::Checkbox("Lines",&show_lines); ImGui::SameLine();
ImGui::Checkbox("Fills",&show_fills);

if (range) {
       ImGui::SetNextItemWidth(200);
       ImGui::BulletText("Monthly Payment");
       ImGui::SliderInt("##Monthly Payment", &k, 100, 10000);
       ImGui::SetNextItemWidth(200);
       ImGui::BulletText("Length of Mortgage");
       ImGui::SliderInt("##Length of Mortgage", &t, 5, 50);
       ImGui::SetNextItemWidth(200);
       ImGui::BulletText("Return Rate");
       ImGui::DragFloat("##Return rate", &r, 0.01f, 0.2f);
       ImGui::SetNextItemWidth(200);
}
if (details) {
       ImGui::SetNextItemWidth(200);
       ImGui::BulletText("Time / t");
       ImGui::SliderInt("##Time / t", &t1, 1, t);
       ImGui::Text("Total amount borrowed = %.3f ", ys1[t1]);
       ImGui::Text("Interest paid = %.3f ", ys2[t1]);
       ImGui::Text("Total payment = %.3f ", ys3[t1]);
}

if (ImPlot::BeginPlot("Mortgage Loan Accumulation")) {
       ImPlot::SetupAxes("t","S(t)");
       ImPlot::SetupAxesLimits(0, 30, 0, 200000);
       ImPlot::SetupLegend(ImPlotLocation_East,
           ImPlotLegendFlags_Outside);


       if (show_fills) {
              ImPlot::PushStyleVar(ImPlotStyleVar_FillAlpha, 0.23f
                  );
              ImPlot::SetNextFillStyle(color2);
              ImPlot::PlotShaded("Amount borrowed", xs1, ys1, t+1,
                   shade_mode == 0, flags);
              ImPlot::SetNextFillStyle(color3);
              ImPlot::PlotShaded("Interest paid", xs1, ys2, t+1,
                  shade_mode == 0, flags);
              ImPlot::SetNextFillStyle(color1);
              ImPlot::PlotShaded("Total payment", xs1, ys3, t+1,
                  shade_mode == 0, flags);
```

```
                    ImPlot::PopStyleVar();
                }
                if (show_lines) {
                    ImPlot::SetNextLineStyle(color2, thickness);
                    ImPlot::PlotLine("Amount borrowed", xs1, ys1, t+1);
                    ImPlot::SetNextLineStyle(color3, thickness);
                    ImPlot::PlotLine("Interest paid", xs1, ys2, t+1);
                    ImPlot::SetNextLineStyle(color1, thickness);
                    ImPlot::PlotLine("Total payment", xs1, ys3, t+1);


                }
                ImPlot::EndPlot();
            }
}

...
```

Code 23: *Hamzstlab-Mathematics/hamzstlab_firstorderdifferentialequations.cpp*

[H*] **Radiocarbon Dating Example:**
An important tool in archeological research is radiocarbon dating, developed by the American chemist Willard F. Libby. This is a means of determining the age of certain wood and plant remains, hence of animal or human bones or artifacts found buried at the same levels.

Radiocarbon dating is based on the fact that some wood or plant remains contain residual amounts of carbon-14, a radioactive isotope of carbon. This isotope is accumulated during the lifetime of the plant and begins to decay at its death. Since the half-life of carbon-14 is long (approximately 5730 years), measurable amounts of carbon-14 is still present, then by appropriate laboratory measurements the proportion of the original amount of carbon-14 that remains can be accurately determined. In other words, if $Q(t)$ is the amount of carbon-14 at time t and Q_0 is the original amount, then the ratio $\frac{Q(t)}{Q_0}$ can be determined, as long as this quantity is not too small. Present measurement techniques permit the use of this method for time periods of 50,000 years or more.

(a) Assuming that Q satisfies the differential equation $Q' = -rQ$, determine the decay constant r for carbon-14.

(b) Find an expression for $Q(t)$ at any time t, if $Q(0) = Q_0$

(c) Suppose that certain remains are discovered in which the current residual amounts of carbon-14 is 20% of the original amount. Determine the age of the remains.

Solution:

(a) The rate that the mass of carbon-14 decreases is assumed to be proportional to how much is present at any given time

$$\frac{dQ}{dt} \propto -Q$$

Change this proportionality to an equation by introducing a constant r on the right

side.

$$Q' = -rQ$$

Divide both sides by Q.

$$\frac{Q'}{Q} = -r$$

The left side can be written as the derivative of a logarithm by the chain rule.

$$\begin{aligned}\frac{d}{dt}\ln\ Q &= -r \\ \ln\ Q &= -rt + C \\ Q(t) &= e^{-rt+C} \\ &= ce^{-rt}\end{aligned}$$

(b) Suppose that there is a certain amount of mass Q_0 initially. Then the initial condition is $Q(0) = Q_0$. We will use it to determine c.

$$\begin{aligned}Q(0) &= ce^{-r(0)} \\ c &= Q_0\end{aligned}$$

Consequently, the mass decays exponentially. Substituting back we will have the solution to the initial value problem as

$$Q(t) = Q_0 e^{-rt}$$

(c) From the formula that we obtain at (b) we know that r can be determined with knowledge of the half-life. For carbon-14, specifically, we know that half the initial mass is lost after 5730 years.

$$\frac{Q_0}{2} = Q_0 e^{-r(5730)}$$

Solve this equation for r

$$\begin{aligned}e^{-5730r} &= \frac{1}{2} \\ \ln|e^{-5730r}| &= \ln\left|\frac{1}{2}\right| \\ r &= -\frac{\ln\left|\frac{1}{2}\right|}{5730} \\ &= \frac{\ln|2|}{5730} \\ r &\approx 0.0001210\end{aligned}$$

r is the rate of the decreasing mass per year. Therefore, the mass of a sample of carbon-14 is

$$Q(t) = Q_0 e^{-\frac{\ln|2|}{5730}t}$$

where t is in years. If some remains have 20% of the original amount of carbon-14, then $Q(t) = 0.2Q_0$. Solve the resulting equation for t to determine how old the

remains are.

$$
\begin{aligned}
0.2Q_0 &= Q_0 e^{-\frac{\ln|2|}{5730}t} \\
e^{-\frac{\ln|2|}{5730}t} &= 0.2 \\
\ln|e^{-\frac{\ln|2|}{5730}t}| &= \ln|0.2| \\
-\frac{\ln|2|}{5730}t &= \ln\frac{1}{5} \\
-\frac{\ln|2|}{5730}t &= -\ln|5| \\
t &= \frac{\ln|5|}{\ln|2|}5730 \\
t &\approx 13,305
\end{aligned}
$$

so the age of the remains are around 13,305 years old.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DSolve and IVP for Radiocarbon Dating ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DSolve and IVP for Radiocarbon Dating ]# ./ma
in

DSolve for Q' = -rQ

Q(t) = C*e^(-t*r)
Q(t=0) = C
For ivp Q(0) = Q0,
C = Q0

Solution for the ivp,
Q(t) = Q0*e^(-t*r)

Half-life problem for carbon-14, the ivp becomes:
Q0*e^(-5730*r)-0.5*Q0 = 0

Determining the rate by solving the ivp solution
r = 0.000120968

Determining the age of the remains with 20% of carbon-14
t = 13304.6
```

Figure 5.8: *The computation of the rate and the age of remains with SymIntegration' **dsolve** and **ivp** functions. (**SymIntegration/Examples/Test SymIntegration DSolve and IVP for Radiocarbon Dating/-main.cpp**).*

Figure 5.9: *The interactive line plot for this radiocabon dating, you can change the half-time and the percentage amount of the carbon-14 in the remains. (**Hamzstlab-Mathematics/examples/First Order Differential Equation/main.cpp**).*

```
...

double division(double x, double y)
{
	return x/y;
}
void Demo_RadiocarbonDating() {
	static int half_life=5730, t1 = 20;
	static double xt1[50000], qt[50000];
	double r = -(division(1,half_life))*log(0.5);
	for (int i = 1; i <= 50000; ++i) {
		xt1[i] = i;
		qt[i] = 1*exp(-r*i) ;
	}

	//static ImVec4 color1 = ImVec4(0.133,0.545,0.133,1); // Forest
	    green
	static ImVec4 color2 = ImVec4(0,0.808,0.82,1); // Dark turquoise
	//static ImVec4 color3 = ImVec4(0.855,0.647,0.125,1); // Goldenrod
	static float thickness = 1;
	static bool range = true;

	ImGui::Checkbox("Change parameters", &range);

	if (range) {
		ImGui::SetNextItemWidth(200);
```

```
            ImGui::BulletText("Half—life of carbon—14 (years)");
            ImGui::SliderInt("##Half—life of carbon—14 (years)", &
                half_life, 100, 10000);
            ImGui::SetNextItemWidth(200);
            ImGui::BulletText("Residual amount of cabon—14 in the
                remains (%)");
            ImGui::SliderInt("##Residual amount of carbon—14 in the
                remains (%) ", &t1, 1, 100);
            ImGui::Text("Age of remains = %.5f ", log(division(t1,100))
                /(—r));
            ImGui::Text("Rate of decay = %.10f ", r);
      }

      if (ImPlot::BeginPlot("Radiocarbon Dating with carbon—14")) {
            ImPlot::SetupAxes("t","Q(t)");
            ImPlot::SetupAxesLimits(0, 30000, 0, 1);
            ImPlot::SetupLegend(ImPlotLocation_East,
                ImPlotLegendFlags_Outside);

            ImPlot::SetNextLineStyle(color2, thickness);
            ImPlot::PlotLine("Q(t)=Q_{0}exp(—rt)", xt1, qt, 50000);

            ImPlot::EndPlot();
      }
}
...
```

Code 24: *Hamzstlab-Mathematics/hamzstlab_firstorderdifferentialequations.cpp*

[H*] **Newton's Law of Cooling Example:**
Newton's law of cooling states that the temperatureof an object changesat a rate proportional to the difference between its temperature and that of its surrounding. Suppose that the temperature of a cup of coffee obeys Newton's law of cooling. If the coffee has a temperature of 200^0F when freshly poured, and 1 minute later has cooled to 190^0in a room at 70^F, determine when the coffee reaches a temperature of 150^0F.

Solution:
Newton's law of cooling can be expressed mathematically as a proportionality. Let $T = T(t)$ be the temperature of the coffee; $\frac{dT}{dt}$then represents the rate the temperature changes. Also, let T_0 be the temperature of the surroundings.

$$\frac{dT}{dt} \propto -(T - T_0)$$

The minus sign in front of the parentheses accounts for the fact that the temperature of an object will decease if it's hotter than the environment. Introduce a proportionality constant k on the right to change this to an equation we can use

$$\frac{dT}{dt} = -k(T - T_0)$$

Solve this ODE by separating variables.

$$\begin{aligned}
\frac{dT}{T-T_0} &= -k\,dt \\
\ln|T-T_0| &= -kt+c \\
|T-T_0| &= e^{-kt+c} \\
|T-T_0| &= e^c e^{-kt} \\
T-T_0 &= \pm Ce^{-kt} \\
T(t) &= T_0 + Ce^{-kt}
\end{aligned}$$

Use the initial condition $T(0) = 200$ to determine C.

$$\begin{aligned}
T(0) &= T_0 + C \\
C &= 200 - T_0
\end{aligned}$$

Also, use the fact that the room temperature is $T_0 = 70$.

$$T(t) = 70 + 130e^{-kt}$$

After 1 minute, the coffee has cooled to 190^0F.

$$\begin{aligned}
T(1) &= 70 + 130e^{-k} \\
190 &= 70 + 130e^{-k} \\
e^{-k} &= \frac{12}{13} \\
\ln e^{-k} &= \ln\frac{12}{13} \\
k &= \ln\frac{13}{12} \\
k &\approx 0.0800\frac{1}{\text{minute}}
\end{aligned}$$

So then

$$T(t) = 70 + 130e^{-t\ln\left(\frac{13}{12}\right)}$$

Set $T(t) = 150$ and solve the resulting equation for t to find when the coffee reaches

150^0F.

$$
\begin{aligned}
T(t) &= 70 + 130e^{-t\ln\left(\frac{13}{12}\right)} \\
150 &= 70 + 130e^{-t\ln\left(\frac{13}{12}\right)} \\
80 &= 130e^{-t\ln\left(\frac{13}{12}\right)} \\
e^{-t\ln\left(\frac{13}{12}\right)} &= \frac{8}{13} \\
\ln\left(e^{-t\ln\left(\frac{13}{12}\right)}\right) &= \ln\left(\frac{8}{13}\right) \\
-t\ln\left(\frac{13}{12}\right) &= \ln\left(\frac{8}{13}\right) \\
t\ln\left(\frac{13}{12}\right) &= \ln\left(\frac{13}{8}\right) \\
t &= \frac{\ln\left(\frac{13}{8}\right)}{\ln\left(\frac{13}{12}\right)} \\
&\approx 6.07 \text{ minute}
\end{aligned}
$$

Therefore, it takes about 6 minutes and 4 seconds for the coffee to cool to 150^0F.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DSolve and IVP for Newton's Law of Cooling ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DSolve and IVP for Newton's Law of Cooling ]# ./main

DSolve for T' = -k(T - Troom)

T(t) = Troom+C*e^(-t*k)

T(t) = C*e^(-t*k)+70

Solution for the ivp T(0) = 200,
C = -Troom*e^(t*k)+200*e^(t*k)
C = 130

T(t) = 130*e^(-t*k)+70

Detemining rate k:
T(1) = 130*e^(-k)+70
k = 0.0800427

The coffee reaches temperature T(t) = 150 Fahrenheit at
t = 6.06561 minutes
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DSolve and IVP for Newton's Law of Cooling ]#
```

Figure 5.10: *The computation of initial value problem solution for the Newton's law of cooling with SymIntegration' **dsolve** function. (**SymIntegration/Examples/Test SymIntegration DSolve and IVP for Newton's Law of Cooling/main.cpp**).*

Figure 5.11: *The interactive line plot for Newton's law of cooling. (**Hamzstlab-Mathematics/examples/First Order Differential Equation/main.cpp**).*

[H*] **Pollutants in a Lake Example:**
Consider a lake of constant volume V containing at time t an amount $Q(t)$ of pollutant, evenly distributed throughout the lake with a concentration $c(t)$, where $c(t) = \frac{Q(t)}{V}$. Assume that water containing a concentration k of pollutant enters the lake at a rate r, and that water leaves the lake at the same rate. Suppose that pollutants are also added directly to the lake at a constant rate P.

Note that given assumptions neglect a number of factors that may, in some cases, be important-for example, the water added or lost by precipitation, absorption, and evaporation; the stratifying effect of temperature differences in a deep lake; the tendency of irregularities in the coastline to produce sheltered bays; and the fact that pollutants are not deposited evenly throughout the lake but (usually) at isolated points around its periphery. The results below must be interpreted in the light of the neglect of such factors as these.

(a) If at time $t = 0$ the concentration of pollutant is c_0, find an expression for the concentation $c(t)$ at any time. What is the limiting concentration as $t \to \infty$?

(b) If the addition of pollutants to the lake is terminated ($k = 0$ and $P = 0$ for $t > 0$), determine the time interval T that must elapse before the concentration of pollutants is reduced to 50% of its original value; to 10% of its original value.

(c) Table 5.1 contains data for several of the Great Lakes. Using these data, determine from part (b) the time T necessary to reduce the contamination of each of these lakes to 10% of the original value.

Solution:

(a) The conservation law that governs the mass of pollutant in the lake is as follows.

Lake	V($km^3 \times 10^3$)	r($km^3/year$)
Superior	12.2	65.2
Michigan	4.9	158
Erie	0.46	175
Ontario	1.6	209

Table 5.1: *Volume and Flow Data for the Great Lakes*

rate of mass accumulation = rate of mass in - rate of mass out

the rate of mass in is the sum of rk and P, and the rate of mass out is $rc(t)$.

$$\begin{aligned}\frac{dQ}{dt} &= rk + P - rc(t) \\ &= rk + P - \frac{r}{V}Q(t) \\ \frac{dQ}{dt} + \frac{r}{V}Q &= rk + P\end{aligned}$$

This is a first-order linear inhomogeneous ODE, so it can be solved by multiplying both sies by an integrating factor $\mu(t)$.

$$\begin{aligned}\mu(t) &= e^{\int \frac{r}{V}\, ds} = e^{\frac{rt}{V}} \\ e^{\frac{rt}{V}}\frac{dQ}{dt} + \frac{r}{V}e^{\frac{rt}{V}}Q &= rke^{\frac{rt}{V}} + Pe^{\frac{rt}{V}} \\ \frac{d}{dt}(e^{\frac{rt}{V}}Q) &= rke^{\frac{rt}{V}} + Pe^{\frac{rt}{V}} \\ e^{\frac{rt}{V}}Q &= \int^t (rke^{\frac{rs}{V}} + Pe^{\frac{rs}{V}})\, ds + C \\ &= rk\left(\frac{V}{r}\right)e^{\frac{rt}{V}} + P\left(\frac{V}{r}\right)e^{\frac{rt}{V}} + C \\ e^{\frac{rt}{V}}Q &= kVe^{\frac{rt}{V}} + \frac{PV}{r}e^{\frac{rt}{V}} + C \\ Q(t) &= kV + \frac{PV}{r}e^{\frac{rt}{V}} + Ce^{\frac{-rt}{V}}\end{aligned}$$

Divide both sides by V to obtain the concentration.

$$c(t) = \frac{Q(t)}{V} = k + \frac{P}{r} + \frac{C}{V}e^{\frac{-rt}{V}}$$

Apply the initial condition $c(0) = c_0$ to determine C.

$$\begin{aligned}c(0) &= k + \frac{P}{r} + \frac{C}{V} \\ c_0 &= k + \frac{P}{r} + \frac{C}{V} \\ \frac{C}{V} &= c_0 - k - \frac{P}{r}\end{aligned}$$

Therefore, the concentration is

$$c(t) = \frac{Q(t)}{V} = k + \frac{P}{r} + \left(c_0 - k - \frac{P}{r}\right) e^{\frac{-rt}{V}}$$

Because of the decaying exponential function, the limit of $c(t)$ as $t \to \infty$ is

$$\lim_{t\to\infty} c(t) = k + \frac{P}{r}$$

(b) If the pollution stops, then the rate of mass in becomes zero in the conservation law.

rate of mass accumulation = - rate of mass out

The rate of mass out is still $rc(t)$.

$$\begin{aligned}
\frac{dQ}{dt} &= -rc(t) \\
&= -\frac{r}{V}Q(t) \\
\frac{\frac{dQ}{dt}}{Q} &= -\frac{r}{V} \\
\frac{dQ}{Q} &= -\frac{r}{V}\, dt \\
\ln Q &= -\frac{rt}{V} + C_1 \\
Q &= e^{-\frac{rt}{V}+C_1} \\
Q(t) &= C_2 e^{-\frac{rt}{V}}
\end{aligned}$$

Since Q is the mass of pollutant, divide both sides by V to get the concentration.

$$c(t) = \frac{Q(t)}{V} = \frac{A_1}{V} e^{-\frac{rt}{V}}$$

Use the initial condition $c(0) = c_0$ to determine C_2

$$c(0) = \frac{C_2}{V} = c_0$$

Therefore

$$c(t) = c_0 e^{-\frac{rt}{V}}$$

Set $c(t) = 0.5c_0$ and solve for $t = T$ to find how long it will take for the concentration to reach half its initial value.

$$\begin{aligned}
0.5c_0 &= c_0 e^{-\frac{rT}{V}} \\
e^{-\frac{rT}{V}} &= 0.5 \\
\ln e^{-\frac{rT}{V}} &= \ln 0.5 \\
-\frac{rT}{V} &= \ln \frac{1}{2} \\
T &= \frac{V}{r} \ln 2
\end{aligned}$$

On the other hand, set $c(t) = 0.1c_0$ and solve for $t = T$ to find how long it will take for the concentration to reach one-tenth its initial value.

$$\begin{aligned} 0.1c_0 &= c_0 e^{-\frac{rt}{V}} \\ e^{-\frac{rt}{V}} &= 0.1 \\ \ln e^{-\frac{rt}{V}} &= \ln 0.1 \\ -\frac{rT}{V} &= \ln \frac{1}{10} \\ T &= \frac{V}{r} \ln 10 \end{aligned}$$

(c) Plug in the numbers fo V and r into the formula at part (b).
Lake Superior:

$$T = \frac{V}{r} \ln 10 = \frac{12.2 \times 10^3}{65.2} \ln 10 \approx 431 \; \textit{years}$$

Lake Michigan:

$$T = \frac{V}{r} \ln 10 = \frac{4.9 \times 10^3}{158} \ln 10 \approx 71.4 \; \textit{years}$$

Lake Erie:

$$T = \frac{V}{r} \ln 10 = \frac{0.46 \times 10^3}{175} \ln 10 \approx 6.05 \; \textit{years}$$

Lake Ontario:

$$T = \frac{V}{r} \ln 10 = \frac{1.6 \times 10^3}{209} \ln 10 \approx 17.6 \; \textit{years}$$

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DSolve and IVP for Pollutant in Great Lakes ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DSolve and IVP for Pollutant in Great Lakes ]# ./main

DSolve for Q' = rk +  P - rc(t) ,        c(t) = Q(t)/V

Q(t) = k*V+P*V*r^(-1)+C*e^(-V^(-1)*t*r)
c(t) = k+P*r^(-1)+C*e^(-V^(-1)*t*r)*V^(-1)
c(0) = k+P*r^(-1)+C*V^(-1)
c0 - c(0) = c0-k-P*r^(-1)-C*V^(-1)
For ivp c(0) = c0,
C/V = c0-k-P*r^(-1)

Therefore, the concentration is,
c(t) = k+P*r^(-1)+c0*e^(-V^(-1)*t*r)-k*e^(-V^(-1)*t*r)-P*r^(-1)*e^(-V^(-1)*t*r)

The limit of c(t) as t -> ∞
lim_{t -> ∞} c(t) = k+P*r^(-1)+c0*e^(-inf*V^(-1)*r)-k*e^(-inf*V^(-1)*r)-P*r^(-1)*e^(-inf*V^(-1)*r)

If the pollution stops, then
Q(t) = C*e^(-V^(-1)*t*r)

The concentration when the pollution stops is
c(t) = C*e^(-V^(-1)*t*r)*V^(-1)

For ivp c(0) = c0,
c0-C*V^(-1) = 0

Therefore the ivp solution will be,
c(t) = c0*e^(-V^(-1)*t*r)

Set c(t) = 0.5 c0 and solve for t=T,
T = 0.693147*V*r^(-1)

Set c(t) = 0.1 c0 and solve for t=T,
T = 2.30259*V*r^(-1)

Time to reduce the contamination in each lakes to 10 percent of the original value

Lake Superior:  T = 430.852

Lake Michigan:  T = 71.4093

Lake Erie:      T = 6.05251

Lake Ontario:   T = 17.6274
```

Figure 5.12: *The computation of initial value problem solution for the pollutant concentration in a lake, $c(t)$, and the time to clean up a lake to make the pollutant concentration less than* 10% *with SymIntegration'* ***dsolve*** *function. (**SymIntegration/Examples/Test SymIntegration DSolve and IVP for Pollutant in Great Lakes/main.cpp**).*

Figure 5.13: *The interactive line plot for pollutant in great lakes. (**Hamzstlab-Mathematics/examples/First Order Differential Equation/main.cpp**).*

[H*] **Fluid Dynamic Example:**
A body falling in a relatively dense fluid, oil for example, is acted on by three forces: a resistive force R, a buoyant force B, and its weight w due to gravity. The buoyant force is equal to the weight of the fluid displaced by the object. For a slowly moving spherical body of radius a, the resistive force is given by Stokes's law, $R = 6\pi\mu a|v|$, where v is the velocity of the body, and μ is the coefficient of viscosity of the surrounding fluid.

(a) Find the limiting velocity of a solid sphere of radius a and density ρ falling freely in a medium of density ρ' and coefficient of viscosity μ.

(b) In 1910 R.A. Millikan studied the motion of tiny droplets of oil falling in an electric field. A field of strength E exerts a force Ee on a droplet with charge e. Assume that E has been adjusted so the droplet is held stationary $(v = 0)$ and that w and B are as given above. Find an expression for e. Millikan repeated this experiment many times, and from the data that he gathered he was able to deduce the charge on an electron.

Solution:
According to Newton's second law, the equation of motion for the mass is

$$\sum \mathbb{F} = m$$

This is a vector equation: it consists of three scalar equations-one for each direction in the chosen coordinate system. Since the mass is projected vertically upward, only the

equation in the y-direction is relevant.

$$\sum F_y = ma_y$$

(a) We will use the free-body diagram to write the equation of motion. (Take the positive y-axis to point downward in the direction of the motion.)

$$w - R - B = ma_y \tag{5.29}$$

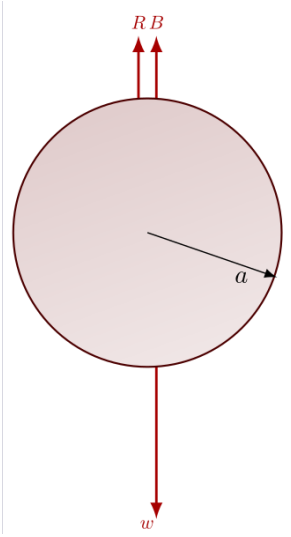

Figure 5.14: *The free-body diagram for a sphere shape body falling in a dense fluid.*

Now, we'll write expressions for each of the variables. w is the weight of the solid ball; multiply its volume by its density by g to get force.

$$w = \frac{4}{3}\pi a^3 \times \rho \times g$$

R is the resistive force given by Stokes's law

$$R = 6\pi\mu a v$$

B is the buoyant force; multiply the volume of the ball by the density of the medium by g to get force.

$$B = \frac{4}{3}\pi a^3 \times \rho' \times g$$

m is the mass of the ball; multiply its volume by its density to get the mass.

$$m = \frac{4}{3}\pi a^3$$

Finally, replace a_y with dv/dt. As a result, equation (5.29) becomes

$$\frac{4}{3}\pi a^3 \rho g - 6\pi\mu a v - \frac{4}{3}\pi a^3 \rho' g = \frac{4}{3}\pi a^3 \rho \frac{dv}{dt}$$

Simplify both sides of the equation.

$$\begin{aligned}\frac{4}{3}\pi a^3(\rho-\rho')g-6\pi\mu a v &= \frac{4}{3}\pi a^3\rho\frac{dv}{dt}\\ \left(1-\frac{\rho'}{\rho}\right)g-\frac{9\mu}{2a^2\rho}v &= \frac{dv}{dt}\\ \frac{dv}{dt}+\frac{9\mu}{2a^2\rho}v &= \left(1-\frac{\rho'}{\rho}\right)g\end{aligned}$$

This is a first-order linear inhomogeneous ODE, so it can be solved by multiplying both sides by an integrating factor $\psi(t)$

$$\psi(t)=e^{\int^t \frac{9\mu}{2a^2\rho}\,ds}=e^{\frac{9\mu t}{2a^2\rho}}$$

Now, to find the general solution

$$\begin{aligned}e^{\frac{9\mu t}{2a^2\rho}}\frac{dv}{dt}+\frac{9\mu}{2a^2\rho}e^{\frac{9\mu t}{2a^2\rho}}v &= \left(1-\frac{\rho'}{\rho}\right)ge^{\frac{9\mu t}{2a^2\rho}}\\ \frac{d}{dt}\left[e^{\frac{9\mu t}{2a^2\rho}}v\right] &= \left(1-\frac{\rho'}{\rho}\right)ge^{\frac{9\mu t}{2a^2\rho}}\\ \int\frac{d}{dt}\left[e^{\frac{9\mu t}{2a^2\rho}}v\right]\,dt &= \int\left(1-\frac{\rho'}{\rho}\right)ge^{\frac{9\mu t}{2a^2\rho}}\,dt\\ e^{\frac{9\mu t}{2a^2\rho}}v &= \frac{2a^2\rho}{9\mu}\left(1-\frac{\rho'}{\rho}\right)ge^{\frac{9\mu t}{2a^2\rho}}+C_1\\ v(t) &= \frac{2a^2}{9\mu}(\rho-\rho')g+C_1e^{-\frac{9\mu t}{2a^2\rho}}\end{aligned}$$

In the limit as $t\to\infty$, the exponential function decays to zero, and only the first term remains.

$$\lim_{t\to\infty}v(t)=\frac{2a^2}{9\mu}(\rho-\rho')g$$

(b) Since the oil droplet is stationary, the sum of the forces must be equal to zero.

$$\begin{aligned}w-F_e-B &= 0\\ w &= \frac{4}{3}\pi a^3\rho g\\ F_e &= eE\\ B &= \frac{4}{3}\pi a^3\rho' g\end{aligned}$$

Therefore, the electron charge is

$$\begin{aligned}\frac{4}{3}\pi a^3\rho g-eE-\frac{4}{3}\pi a^3\rho' g &= 0\\ eE &= \frac{4}{3}\pi a^3(\rho-\rho')g\\ e &= \frac{4}{3E}\pi a^3(\rho-\rho')g\end{aligned}$$

```
xterm
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DSolve and IVP for Fluid Dynamics ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DSolve and IVP for Fluid Dynamics ]# ./main

DSolve for v' = ((-9*μ)/(2*a*a*ρ))*gv+(1-(ρ2/ρ))*g

v(t) = -2/9*ρ2*g*μ^(-1)*a^(2)+2/9*g*μ^(-1)*a^(2)*ρ+C*e^(-9/2*μ*a^(-2)*ρ^(-1)*t)
w - Fe - B = 0
1.33333*π*a^(3)*ρ*g-e*E-1.33333*π*a^(3)*ρ2*g = 0
e = 1.33333*π*a^(3)*ρ*g*E^(-1)-1.33333*π*a^(3)*ρ2*g*E^(-1)
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DSolve and IVP for Fluid Dynamics ]#
```

Figure 5.15: *The computation of initial value problem solution for the case of a body falling in a relatively dense fluid with SymIntegration' **dsolve** function. (**SymIntegration/Examples/Test SymIntegration DSolve and IVP for Fluid Dynamics/main.cpp**).*

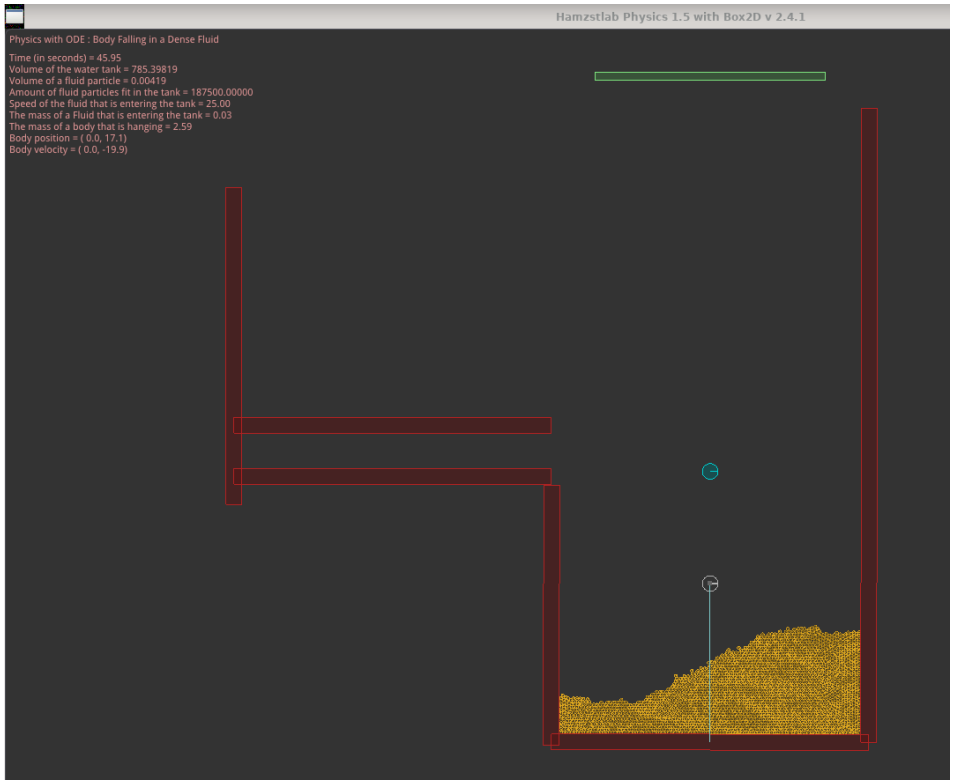

Figure 5.16: *The 2D simulation of body falling in a relatively dense fluid with Hamzstlab Physics 2D that is using Box2D version 2.4.2 as the backend library (**Hamzstlab-Physics2D/tests/ode_bodyfallinginadensefluid.cpp**).*

Figure 5.17: *The 2D simulation of body that has already fallen in a relatively dense fluid (**Hamzstlab-Physics2D/tests/ode_bodyfallinginadensefluid.cpp**).*

[H*] **Sky Diver Example:**
A sky diver weighing 180 lb (including equipment) falls vertically downward from an altitude of 5000 ft and opens parachute after 10 s of free fall. Assume that the force of air resistance is $0.75|v|$ when the parachute is closed and $12|v|$ when the parachute is open, where the velocity v is measured in ft/s.

(a) Find the speed of the sky diver when the parachute opens.
(b) Find the distance fallen before the parachute opens.
(c) What is the limiting velocity v_L after the parachute opens?
(d) Determine how long the sky diver is in the air after the parachute opens.
(e) Plot the graph of velocity versus time from the beginning of the fall until the skydiver reaches the ground.

Solution:
Split up the diver's motion into two parts: one where the parachute is closed and the other where the parachute is open. According to Newton's second law, the equation of motion for the diver is

$$\sum \mathbb{F} = m$$

This is a vector equation, so it actually represents three scalar equations-one for each direction in the chosen coordinate system. Since the diver only moves vertically, only the y-equation is relevant.

$$\sum F_y = ma_y$$

We will replace a_y with $\frac{dv}{dt}$.

Minus signs are placed in front of v, thus it becomes $-0.75v$, because the positive y-axis points upward and the diver is moving downward.

(a) Before the parachute opens:

$$\begin{aligned}
-0.75v - W &= ma_y \\
\frac{dv}{dt} + \frac{0.75}{m}v &= -\frac{W}{m} \\
e^{\frac{0.75t}{m}}\frac{dv}{dt} + \frac{0.75}{m}e^{\frac{0.75t}{m}}v &= -\frac{W}{m}e^{\frac{0.75t}{m}} \\
\frac{d}{dt}\left(e^{\frac{0.75t}{m}}v\right) &= -\frac{W}{m}e^{\frac{0.75t}{m}} \\
e^{\frac{0.75t}{m}}v &= -\frac{W}{0.75}e^{\frac{0.75t}{m}} + C_1 \\
v(t) &= -\frac{W}{0.75} + C_1 e^{-\frac{0.75t}{m}}
\end{aligned}$$

Integrate the velocity to get the position.

$$y(t) = -\frac{W}{0.75}t - \frac{C_1 m}{0.75}e^{-\frac{0.75t}{m}} + C_4$$

After the parachute opens:

$$\begin{aligned}
-12v - W &= ma_y \\
\frac{dv}{dt} + \frac{12}{m}v &= -\frac{W}{m} \\
e^{\frac{12t}{m}}\frac{dv}{dt} + \frac{12}{m}e^{\frac{12t}{m}}v &= -\frac{W}{m}e^{\frac{12t}{m}} \\
\frac{d}{dt}\left(e^{\frac{12t}{m}}v\right) &= -\frac{W}{m}e^{\frac{12t}{m}} \\
e^{\frac{12t}{m}}v &= -\frac{W}{12}e^{\frac{12t}{m}} + C_2 \\
v(t) &= -\frac{W}{12} + C_2 e^{-\frac{12t}{m}} \\
v(t) &= -\frac{W}{12} + C_3 e^{-\frac{12(t-10)}{m}}
\end{aligned}$$

Integrate the velocities to get the positions.

$$\begin{aligned}
y(t) &= -\frac{W}{12}t - \frac{C_3 m}{12}e^{-\frac{12(t-10)}{m}} + C_5 \\
y(t) &= -\frac{W}{12}(t-10) - \frac{C_3 m}{12}e^{-\frac{12(t-10)}{m}} + C_6
\end{aligned}$$

Our task now is to determine the integration constants, C_1, C_3, C_4, and C_6. Assuming that the diver falls from rest, the initial condition $v(0) = 0$ can be used to find C_1.

Before the parachute opens:

$$\begin{aligned} v(0) &= 0 \\ -\frac{W}{0.75} + C_1 &= 0 \\ C_1 &= \frac{W}{0.75} \end{aligned}$$

So then the velocity will be

$$v(t) = -\frac{W}{0.75} + \frac{W}{0.75}e^{-\frac{0.75t}{m}} = -\frac{W}{0.75}(1 - e^{-\frac{0.75t}{m}})$$

The parachute opens at 10 seconds. The velocity at this time is

$$v(10) = -\frac{W}{0.75}(1 - e^{-\frac{7.5}{m}})$$

and it serves as the initial condition that we can use to find C_3.

After the parachute opens:

$$\begin{aligned} v(10)_{before} &= v(10)_{after} \\ -\frac{W}{0.75}(1 - e^{-\frac{7.5}{m}}) &= -\frac{W}{12} + C_3 e^{-\frac{12(10-10)}{m}} \\ -\frac{W}{0.75}(1 - e^{-\frac{7.5}{m}}) &= -\frac{W}{12} + C_3 \\ C_3 &= \frac{W}{12} - \frac{W}{0.75}(1 - e^{-\frac{7.5}{m}}) \end{aligned}$$

So then the velocity after the parachute opens becomes

$$v(t) = -\frac{W}{12}\left[\frac{W}{12} - \frac{W}{0.75}(1 - e^{-\frac{7.5}{m}})\right] e^{-\frac{12(t-10)}{m}}$$

With these values of C_1 and C_3, the position before the parachute opens becomes

$$y(t) = -\frac{W}{0.75}\left(t + \frac{m}{0.75}e^{-\frac{0.75t}{m}}\right) + C_4$$

and the position after the parachute opens becomes

$$y(t) = -\frac{W}{12}(t - 10) - \left[\frac{W}{12} - \frac{W}{0.75}(1 - e^{-\frac{7.5}{m}})\right] \frac{m}{12} e^{-\frac{12(t-10)}{m}} + C_6$$

Use the fact that the diver starts at 5000 feet to find C_4.

$$\begin{aligned} y(0) &= -\frac{W}{0.75}\left(\frac{m}{0.75}\right) + C_4 \\ 5000 &= -\frac{W}{0.75}\left(\frac{m}{0.75}\right) + C_4 \\ C_4 &= 5000 + \frac{Wm}{0.5625} \end{aligned}$$

So then, before the parachute opens:

$$y(t) = -\frac{W}{0.75}\left(t + \frac{m}{0.75}e^{-\frac{0.75t}{m}}\right) + 5000 + \frac{Wm}{0.5625}$$

The position of the diver at 10 seconds (when the parachute is opened) is

$$y(10) = -\frac{W}{0.75}\left(10 + \frac{m}{0.75}e^{-\frac{7.5}{m}}\right) + 5000 + \frac{Wm}{0.5625}$$

Because the position of the diver is continuous, this serves as the initial condition to find C_6.

After the parachute opens, the position will be:

$$\begin{aligned} y(10)_{before} &= y(10)_{after} \\ -\left[\frac{W}{12} - \frac{W}{0.75}(1 - e^{-\frac{7.5}{m}})\right]\frac{m}{12} + C_6 &= -\frac{W}{0.75}\left(10 + \frac{m}{0.75}e^{-\frac{7.5t}{m}}\right) + 5000 + \frac{Wm}{0.5625} \\ C_6 &= -\frac{W}{0.75}\left(10 + \frac{m}{0.75}e^{-\frac{7.5}{m}}\right) + 5000 + \frac{Wm}{0.5625} \\ &\quad + \left[\frac{W}{12} - \frac{W}{0.75}(1 - e^{-\frac{7.5}{m}})\right]\frac{m}{12} \end{aligned}$$

Now, the numbers w and m will be plugged in. The weight is $W = 180$ lb, and the mass is

$$m = \frac{W}{g} = \frac{180}{9.81 m.s^2 \times 3.28 ft/m} \approx 5.59 \frac{lb \cdot s^2}{ft}$$

As a result, the constants are

$$\begin{aligned} C_1 &= 240 \\ C_3 &\approx -162.2 \\ C_4 &\approx 6790 \\ C_6 &\approx 3846 \end{aligned}$$

and the position and velocity are

$$y(t) \approx \begin{cases} 6790 - 240(7.459e^{-0.13407t} + t), & 0 \le t \le 10 \\ 3846 - 15(t-10) + 75.61e^{-2.145(t-10)}, & 10 \le t \lessapprox 266.4 \end{cases}$$

$$v(t) \approx \begin{cases} -240(1 - e^{-0.13407t} + t), & 0 \le t \le 10 \\ -15 - 162.2e^{-1.2(t-10)}, & 10 \le t \lessapprox 266.4 \end{cases}$$

266.4 seconds is the time it takes for the diver to reach the ground. It was found by using Newton's method :

$$\begin{aligned} y(t)_{after} &= 0 \\ 3846 - 15(t-10) + 75.61e^{-2.145(t-10)} &= 0 \\ t &\approx 266.40608 \end{aligned}$$

we can find the solution of t with Newton's method (a numerical method), since it has a very complicated form to be solved by analytical method.

Now, we already determine the general solution for the position and velocity and obtain all the constants, we can compute the speed of the sky diver when the parachute opens, it is

$$\begin{aligned} |v(10)_{before}| &= |v(10)_{after}| \\ &= |-240(1 - e^{-0.13407(10)} + 10)| \\ |v(10)| &= 177.2 \ \frac{ft}{s} \end{aligned}$$

(b) To determine the distance fallen before the parachute opens, we can use the general solution of the position formula:

$$\begin{aligned} y(10)_{before} &= 6790 - 240(7.459e^{-0.13407(10)} + 10) \\ &= 3921.7 \ ft \end{aligned}$$

thus, since the sky diver starts at 5000 ft, then the sky diver will have travelled a distance of $5000 - 3921.7 = 1078.3$ ft after 10 seconds.

(c) The limiting velocity after the parachute opens can be determined with taking the limit of t to ∞

$$\lim_{t\to\infty} v(t)_{after} = \lim_{t\to\infty} -15 - 162.2e^{-1.2(t-10)} = -15 \ \frac{ft}{s}$$

the minus sign means the sky diver is going downward, and we already make the positive y-axis as upward movement.

(d) The time to reach the ground is 266.4 seconds, and it takes 10 seconds before the parachute opens, thus the time the sky diver is in the air after the parachute opens is

$$266.4 - 10 = 254.4 \ s$$

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DSolve and IVP for Sky Diver ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DSolve and IVP for Sky Diver ]# ./main

DSolve for v' + (0.75/m)v = -W/m

Before parachute opens,
v(t) = -1.33333*W+C1*e^(-0.75*m^(-1)*t)

After parachute opens,
v(t) = -1/12*W+C3*e^(-12*m^(-1)*t+120*m^(-1))

Before parachute opens,
y(t) = -1.33333*W*t-1.33333*C1*e^(-0.75*m^(-1)*t)*m+C4

After parachute opens,
y(t) = -1/12*W*t+5/6*W-1/12*C3*e^(-12*m^(-1)*t+120*m^(-1))*m+C6

C1 = 1.33333*W

Before parachute opens,
v(t) = -1.33333*W+1.33333*W*e^(-0.75*m^(-1)*t)

When the parachute opens,
v(10)_{before} = -1.33333*W+1.33333*W*e^(-7.5*m^(-1))

v(10)_{after} = -1/12*W+C3

C3 = -1.25*W+1.33333*W*e^(-7.5*m^(-1))

So then,
v(t)_{after} = -1/12*W-1.25*W*e^(-12*m^(-1)*t+120*m^(-1))+1.33333*W*e^(112.5*m^(-1)-12*m^(-1)*t)

Thus,
y(t)_{before} = -1.33333*W*t-1.77778*W*e^(-0.75*m^(-1)*t)*m+C4

y(t)_{after} = -1/12*W*t+5/6*W+0.104167*W*e^(-12*m^(-1)*t+120*m^(-1))*m-0.111111*W*e^(112.5*m^(-1)-12*m^(-1)*t)*m+C6

The diver starts at 5000 feet, thus
C4 = 1.77778*W*m+5000

The position of the diver after the parachute opens at 10 seconds,
y(10)_{before} = -13.3333*W-1.77778*W*e^(-7.5*m^(-1))*m+1.77778*W*m+5000

y(10)_{after} = 0.104167*W*m-0.111111*W*e^(-7.5*m^(-1))*m+C6

Thus,
C6 = -13.3333*W-1.66667*W*e^(-7.5*m^(-1))*m+1.67361*W*m+5000
```

Figure 5.18: *The computation of initial value problem solution for the case of sky diver with SymIntegration, part 1. (**SymIntegration/Examples/Test SymIntegration DSolve and IVP for Sky Diver/main.cpp**).*

```
We have W = 180 lb, g = 9.81 m/s^2, then
m = 5.59409

As a result, the constants are
C1 = 240
C3 = 240*e^(-1.3407)-225 = -162.201
C4 = 6790.11
C6 = -1678.23*e^(-1.3407)+4285.22 = 3846.09

*** Newton' Method ***


f(x) = -15*x+75.614*e^(-2.14512*x+21.4512)+3996.09

f'(x) = -162.201*e^(-2.14512*x+21.4512)-15

     n          p_{n}
     0          15.707963943481
     1          266.39306020632
     2          266.40608181366
     3          266.40608181366
The procedure was successful.
*****************************

The time till the diver reaches the ground:
t = 266.40608181366

The general solution for the position:
For 0 <= t <= 10,
y(t) = -240*t-1790.1096442157*e^(-0.13407*t)+6790.1096442157

For 10 <= t <= 266.4,
y(t) = -15*t+75.613958296091*e^(-2.14512*t+21.4512)+3996.0912272048

The general solution for the velocity:
For 0 <= t <= 10,
v(t) = 240*e^(-0.13407*t)-240

For 10 <= t <= 266.4,
v(t) = -162.20101422011*e^(-2.14512*t+21.4512)-15

The speed of the sky diver at 10 second:
v(10) = -177.20101422011

The position of the sky diver at 10 second:
y(10) = 3921.7051855009
```

Figure 5.19: *The computation of initial value problem solution for the case of sky diver with SymIntegration, part 2. (**SymIntegration/Examples/Test SymIntegration DSolve and IVP for Sky Diver/main.cpp**).*

(e) The plot of $y(t)$ and $v(t)$ can be done with either Hamzstplot or Hamzstlab Mathematics, we will choose Hamzstlab Mathematics this time for the interactive feature, we code more but we gain more, if we change the parameters such as the weight of the sky diver or the starting height at the runtime, the plot will adjust in an instant. It is very useful in sky diving field and early warning system.

Figure 5.20: *The interactive plots for the general solution of $y(t)$ and $v(t)$ for the case of sky diver with Hamzstlab Mathematics. (**Hamzstlab-Mathematics/examples/First Order Differential Equation/main.cpp**).*

[H*] **Baseball Example:**
Let $v(t)$ and $w(t)$, respectively, be the horizontal and vertical components of the velocity of the batted (or thrown) baseball. the equations of motion are

$$\frac{dv}{dt} = -rv, \quad \frac{dw}{dt} = -g - rw$$

where r is the coefficient of resistance

(a) Determine $v(t)$ and $w(t)$ in terms of initial speed u and initial angle of elevation A.

(b) Find $x(t)$ and $y(t)$ if $x(0) = 0$ and $y(0) = h$.

(c) Plot the trajectory of the ball for $r = \frac{1}{5}\ s^{-1}, u = 125$ ft/s and $h = 3$ ft, and for several values of A. How do the trajectories differ from those with $r = 0$?

(d) Assuming that $r = \frac{1}{5}\ s^{-1}$ and $h = 3$ ft, find the minimum initial velocity u and the optimal angle A for which the ball will clear a wall that is 350 ft distant and 10 ft high.

Solution:

(a) Both the ODE for v and w can be solved by multiplying both sides by an integrating

factor.

$$\begin{aligned}
\frac{dv}{dt} &= -rv \\
\frac{dv}{dt} + rv &= 0 \\
e^{rt}\frac{dv}{dt} + re^{rt}v &= 0 \\
\frac{d}{dt}(e^{rt}v) &= 0 \\
e^{rt}v &= C_1 \\
v(t) &= C_1 e^{-rt}
\end{aligned}$$

$$\begin{aligned}
\frac{dw}{dt} &= -g - rw \\
\frac{dw}{dt} + rw &= -g \\
e^{rt}\frac{dw}{dt} + re^{rt}w &= -ge^{rt} \\
\frac{d}{dt}(e^{rt}w) &= -ge^{rt} \\
e^{rt}w &= -\frac{g}{r}e^{rt} + C_2 \\
w(t) &= -\frac{g}{r} + C_2 e^{-rt}
\end{aligned}$$

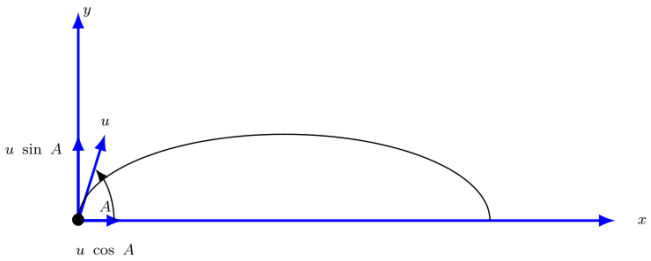

Figure 5.21: *A schematic of the baseball being launched with initial speed u at an angle A, the initial velocity then being decomposed into its components along the x- and y- axes.*

We can then use the initial velocity to determine C_1 and C_2.

$$\begin{aligned}
v(0) &= u\cos\ A \\
C_1 e^{-r(0)} &= u\cos\ A \\
C_1 &= u\cos\ A
\end{aligned}$$

$$\begin{aligned}
w(0) &= u\ \sin\ A \\
-\frac{g}{r} + C_2 e^{-r(0)} &= u\sin\ A \\
C_2 &= \frac{g}{r} + u\sin\ A
\end{aligned}$$

Therefore,

$$\begin{aligned} v(t) &= ue^{-rt}\cos\ A \\ w(t) &= -\frac{g}{r} + \left(\frac{g}{r} + u\sin\ A\right)e^{-rt} \end{aligned}$$

(b) Replace v with $\frac{dx}{dt}$ and replace w with $\frac{dy}{dt}$.

$$\begin{aligned} \frac{dx}{dt} &= ue^{-rt}\cos\ A \\ \frac{dy}{dt} &= -\frac{g}{r} + \left(\frac{g}{r} + u\sin\ A\right)e^{-rt} \end{aligned}$$

Integrate the velocities to get the positions

$$\begin{aligned} x(t) &= -\frac{u}{r}e^{-rt}\cos\ A + C_3 \\ y(t) &= -\frac{g}{r}t - \frac{1}{r}\left(\frac{g}{r} + u\sin\ A\right)e^{-rt} + C_4 \end{aligned}$$

Apply the initial conditions to determine C_3 and C_4.

$$\begin{aligned} x(0) &= 0 \\ -\frac{u}{r}e^{-r(0)}\cos\ A + C_3 &= 0 \\ C_3 &= \frac{u}{r}\cos\ A \end{aligned}$$

$$\begin{aligned} y(0) &= h \\ -\frac{g}{r}(0) - \frac{1}{r}\left(\frac{g}{r} + u\sin\ A\right)e^{-r(0)} + C_4 &= h \\ C_4 &= h + \frac{1}{r}\left(\frac{g}{r} + u\sin\ A\right) \end{aligned}$$

Therefore,

$$\begin{aligned} x(t) &= -\frac{u}{r}(1 - e^{-rt})\cos\ A \\ y(t) &= h - \frac{g}{r}t + \frac{1}{r}\left(\frac{g}{r} + u\sin\ A\right)(1 - e^{-rt}) \end{aligned}$$

(c) The trajectory plot with air drag:

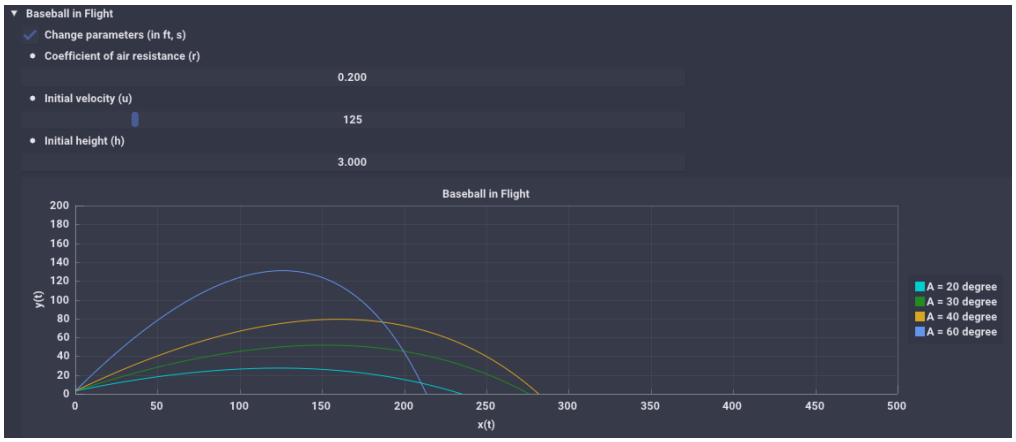

Figure 5.22: *The interactive plots is a parametric plot of $(x(t), y(t))$ with air drag for the case of baseball in flight with Hamzstlab Mathematics. (**Hamzstlab-Mathematics/examples/First Order Differential Equation/-main.cpp**).*

for comparison, without air drag:

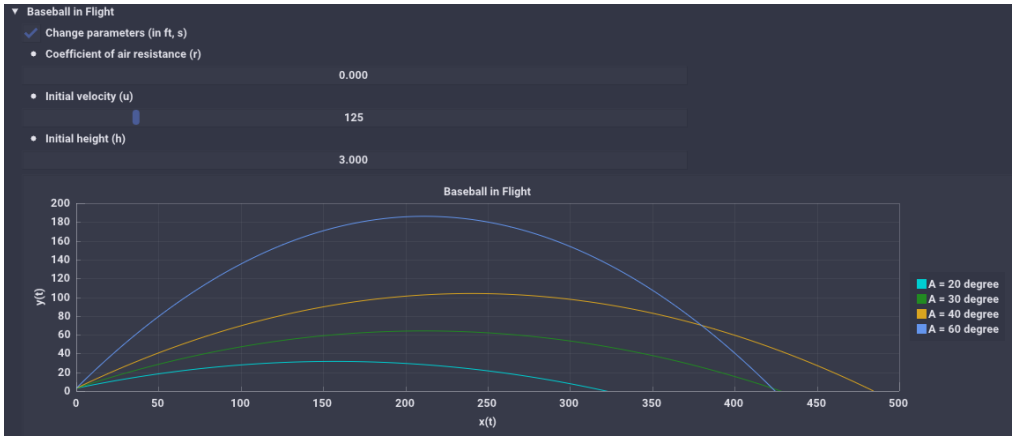

Figure 5.23: *The interactive plots is a parametric plot of $(x(t), y(t))$ without air drag for the case of baseball in flight with Hamzstlab Mathematics. (**Hamzstlab-Mathematics/examples/First Order Differential Equation/main.cpp**).*

(d) Suppose the outfield wall is at a distance L and has height H.
From the result in part (b), we set $x(t) = L$ and $y(t) = H$.

$$L = -\frac{u}{r}(1 - e^{-rt}) \cos\ A$$
$$H = h - \frac{g}{r}t + \frac{1}{r}\left(\frac{g}{r} + u \sin\ A\right)(1 - e^{-rt})$$

Now, We will set $x(t) = 350$ ft, $y(t) = 10$ ft, $r = \frac{1}{5}s^{-1}$, $h = 3$ ft, and $g = 32\ ft/s^2$.

$$350 = 5u(1 - e^{-\frac{t}{5}} \cos\ A)$$
$$10 = 3 - 160t + 5(160 + u \sin\ A)(1 - e^{-\frac{t}{5}})$$

Since we want a formula involving u and A, we will eliminate t. Solve the first equation for $e^{-\frac{t}{5}}$ and t.

$$\begin{aligned} 350 &= 5u(1 - e^{-\frac{t}{5}} \cos\ A) \\ e^{-\frac{t}{5}} &= 1 - \frac{350}{5u \cos\ A} \\ t &= -5 \ln \left(1 - \frac{350}{5u \cos\ A}\right) \end{aligned}$$

and then plug these formulas into the second equation to eliminate t.

$$\begin{aligned} 10 &= 3 + 800 \ln \left(1 - \frac{350}{5u \cos\ A}\right) + 5(160 + u \sin\ A) \left(\frac{350}{5u \cos\ A}\right) \\ 7 &= 800 \ln \left(1 - \frac{70}{u \cos\ A}\right) + \frac{56000}{u \cos\ A} + 350 \tan\ A \end{aligned}$$

In order to clear the wall that is 350 ft distant and 10 ft high, we have to find the solution for this function of 2 variables:

$$f(u, A) = 800 \ln \left(1 - \frac{70}{u \cos\ A}\right) + \frac{56000}{u \cos\ A} + 350 \tan\ A - 7$$

We will use downhill simplex method for $-f(u, A)$ instead of $f(u, A)$, because if we minimize $f(u, A)$ then the ball will never clear the wall as it will looks for minimum value of $f(u, A)$.

When we apply the downhill simplex to

$$-f(u, A) = -\left(800 \ln \left(1 - \frac{70}{u \cos\ A}\right) + \frac{56000}{u \cos\ A} + 350 \tan\ A - 7\right)$$

It is enough until the iteration showing the value of $-f(u, A)$ surpassing zero to the negative value. This is the logical explanation, we want to have this condition:

$$f(u, A) \geq 0$$

downhill simplex will find the value of u and A that will make the value of $f(u, A)$ minimum, that is, if $f(u, A) = 0$ or if $f(u, A) < 0$ then the condition will be fulfilled. So to make the downhill simplex working properly in this context we apply it to $-f(u, A)$ to have this condition:

$$-f(u, A) < 0$$

the more negative the value of $-f(u, A)$ is better, meaning the ball will surpass the wall more than expected, like doubling the home run distance. But we will only need the value of u and A when the downhill simplex compute the value when $-f(u, A)$ barely pass 0 towards negative value. This is the minimum value for u and optimum angle for A (the explanation for the downhill simplex method is available at chapter 11 in this book). We use this method because computing the derivative of $f(u, A)$ is not easy.

After 20 iterations we obtain the vector

$$\begin{bmatrix} u \\ A \end{bmatrix} = \begin{bmatrix} 145.381 \\ 0.6375 \end{bmatrix}$$

as the minimum value for u and A(in radian) to clear the wall. We can of course add the number of iteration or change the initial value when computing with downhill simplex method to obtain the value that is negative and near zero, but this is enough for now. Just to demonstrate the technique to solve this problem with downhill simplex method.

Now, if we input $u = 145.381$ and $A = 0.6375$ radian (equal to 36.526^0) we will obtain

$$f(u, A) = 0.18157$$

which satisfy the requirement to clear the wall of 350 ft distant and 10 ft high.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test Downhill Simplex with Eigen for Baseball in Flight ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb  main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test Downhill Simplex with Eigen for Baseball in Flight ]# ./main
f(x,y) = -800*ln(-70*x^(-1)*cos(y)^(-1)+1)-56000*x^(-1)*cos(y)^(-1)-350*sin(y)*cos(y)^(-1)+7

Total iteration:         20
Reflection:              3
Expansion:               7
Outside contraction:     0
Inside contraction:      10
Shrink:                  0

x_{0}:
145.431
 0.6375
x_{1}:
145.403
0.64375
x_{2}:
145.381
 0.6375

f(x,y) from x_{0}= -0.329485723352
f(x,y) from x_{1}= -0.182805997698
f(x,y) from x_{2}= -0.574296479311
root [ ~/SourceCodes/CPP/C++ Create Library/Test Downhill Simplex with Eigen for Baseball in Flight ]#
```

Figure 5.24: *The computation of the minimum velocity u and optimum angle A to clear a wall of 350 ft distant and 10 ft high for the baseball in flight case with downhill simplex method. (**SymIntegration/Examples/Test Downhill Simplex with Eigen for Baseball in Flight/main.cpp**).*

[H*] **Brachistochrone Example:**
One of the famous problems in the history of mathematics is the brachistochrone problem; to find the curve along which a particle will slide without friction in the minimum time from one given point P to another Q, the second point being lower than the first but not directly beneath it. This problem was posed by Johann bernoulli in 1696 as a challenge problem to the mathematicians of his day. Correct solutions were found by Johann Bernoulli and his brother Jakob Bernoulli and by Isaac Newton, Gottfried Leibniz, and the Marquis de L'Hospital. The brachistochrone problem is important in the development of mathematics as one of the forerunners of the calculus of variations.

In solving this problem, it is convenient to take the origin as the upper point P and to orient the axes. The lower point Q has coordinates (x_0, y_0). It is then possible to show that the curve of minimum time is given by a function $y = \phi(x)$ that satisfies the differential equation

$$(1 + y'^2)y = k^2 \tag{5.30}$$

where k^2 is a certain positive constant to be determined later.

(a) Solve Eq. (5.30) for y'.Why is it necessary to choose the positive square root?

(b) Introduce the new variable t by the relation

$$y = k^2 \sin^2 t \tag{5.31}$$

Show that the equation found in part (a) then takes the form

$$2k^2 \sin^2 t \, dt = dx \tag{5.32}$$

(c) Letting $\theta = 2t$, show that the solution of Eq. (5.32) for which $x = 0$ when $y = 0$ is given by

$$\begin{aligned} x &= k^2 \frac{\theta - \sin\ \theta}{2} \\ y &= k^2 \frac{1 - \cos\ \theta}{2} \end{aligned} \tag{5.33}$$

Equations (5.33) are parametric equations of the solution of Eq. (5.30) that passes through $(0, 0)$. The graphs of Eqs. (5.33) is called a cycloid.

(d) If we make a proper choice of the constant k, then the cycloid also passes through the point (x_0, y_0) and is the solution of the brachistochrone problem. Find k if $x_0 = 1$ and $y_0 = 2$.

Solution:

(a) We will have

$$\begin{aligned} (1 + y'^2)y &= k^2 \\ 1 + y'^2 &= \frac{k^2}{y} \\ y'^2 &= \frac{k^2}{y} - 1 \\ y' &= \pm\sqrt{\frac{k^2}{y} - 1} \end{aligned}$$

We choose the positive sign for y' (the slope) because Q is lower than P and the positive y-axis points downward. Therefore

$$y' = \sqrt{\frac{k^2}{y} - 1}$$

(b) Let $y = k^2 \sin^2 t$. Differentiate both siddes with respect to t then use chain rule for

$\frac{dy}{dt}$

$$\begin{aligned}
\frac{dy}{dt} &= 2k^2 \sin\ t\ \cos\ t \\
\frac{dy}{dx}\frac{dx}{dt} &= 2k^2 \sin\ t\ \cos\ t \\
\sqrt{\frac{k^2}{y} - 1}\frac{dx}{dt} &= 2k^2 \sin\ t\ \cos\ t \\
\sqrt{\frac{k^2}{k^2 \sin^2 t} - 1}\frac{dx}{dt} &= 2k^2 \sin\ t\ \cos\ t \\
\sqrt{\csc^2 t - 1}\frac{dx}{dt} &= 2k^2 \sin\ t\ \cos\ t \\
\sqrt{\cot^2 t}\frac{dx}{dt} &= 2k^2 \sin\ t\ \cos\ t \\
\cot\ t\frac{dx}{dt} &= 2k^2 \sin\ t\ \cos\ t \\
\frac{dx}{dt} &= 2k^2 \sin^2 t \\
dx &= 2k^2 \sin^2 t\ dt
\end{aligned}$$

(c) Continue from the result of part (b)

$$\begin{aligned}
dx &= 2k^2 \sin^2 t\ dt \\
x &= \int 2k^2 \sin^2 t\ dt \\
&= 2k^2 \int \sin^2 t\ dt \\
&= 2k^2 \int \frac{1}{2}(1 - \cos\ 2t)\ dt \\
&= k^2 \left(t - \frac{1}{2} \sin\ 2t \right) + C_1
\end{aligned}$$

We also have

$$\begin{aligned}
y &= k^2 \sin^2 t \\
&= \frac{k^2}{2}(1 - \cos\ 2t)
\end{aligned}$$

In order to have $x = 0$ when $y = 0$ (that is, $t = 0$), C_1 must be zero:

$$C_1 = 0$$

so

$$\begin{aligned}
x(t) &= k^2 \left(t - \frac{1}{2} \sin\ 2t \right) \\
y(t) &= \frac{k^2}{2}(1 - \cos\ 2t)
\end{aligned}$$

Now replace $\theta = 2t$ to get the desired result

$$x(t) = k^2 \left(\frac{\theta}{2} - \frac{1}{2} \sin\ \theta \right)$$
$$y(t) = \frac{k^2}{2}(1 - \cos\ \theta)$$

Therefore,

$$x(t) = \frac{k^2}{2} (\theta - \sin\ \theta)$$
$$y(t) = \frac{k^2}{2}(1 - \cos\ \theta)$$

(d) The cycloid has to go through the point $(x = 1, y = 2)$ at some value of θ, say θ_0. Solve the resulting system of equation for k and θ_0.

$$\begin{aligned} 1 &= \frac{k^2}{2} (\theta_0 - \sin\ \theta_0) \\ 2 &= \frac{k^2}{2}(1 - \cos\ \theta_0) \end{aligned} \tag{5.34}$$

Now to solve the equations above for k and θ_0

$$\begin{aligned} 2x(t) &= y(t) \\ k^2(\theta_0 - \sin\ \theta_0) &= \frac{k^2}{2}(1 - \cos\ \theta_0) \\ 2\theta_0 - 2\sin\ \theta_0 &= 1 - \cos\ \theta_0 \\ 2\theta_0 - 2\sin\ \theta_0 + \cos\ \theta_0 - 1 &= 0 \end{aligned}$$

We can then find the root for the equation above to determine θ_0. There are two ways to find θ_0:

- Find the intersection point between $2\theta_0 - 2\sin\ \theta_0$ and $1 - \cos\ \ theta_0$, we can do this with **solve** function.
- Find the root of $2\theta_0 - 2\sin\ \theta_0 + \cos\ \theta_0 - 1$, this is easier since we can apply numerical method here, e.g. bisection method.

Either way will produce the same result which is

$$\theta_0 \approx 1.40149$$

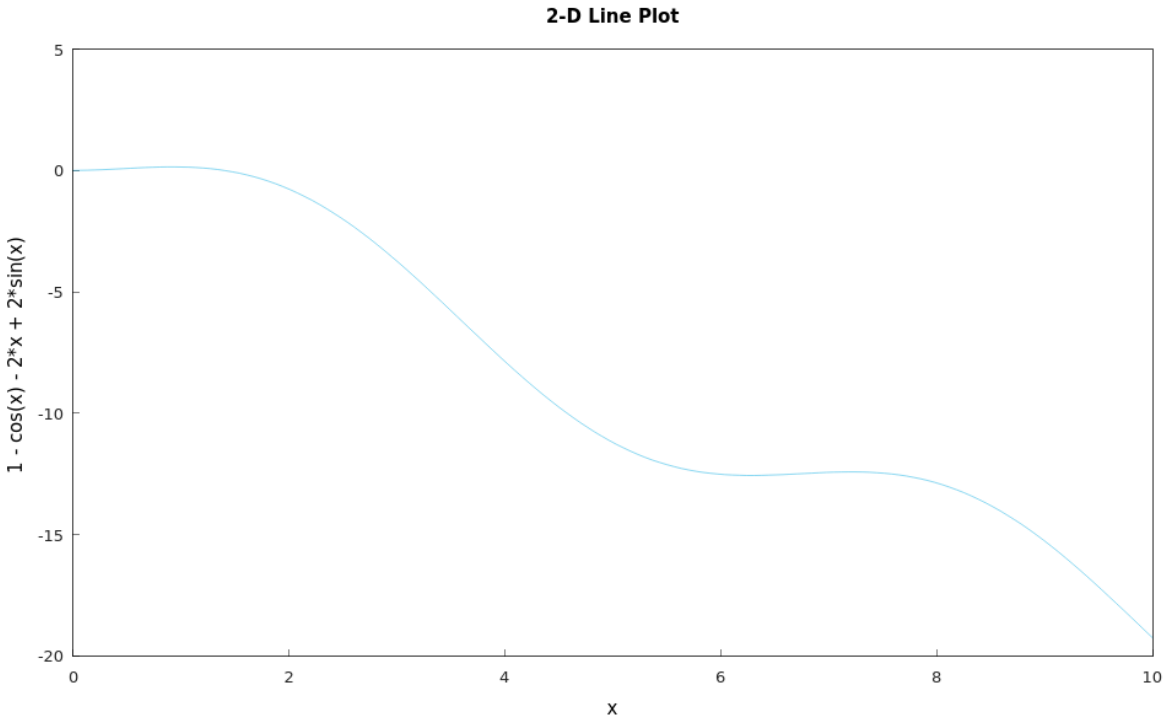

Figure 5.25: *The graph of* $f(\theta_0) = 2\theta_0 - 2\sin\ \theta_0 + \cos\ \theta_0 - 1$ *(**Hamzstplot/Examples with Makefile/Plot 2D Brachistochrone/main.cpp**).*

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DSolve Brachisto
chrone ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DSolve Brachistochrone ]# ./main
f(t,y) = y'^(2)*y+y-k^(2)

Solve for y'

y' = [ y' == 1/2*(-4*y^(2)+4*y*k^(2))^(1/2)*y^(-1),
  y' == -1/2*(-4*y^(2)+4*y*k^(2))^(1/2)*y^(-1) ]

Take the positive root only.

y' = 1/2*(-4*y^(2)+4*y*k^(2))^(1/2)*y^(-1)

dy/dt = 2*k^(2)*sin(t)*cos(t)

dy/dx = cos(t)*(csc(t))

dx/dt = -k^(2)*cos(2*t)+k^(2)

x(t) = -0.5*k^(2)*sin(2*t)+k^(2)*t
y(t) = -0.5*k^(2)*cos(2*t)+0.5*k^(2)

x(t) = -0.5*k^(2)*sin(θ)+0.5*k^(2)*θ
y(t) = -0.5*k^(2)*cos(θ)+0.5*k^(2)

The cycloid has to go through the point (x=1, y=2) at some value of θ, say θ0.

1 = -0.5*k^(2)*sin(θ)+0.5*k^(2)*θ
2 = -0.5*k^(2)*cos(θ)+0.5*k^(2)

-k^(2)*sin(θ)+k^(2)*θ = -0.5*k^(2)*cos(θ)+0.5*k^(2)
f(θ) = -2*sin(θ)+2*θ+cos(θ)-1
k = 2.19648

Time taken by function: 2143224 microseconds
```

Figure 5.26: *The computation to obtain the solution for Brachistochrone:* $x(t)$ *and* $y(t)$ *and then to compute the (**SymIntegration/Examples/Test SymIntegration DSolve Brachistochrone/main.cpp**).*

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Bisection Method with Function ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Bisection Method with Function ]# ./main
f = -cos(θ0)-2*θ0+2*sin(θ0)+1
iteration             a                     b                     p                     f(p)
     1                1                     2                     1.5                   -0.0757472

     2                1                     1.5                   1.25                  0.0826469

     3                1.25                  1.5                   1.375                 0.0172384

     4                1.375                 1.5                   1.4375                -0.0256436

     5                1.375                 1.4375                1.40625               -0.00331926

     6                1.375                 1.40625               1.39062               0.00717788

     7                1.39062               1.40625               1.39844               0.00198421

     8                1.39844               1.40625               1.40234               -0.000653763

     9                1.39844               1.40234               1.40039               0.000668658

    10                1.40039               1.40234               1.40137               8.30682e-06

    11                1.40137               1.40234               1.40186               -0.000322513

    12                1.40137               1.40186               1.40161               -0.00015705

    13                1.40137               1.40161               1.40149               -7.43579e-05
Procedure completed successfully
solution = 1.40149

Time taken by function: 30471 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Bisection Method with Function ]#
```

Figure 5.27: *The computation to obtain θ_0 from function $f(\theta_0) = 2\theta_0 - 2\sin\ \theta_0 + \cos\ \ theta_0 - 1$ with bisection method (**SymIntegration/Examples/Test SymIntegration Bisection Method with Function for Brachistochrone/main.cpp**).*

Now, we can plug $\theta_0 = 1.4$ into one of the equation from Eqs. (5.34) to determine k

$$
\begin{aligned}
2 &\approx \frac{k^2}{2}(1 - \cos\ 1.4) \\
k &\approx \sqrt{\frac{4}{1 - \cos\ 1.4}} \\
k &\approx 2.193
\end{aligned}
$$

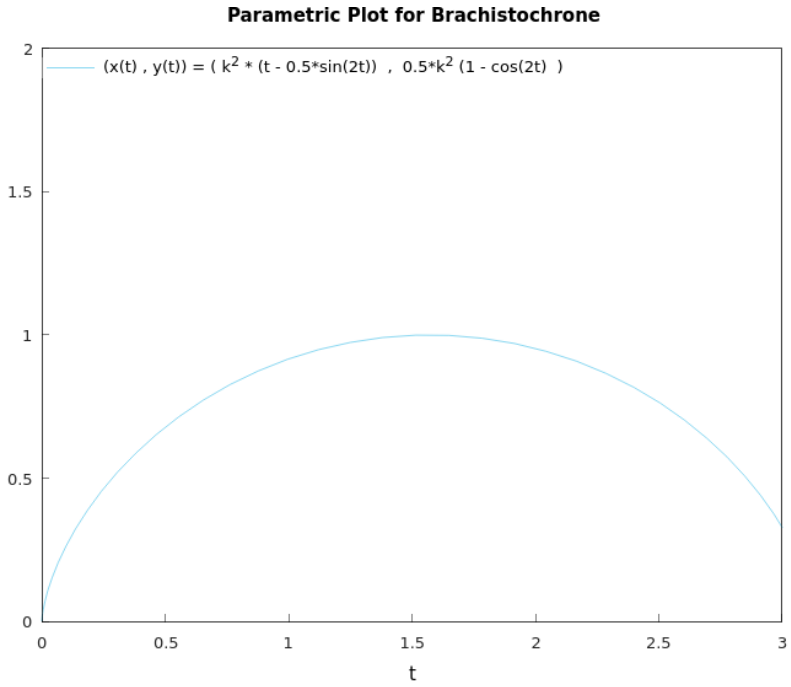

Figure 5.28: *The parametric plot of* $(x(t), y(t)) = \left(k^2\left(t - \frac{1}{2}\sin 2t\right), \frac{k^2}{2}(1 - \cos 2t)\right)$ *(**Hamzstplot/Examples with Makefile/Plot 2D Parametric Plot for Brachistochrone/main.cpp**).*

- Differences Between Linear and Nonlinear Equations

[H*] If you encounter an initial value problem in linear differential equation, you are guaranteed that a solution exists.

Theorem 5.1: Existence and Uniqueness of Solutions for Linear Differential Equation

If the functions p and g are continuous on an open interval $I : \alpha < t < \beta$ containing point $t = t_0$, then there exists a unique function $y = \phi(t)$ that satisfies the differential equation

$$y' + p(t)y = g(t) \tag{5.35}$$

for each t in I, and that also satisfies the initial condition

$$y(t_0) = y_0 \tag{5.36}$$

where y_0 is an arbitrary prescribed initial value.

The theorem states that the solution exists throughout any interval I containing the initial point t_0 in which the coefficients p and g are continuous.

The solution can be discontinuous or fail to exist only at points where at least one of p and g is discontinuous.

[H*] For the nonlinear differential equations, we will use more general theorem

Theorem 5.2: Existence and Uniqueness of Solutions for Nonlinear Differential Equations

Let the functions f and $\frac{\partial f}{\partial y}$ be continuous in some rectangle $\alpha < t < \beta, \gamma < y < \delta$ containing the point (t_0, y_0). Then, in some interval $t_0 - h < t < t_0 + h$ contained in $\alpha < t < \beta$, there is a unique solution $y = \phi(t)$ of the initial value problem

$$y' = f(t, y), \quad y(t_0) = y_0 \tag{5.37}$$

[H*] **Example:**
Solve the initial value problem

$$\frac{dy}{dx} = \frac{3x^2 + 4x + 2}{2(y-1)}, \quad y(0) = -1$$

The differential equation is nonlinear, so

$$f(x, y) = \frac{3x^2 + 4x + 2}{2(y-1)}$$
$$\frac{\partial f}{\partial y}(x, y) = -\frac{3x^2 + 4x + 2}{2(y-1)^2}$$

Thus each of these functions is continuous everywhere except on the line $y = 1$. Consequently, a rectangle can be drawn about the initial point $(0, -1)$ in which both f and $\frac{\partial f}{\partial y}$ are continuous.

The theorem guarantes that the initial value problem has a unique solution in some interval about $x = 0$. However, even though the rectangle can be stretched infinitely far in both the positive and negative x directions, this does not necessarily mean that the solution exists for all x.

We can use separable equations technique

$$\frac{dy}{dx} = \frac{3x^2 + 4x + 2}{2(y-1)}$$
$$2(y-1)dy = (3x^2 + 4x + 2)dx$$
$$y^2 - 2y = x^3 + 2x^2 + 2x + c$$

Now to solve the initial value problem we substitute $x = 0$ and $y = -1$ so we will obtain $c = 3$, thus

$$y^2 - 2y - (x^3 + 2x^2 + 2x + 3) = 0$$
$$y = \frac{-(-2) \pm \sqrt{(-2)^2 - 4(1)(-(x^3 + 2x^2 + 2x + 3))}}{2(1)}$$
$$y = \frac{2 \pm \sqrt{4 + (4x^3 + 8x^2 + 8x + 12)}}{2}$$
$$y = 1 \pm \sqrt{x^3 + 2x^2 + 2x + 4}$$

To choose the plus or minus sign we need to check back again with the initial condition.

$$y(0) = 1 - \sqrt{(0)^3 + 2(0)^2 + 2(0) + 4}$$
$$y(0) = -1$$

so we choose the minus sign for the solution. The initial value problem of $\frac{dy}{dx} = \frac{3x^2+4x+2}{2(y-1)}$, $y(0) = -1$ has the solution of

$$y(x) = 1 - \sqrt{x^3 + 2x^2 + 2x + 4} \tag{5.38}$$

We must find the interval in which the quantity under the radical is positive (under the square root), since if it is negative then it will return complex number. The only real zero of this expression is $x = -2$, so the desired interval is $x > -2$, that means the solution exists only for $x > -2$.

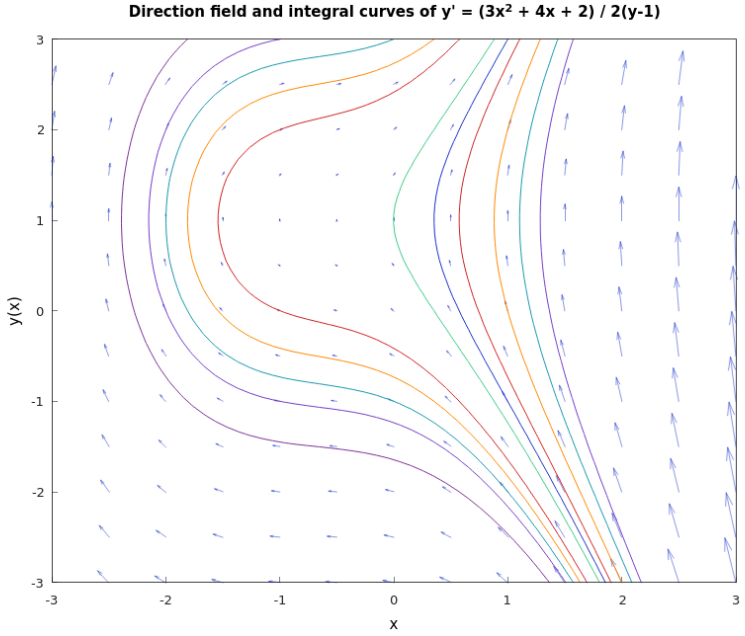

Figure 5.29: *The direction field and integral curves of* $\frac{dy}{dx} = \frac{3x^2+4x+2}{2(y-1)}$ *(**Hamzstplot/Examples with Makefile/Plot First Order Differential Equation Direction Field and Solutions Curves for Separable Equations Example 2/main.cpp**).*

[H*] **Bernoulli Equations**

Sometimes it is possible to solve a nonlinear equation by making a change of the dependent variable that converts it into a linear equation. The most important such equation has the form

$$y' + p(t)y = q(t)y^n \tag{5.39}$$

it is called a Bernoulli equation after Jakob Bernoulli.

[H*] **Example:**

(a) Solve Bernoulli' equation when $n = 0$ and when $n = 1$.

(b) Show that if $n \neq 0, 1$, then the substitution $v = y^{1-n}$ reduces Bernoulli's equation to a linear equation. This method of solution was found by Leibniz in 1696.

Solution:

(a) For $n = 0$ we will have

$$y' + p(t)y = q(t)$$

it is linear, so the solution is

$$y = \frac{1}{\mu(t)} \left[\int_{t_0}^{t} \mu(s)q(s)\ ds + c \right] \tag{5.40}$$

with

$$\mu(t) = e^{\int p(t)\ dt} \tag{5.41}$$

For $n = 1$ we will have

$$y' + p(t)y = q(t)y$$

it is linear again, so the solution is

$$y = \frac{1}{\mu(t)} [c] \tag{5.42}$$

with

$$\mu(t) = e^{\int p(t) - q(t)\ dt} \tag{5.43}$$

(b) Let $v = y^{1-n}$, then

$$\frac{dv}{dt} = (1-n)y^{-n}\frac{dy}{dt}$$
$$\frac{dy}{dt} = \frac{1}{1-n}y^{n}\frac{dv}{dt}$$

which makes sense when $n \neq 0, 1$. Substituting into the differential equation yields

$$\begin{aligned} \frac{y^n}{1-n}\frac{dv}{dt} + p(t)y &= q(t)y^n \\ v' + (1-n)p(t)y^{1-n} &= (1-n)q(t) \end{aligned} \tag{5.44}$$

Setting $v = y^{1-n}$ then yields a linear differential equation for v. The Eq. (5.39) can be solved with method of integrating factors, it can also be written like this

$$v' + (1-n)p(t)v = (1-n)q(t) \tag{5.45}$$

[H*] **Example:**

The given equation is Bernoulli equation. In each case solve it by using the substitution method found by Leibniz in 1696.

(a) $y' = \epsilon y - \sigma y^3$, $\epsilon > 0$ and $\sigma > 0$. This equation occurs in the study of the stability of fluid flow.

(b) $y' = (\Gamma \cos\ t + T)y - y^3$, where Γ and T are constants. This equation also occurs in the study of the stability of fluid flow.

Solution:

(a) Since $n = 3$, set $v = y^{-2}$. It follows that

$$\frac{dv}{dt} = -2y^{-3}\frac{dy}{dt}$$
$$\frac{dy}{dt} = -\frac{y^3}{2}\frac{dv}{dt}$$

Substituting into the differential equation yields

$$y' = \epsilon y - \sigma y^3$$
$$-\frac{y^3}{2}\frac{dv}{dt} = \epsilon y - \sigma y^3$$
$$-\frac{y^3}{2}\frac{dv}{dt} - \epsilon y = -\sigma y^3$$
$$\frac{dv}{dt} + 2\epsilon y^{-2} = 2\sigma$$
$$v' + 2\epsilon v = 2\sigma$$

This differential equation $v' + 2\epsilon v = 2\sigma$ is linear, and can be written as

$$(v \cdot e^{2\epsilon t})' = 2\sigma e^{2\epsilon t} \tag{5.46}$$

with the integrating factor $\mu(t) = e^{2\epsilon t}$. The solution is given by

$$v(t) = \frac{\sigma}{\epsilon} + ce^{-2\epsilon t} \tag{5.47}$$

Converting back to the original dependent variable, we will have

$$y = \pm v^{-\frac{1}{2}}$$
$$y(t) = \pm\frac{1}{\sqrt{\frac{\sigma}{\epsilon} + ce^{-2\epsilon t}}} \tag{5.48}$$

(b) Since $n = 3$, set $v = y^{-2}$. It follows that

$$\frac{dv}{dt} = -2y^{-3}\frac{dy}{dt}$$
$$\frac{dy}{dt} = -\frac{y^3}{2}\frac{dv}{dt}$$

The differential equation is written as

$$y' = (\Gamma\cos\ t + T)y - y^3$$
$$-\frac{y^3}{2}\frac{dv}{dt} = (\Gamma\cos\ t + T)y - y^3$$
$$-\frac{y^3}{2}\frac{dv}{dt} - (\Gamma\cos\ t + T)y = -y^3$$
$$\frac{dv}{dt} + 2(\Gamma\cos\ t + T)y^{-2} = 2$$
$$v' + 2(\Gamma\cos\ t + T)v = 2$$

This ordinary differential equation is linear, with integrating factor

$$\mu(t) = e^{2(\Gamma \sin\ t + Tt)}$$

Thus, we will have

$$(v \cdot e^{2(\Gamma \sin\ t + Tt)})' = 2e^{2(\Gamma \sin\ t + Tt)} \tag{5.49}$$

Then we can find the solution with the method of integrating factor

$$v(t) = \frac{1}{\mu(t)} \left[\int_{t_0}^{t} 2e^{2(\Gamma \sin\ s + Ts)}\ ds + c \right] \tag{5.50}$$

the solution is given by

$$v(t) = \frac{1}{e^{2(\Gamma \sin\ t + Tt)}} \left[\int_{t_0}^{t} 2e^{2(\Gamma \sin\ s + Ts)}\ ds + c \right] \tag{5.51}$$

the integral $\int_{t_0}^{t} 2e^{2(\Gamma \sin\ s + Ts)}\ ds$ is not an easy one to be integrated, even with SymPy in 2025, so we will keeping it that way. We can also write it in this form

$$v(t) = (e^{-2(\Gamma \sin\ t + Tt)}) \int_{t_0}^{t} 2e^{2\Gamma \sin\ s} e^{2Ts}\ ds + c(e^{-2(\Gamma \sin\ t + Tt)}) \tag{5.52}$$

- Autonomous Equations and Population Dynamics

[H*] An important class of first order equations consists of those in which the independent variable does not appear explicitly. Such equations are called autonomous and have the form

$$\frac{dy}{dt} = f(y) \tag{5.53}$$

Equation (5.53) is separable, but the main purpose of this section is to show how geometrical methods can be used to obtain important qualitative information directly from the differential equation, without solving the equation.

[H*] **Exponential Growth**

Let $y = \phi(t)$ be the population of the given species at time t. The simplest hypothesis concerning the variation of population is that the rate of change of y is proportional to the current value of y; that is,

$$\frac{dy}{dt} = ry \tag{5.54}$$

where the constant of proportionality r is called the rate of growth or decline, depending on whether it is positive or negative. Here, we assume that $r > 0$, so the population is growing.

Solving Eq. (5.54) subject to the initial condition

$$y(0) = y_0 \tag{5.55}$$

we obtain

$$y = y_0 e^{rt} \tag{5.56}$$

Thus the mathematical model consisting of the initial value problem (5.54), (5.55) with $r > 0$ predicts that the population will grow exponentially for all time. However, it is clear that such ideal conditions cannot continue indefinitely; eventually, limitations on space, food supply, or other resources will reduce the growth rate and bring an end to uninhibited exponential growth.

[H*] **Logistic Growth**

Consider the differential equation that the growth rate actually depends on the population

$$\frac{dy}{dt} = h(y)y \tag{5.57}$$

We now want to choose $h(y)$ so that $h(y) \equiv r > 0$ when y is small, $h(y)$ decreases as y grows larger, and $h(y) < 0$ when y is sufficiently large. The simplest function that has these properties is

$$h(y) = r - ay$$

where a is also a positive constant. Using this function in Eq. (5.57), we obtain

$$\frac{dy}{dt} = (r - ay)y \tag{5.58}$$

Equation (5.58) is known as the Verhulst equation or the logistic equation. It is often convenient to write the logistic equation in the equivalent form

$$\frac{dy}{dt} = r\left(1 - \frac{y}{K}\right)y \tag{5.59}$$

where $K = r/a$. The constant r is called the intrinsic growth rate, that is, the growth rate in the absence of any limiting factors.

We first seek solutions of Eq. (5.59) of the simplest possible type, that is, constant functions.

For such a solution $\frac{dy}{dt} = 0$ for all t, so any constant solution of Eq. (5.59) must satisfy the algebraic equation

$$r\left(1 - \frac{y}{K}\right)y = 0$$

Thus the constant solutions are $y = \phi_1(t) = 0$ and $y = \phi_2(t) = K$. These solutions are called equilibrium solutions of Eq. (5.59) because they correspond to no change or variation in the value of y as t increases.

In the same way, any equilibrium solutions of the more general Eq. (5.53) can be found by locating the roots of $f(y) = 0$. The zeros of $f(y)$ are also called critical points.

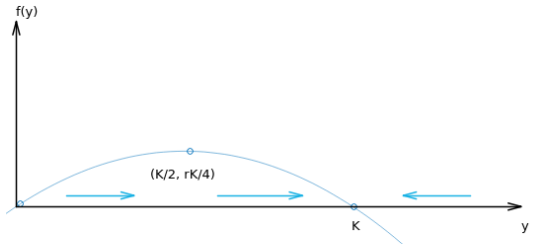

Figure 5.30: *The equilibrium solution $f(y) = 0$ or $r(1 - y/K)y = 0$ to visualize other solutions of Eq. (5.59) (**Hamzstplot/Examples with Makefile/Plot 2D Equilibrium Solution for Logistic Growth/main.cpp**).*

The graph of $f(y)$ versus y above is a parabola with the intercepts are $(0, 0)$ and $(K, 0)$, corresponding to the critical points of Eq. (5.59), and the vertex of the parabola is

$(K/2, rK/4)$.

Observe that $\frac{dy}{dt} > 0$ for $0 < y < K$; therefore, y is an increasing function of t when y is in this interval; this is indicated by the rightward-pointing arrows near the y-axis. Similarly, if $y > K$, then $\frac{dy}{dt} < 0$; hence y is decreasing, as indicated by the leftward-pointing arrow.

The y-axis is often called the phase line, and it is usually drawn with vertical line (rotating it 90 degree counterclockwise).

The dots at $y = 0$ and $y = K$ are the critical points or equilibrium solutions. The arrows again indicate that y is increasing whenever $0 < y < K$ and that y is decreasing whenever $y > K$.

Note that, if y is near zero or K, then the slope $f(y)$ is near zero, so the solution curves are relatively flat. They become steeper as the value of y leaves the neighborhood of zero or K.

To sketch the graphs of solutions of Eq. (5.59) in the ty-plane, we start with the equilibrium solutions $y = 0$ and $y = K$; then we draw other curves that are increasing when $0 < y < K$, decreasing when $y > K$, and flatten out as y approaches either the values 0 or K. Thus, the graphs of solutions of Eq. (5.59) must have the general shape regardless of the values of r and K.

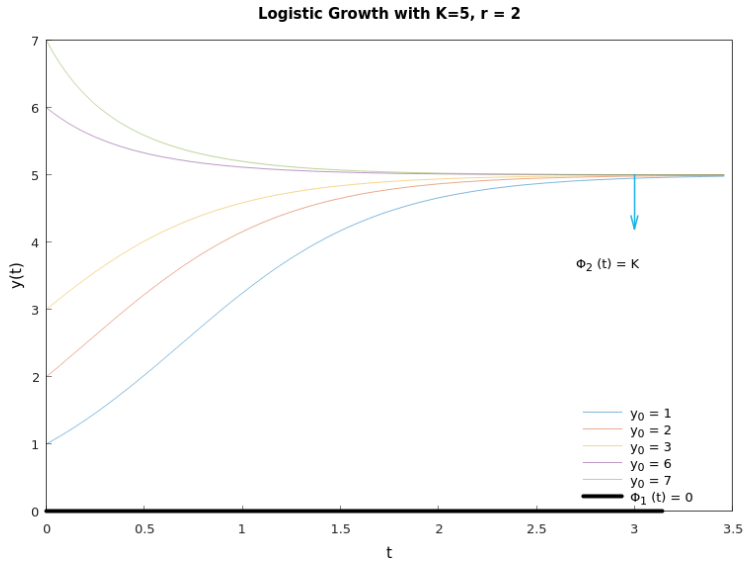

Figure 5.31: *The solution curves for $y(t) = \frac{y_0 K}{y_0 + (K - y_0)e^{-rt}}$ to visualize all possible solutions of Eq. (5.59) with different y_0 (**Hamzstplot/Examples with Makefile/Plot 2D Solution Curves for Logistic Growth/-main.cpp**).*

It may seems that other solutions intersect the equilibrium solution at $y(t) = K$ in Figure (5.31), but it is not possible.

Based on the uniqueness part of Theorem 4.2, the fundamental existence and uniqueness

theorem, states that only one solution can pass through a given point in the ty-plane. Thus, although other solutions may be asymptotic to the equilibrium solution as $t \to \infty$, they cannot intersect it at any finite time.

We can determine the concavity of the solution curves and the location of inflection points by finding $\frac{d^2y}{dt^2}$. From the differential equation (5.53) we obtain (using the chain rule)

$$\frac{d^2y}{dt^2} = \frac{d}{dt}\frac{dy}{dt} = \frac{d}{dt}f(y) = f'(y)\frac{dy}{dt} = f'(y)f(y) \tag{5.60}$$

The graph of y versus t is concave up when $y'' > 0$, that is, when f and f' have the same sign.

Similarly, it is concave down when $y'' < 0$, which occurs when f and f' have opposite signs. The signs of f and f' can be easily identified from the graph of $f(y)$ versus y. Inflection points may occur when $f'(y) = 0$.

In the case of Eq. (5.59), solutions are concave up for $0 < y < K/2$ where f is positive and increasing, so that both f and f' are positive. Solutions are also concave up for $y > K$ where f is negative and decreasing (both f and f' are negative). For $K/2 < y < K$, solutions are concave down since here f is positive and decreasing, so f is positive but f' is negative. There is an inflection point whenever the graph of y versus t crosses the line $y = K/2$.

Finally, observe that K is the upper bound that is approached, but not exceeded, by growing populations starting below this value. Thus it is natural to refer to K as the saturation level, or the environmental carrying capacity, for the given species.

It reveals that solutions of the nonlinear equation (5.59) are strikingly different from those of the linear equation (5.53), at least for large values of t. Regardless of the value of K, that is, no matter how small the nonlinear term in Eq. (5.59), solutions of that equation approach a finite values as $t \to \infty$, whereas solutions of Eq. (5.53) grow (exponentially) without bound as $t \to \infty$. Thus, even a tiny nonlinear term in the differential equation (5.59) has a decisive effect on the solution for large t.

In many situations it is sufficient to have the qualitative information about a solution $y = \phi(t)$ of Eq. (5.59). This information was obtained entirely from the graph of $f(y)$ versus y, and without solving the differential equation (5.59). However, if we wish to know the value of the population at some particular time, then we must solve Eq. (5.59) subject to the initial condition (5.55). Provided that $y \neq 0$ and $y \neq K$, we can write Eq. (5.59) in the form

$$\begin{aligned} \frac{dy}{\left(1 - \frac{y}{K}\right) y} &= r\, dt \\ \left(\frac{1}{y} - \frac{1/K}{1 - y/K}\right) dy &= r\, dt \\ \ln|y| - \ln\left|1 - \frac{y}{K}\right| &= rt + c \end{aligned} \tag{5.61}$$

where c is an arbitrary constant of integration to be determined from the initial condition

$y(0) = y_0$. We have already noted that if $0 < y_0 < K$, then y remains in this interval for all time. Thus in this case we can remove the absolute value bars in Eq. (5.61), and by taking the exponential of both sides, we find that

$$\frac{y}{1-(y/K)} = Ce^{rt} \tag{5.62}$$

where $C = e^c$. In order to satisfy the initial condition $y(0) = y_0$, we must choose $C = \frac{y_0}{1-(y_0/K)}$. Using this value of C in Eq. (5.62) and solving for y, we obtain

$$y(t) = \frac{y_0 K}{y_0 + (K - y_0)e^{-rt}} \tag{5.63}$$

We have derived the solution (5.63) under the assumption that $0 < y_0 < K$. If $y_0 > K$, then the details of dealing with Eq. (5.61) are only slightly different, and Eq, (5.63) is also valid in this case.

Note that Eq. (5.63) also contains the equilibrium solutions $y = \phi_1(t) = 0$ and $y = \phi_2(t) = K$ corresponding to the initial conditions $y_0 = 0$ and $y_0 = K$, respectively.

In particular, if $y_0 = 0$, then Eq. (5.63) requires that $y(t) = 0$ for all t. If $y_0 > 0$, and if we let $t \to \infty$ in Eq. (5.63), then we obtain

$$\lim_{t\to\infty} y(t) = \frac{y_0 K}{y_0} = K$$

Thus, for each $y_0 > 0$, the solution approaches the equilibrium solution $y = \phi_2(t) = K$ asymptotically as $t \to \infty$. Therefore we say that the solution $\phi_2(t) = K$ is an asymptotically stable solution of Eq. (5.59) or that the point $y = K$ is an asymptotically stable equilibrium or critical point.
After a long time, the population is close to the saturation level K regardless of the initial population size, as long as it is positive. Other solutions approach the equilibrium solution more rapidly as r increases.

On the other hand, the situation for the equilibrium solution $y = \phi_1(t) = 0$ is quite different. Even solutions that start very near zero grow as t increases and, as we have seen, approach K as $t \to \infty$. We say that $\phi_1(t) = 0$ is an unstable equilibrium solution or that $y = 0$ is an unstable equilibrium critical point. This means that the only way to guarantee that the solution remains near zero is to make sure its initial value is exactly equal to zero.

If we want to find the time, t, when the population reach certain level $y(t)$, e.g. when the tuna population in Pacific Ocean reach 85% ($y(t) = 0.85K$) of its maximum limit that the environment can carry, then we will use this formula from derivating of Eq. (5.63):

$$t = -\frac{1}{r} \ln \frac{\left(\frac{y_0}{K}\right)\left[1 - \left(\frac{y}{K}\right)\right]}{\left(\frac{y}{K}\right)\left[1 - \left(\frac{y_0}{K}\right)\right]} \tag{5.64}$$

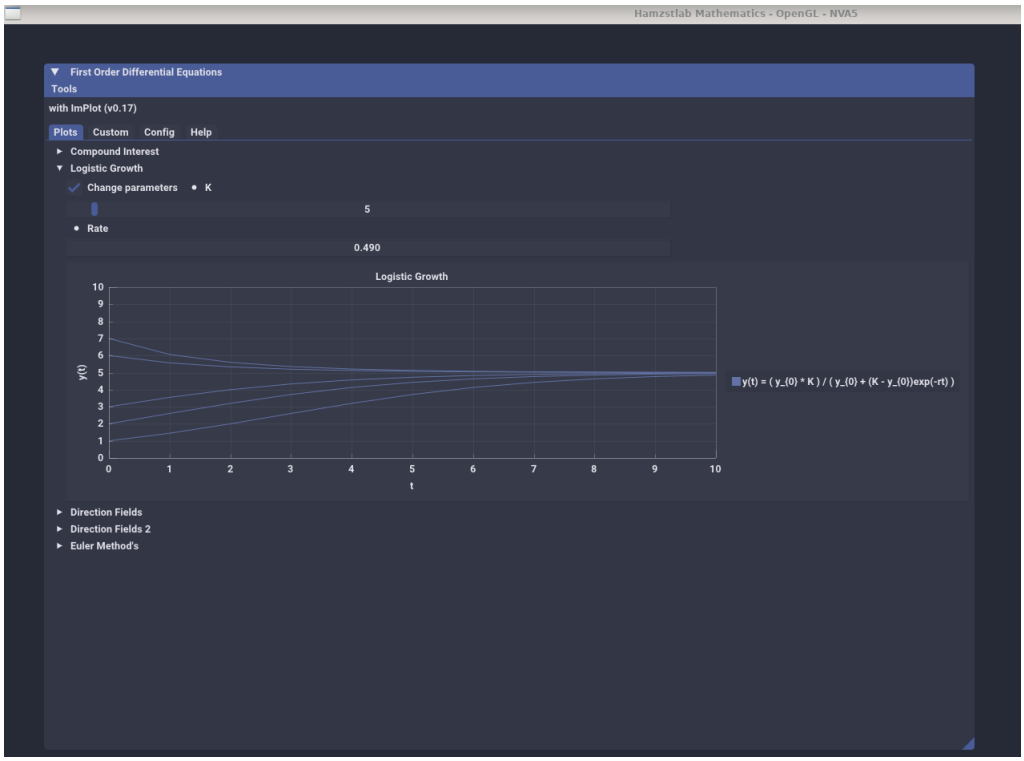

Figure 5.32: *The solution curves for* $y(t) = \frac{y_0 K}{y_0 + (K - y_0)e^{-rt}}$ *we set* y_0 *with fixed values at first and the parameters* K *and* r *can be changed at in instant in Hamzstlab Mathematics (**Hamzstlab-Mathematics/examples/First Order Differential Equation/main.cpp**).*

[H*] **A Critical Threshold**

We now turn to a consideration of the equation

$$\frac{dy}{dt} = -r\left(1 - \frac{y}{T}\right)y \tag{5.65}$$

where r and T are given positive constants. Observe that this equation differs from the logistic equation (5.59) in the presence of the minus sign on the right side, along with T replacing K.

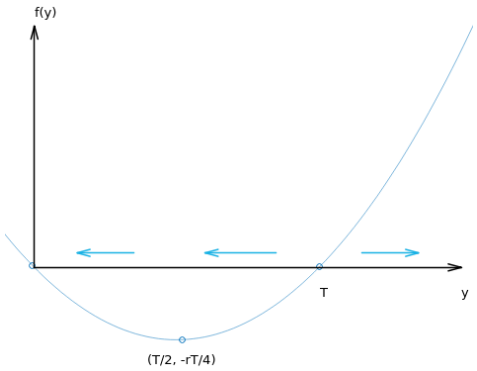

Figure 5.33: *The graph of $f(y)$ for $\frac{dy}{dt} = -r\left(1 - \frac{y}{T}\right)y$ (**Hamzstplot/Examples with Makefile/Plot 2D Equilibrium Solution for Critical Threshold/main.cpp**).*

The intercepts on the y-axis are the critical points $y = 0$ and $y = T$, corresponding to the equilibrium solutions $\phi_1(t) = 0$ and $\phi_2(t) = T$.

The dots at $y = 0$ and $y = T$ are the critical points, or equilibrium solutions, and the arrows indicate where solutions are either increasing or decreasing.

If $0 < y < T$, then $\frac{dy}{dt} < 0$, and y decreases as T increases. Thus $\phi_1(t) = 0$ is an asymptotically stable equilibrium solution and $\phi_2(t) = T$ is an unstable one.

Further, $f'(y)$ is negative for $0 < y < T/2$ and positive for $T/2 < y < T$, so the graph of y versus t is concave up and concave down, respectively, in these intervals. Also, $f'(y)$ is positive for $y > T$, so the graph of y versus t is also concave up here.

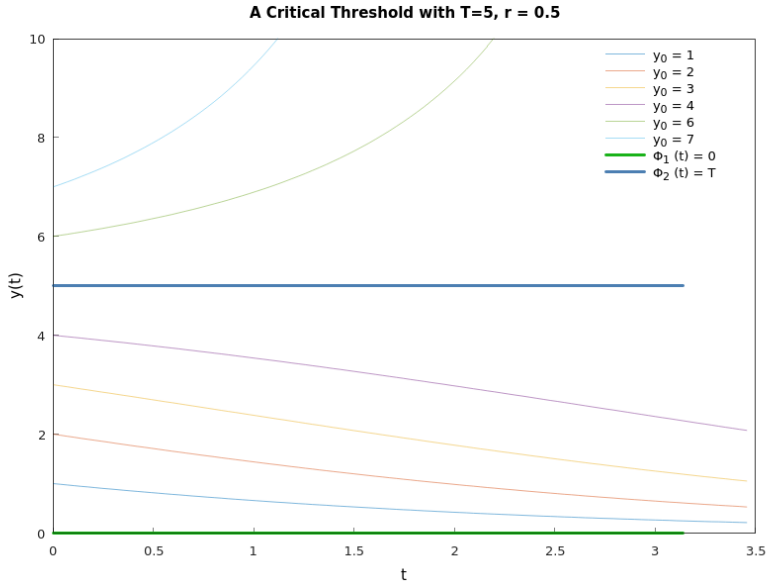

Figure 5.34: *The solution curves for growth with a threshold for $\frac{dy}{dt} = -r\left(1 - \frac{y}{T}\right)y$ (**Hamzstplot/Examples with Makefile/Plot 2D Solution Curves for Critical Threshold/main.cpp**).*

Figure (5.34) shows the solution curves of Eq. (5.65). After we draw the two thick horizontal line at $y = 0$ and $y = T$, we can then sketch the curves in the strip $0 < y < T$ that are decreasing as t increases and change concavity as they cross the line $y = T/2$.

Next draw some curves above $y = T$ that increase more and more steeply as t and y increase. Make sure that all curves become flatter as y approaches either zero or T.

From this figure it appears that as time increases, y either approaches zero or grows without bound, depending on whether the initial value y_0 is less than or greater than T. Thus T is a threshold level, below which growth does not occur.

By solving the differential equation (5.65) with separating the variables and integrating we will obtain

$$y(t) = \frac{y_0 T}{y_0 + (T - y_0)e^{rt}} \tag{5.66}$$

which is the solution of Eq. (5.65) subject to the initial condition $y(0) = y_0$.

If $0 < y_0 < T$, then it follows from Eq. (5.66) that $y \to 0$ as $t \to \infty$. This agrees with our qualitative geometric analysis. If $y_0 > T$, then the denominator on the right side of Eq. (5.66) is zero for a certain finite value of t. We denote this value by t^* and calculate it from

$$y_0 - (y_0 - T)e^{rt^*} = 0$$

which gives

$$t^* = \frac{1}{r} \ln \frac{y_0}{y_0 - T} \tag{5.67}$$

Thus, if the initial population y_0 is above the threshold T, the threshold model predicts that the graph of y versus t has a vertical asymptote at $t = t^*$; in other words, the population becomes unbounded in a finite time, whose value depends on y_0, T, and r. The existence and location of this asymptote were not apparent from the geometric analysis, so in this case the explicit solution yields additional important qualitative, as well as quantitative, information.

The population of some species exhibit the threshold phenomenon. If too few are present, then the species cannot propagate itself successfully and the population becomes extinct. However, if the population is alrger than the threshold level, then further growth occurs. Of course, the population cannot become unbounded, so eventually Eq. (5.65) must be modified to take this into account.

Critical thresholds also occur in other circumstances. For example, in fluid mechanics, equations of the form (5.59) and (5.65) often govern the evolution of a small disturbance y in a laminar (or smooth) fluid flow.

For instance, if Eq. (5.65) holds and $y < T$, then the disturbance is damped out and laminar flow persists.
However, if $y > T$, then the disturbance grows larger and the laminar flow breaks up into a turbulent one. In this case T is referred to as the critical amplitude. Experimenters speak of keeping the disturbance level in a wind tunnel sufficiently low so taht they can study laminar flow over an airfoil, for example.

[H*] **Logistic Growth with a Threshold**
This model comes after we modified the threshold model (5.65) so that unbounded growth does not occur when y is above the threshold T. The simplest way to do this is to introduce another factor that will have the effect of making $\frac{dy}{dt}$ negative when y is large. Thus we consider

$$\frac{dy}{dt} = -r\left(1 - \frac{y}{T}\right)\left(1 - \frac{y}{K}\right)y \tag{5.68}$$

where $r > 0$ and $0 < T < K$.

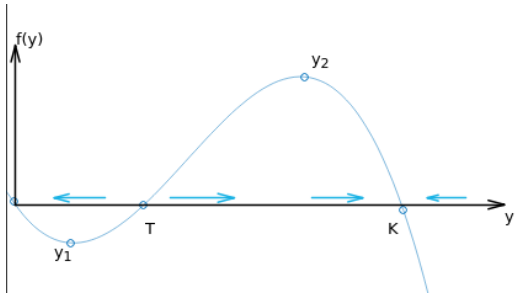

Figure 5.35: *The graph of $f(y)$ versus y for logistic growth with a threshold for $\frac{dy}{dt} = -r\left(1 - \frac{y}{T}\right)\left(1 - \frac{y}{K}\right)y$ (**Hamzstplot/Examples with Makefile/Plot 2D Equilibrium Solution for Logistic Growth with a Threshold/main.cpp**).*

The equilibrium solutions are found by setting $\frac{dy}{dt} = 0$.
In this problem there are three critical points, $y = 0, y = T$, and $y = K$, corresponding to the equilibrium solutions $\phi_1(t) = 0, \phi_2(t) = T$, and $\phi_3(t) = K$, respectively.

From Figure 5.35 we observe that $\frac{dy}{dt} > 0$ for $T < y < K$, and consequently y is increasing there. Further, $\frac{dy}{dt} < 0$ for $y < T$ and for $y > K$, so y is decreasing in these intervals.

Consequently, the equilibrium solutions $\phi_1(t)$ and $\phi_3(t)$ are asymptotically stable, and the solution $\phi_2(t)$ is unstable.

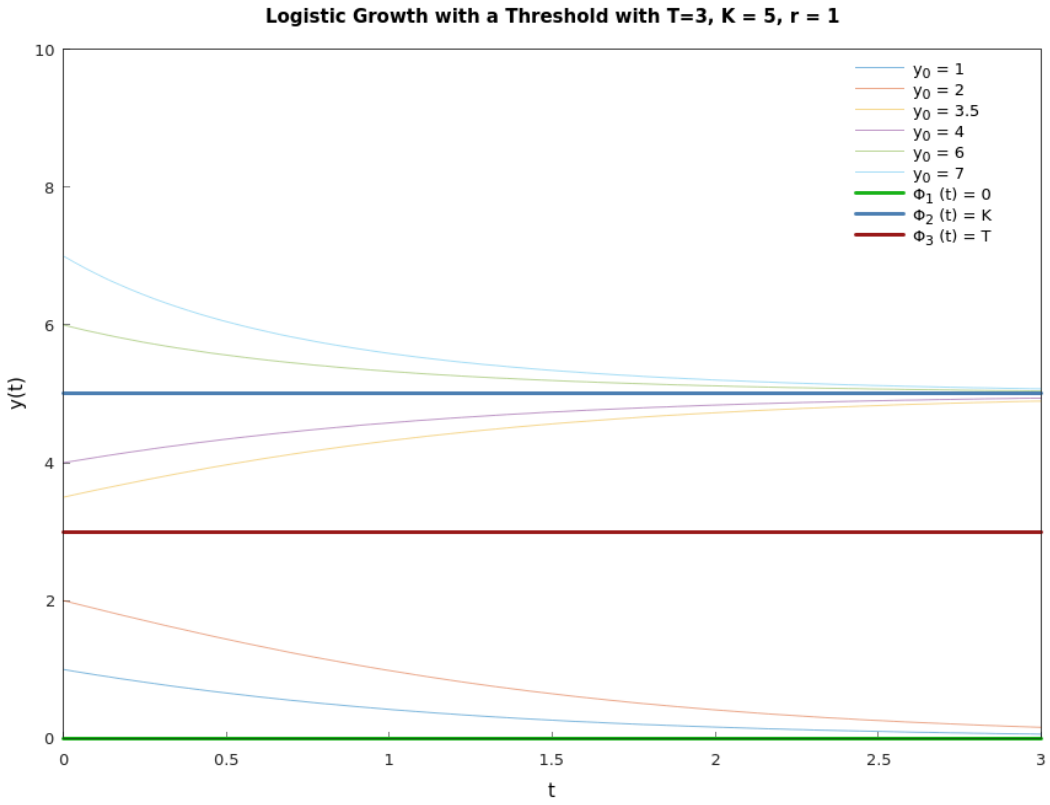

Figure 5.36: *The solution curves for logistic growth with a threshold for* $\frac{dy}{dt} = -r\left(1 - \frac{y}{T}\right)\left(1 - \frac{y}{K}\right)y$ *(**Hamzstplot/Examples with Makefile/Plot 2D Solution Curves for Logistic Growth with a Threshold/main.cpp**).*

You should make sure that you understand the relation between these two Figure (5.35) and Figure (5.36).

We see that if y starts below the threshold T, then y declines to ultimate extinction. On the other hand, if y starts above T, then y eventually approaches the carrying capacity K. The inflection points on the graphs of y versus t in Figure (5.36) correspond to the maximum and minimum points y_1 and y_2, respectively, on the graphy of $f(y)$ versus y in Figure (5.35). These values can be obtained by differentiating the right side of Eq. (5.68) with respect to y, setting the result equal to zero, and solving for y. We obtain

$$y_{1,2} = \frac{\left(K + T \pm \sqrt{K^2 - KT + T^2}\right)}{3} \tag{5.69}$$

where the plus sign yields y_1 and the minus sign yields y_2.

The logistic equation with a threshold population at Eq. (5.68) can be solved using the method of separation of variables. However, it is very difficult to get the solution as an explicit function of t, so we are opting for the implicit function instead.

$$\frac{dy}{dt} = -r\left(1-\frac{y}{T}\right)\left(1-\frac{y}{K}\right)y$$
$$\frac{dy}{\left(1-\frac{y}{T}\right)\left(1-\frac{y}{K}\right)y} = -r\,dt$$
$$\ln|y| - \ln|T-y| - \ln|K-y| = -rt + C$$

Solving for C with the initial value of $y(0) = y_0$, we will obtain

$$C = \ln|y_0| - \ln|T-y_0| - \ln|K-y_0|$$

thus

$$\ln|y(t)| - \ln|T-y(t)| - \ln|K-y(t)| = -rt + \ln|y_0| - \ln|T-y_0| - \ln|K-y_0| \quad (5.70)$$

where

* $y(t)$ represents the population size at time t.
* r is a positive constant representing the intrinsic growth rate.
* T is the threshold population below which the population declines towards extinction.
* K is the carrying capacity, the maximum sustainable population size.
* $0 < T < K$

The good thing is, sometimes you can solve a very complex problem by dividing it into smaller partition and see the pattern at each partition, in C++ code we can use **if.. then** conditional that can be very useful for drawing the solution curves of this type. Without even solving the Eq. (5.68), we can use Eq. (5.63) and Eq. (5.66) under the **if .. then .. if else .. then ..** conditional.

[H*] **Phase Line Analysis**

In order to determine the stability for single variable function, we use phase line analysis.

First, find the equilibrium solutions by finding the root of $f(y) = 0$ from

$$\frac{dy}{dt} = f(y)$$

Then, draw a horizontal line representing the variable (e.g. y). Mark the equilibrium points on the line.

Analyze the sign of the derivative $\frac{dy}{dt}$ on either side of each equilibrium point.

If the derivative changes from positive to negative as y increases, the equilibrium point is a sink (stable).

If it changes from negatives to positive, it's a source (unstable).

If there's no sign change, it's a node (semistable).

To draw the phase line you can just rotate the horizontal line from above counterclockwise 90^0 to become a vertical line.

The benefits of phase line analysis:

1. Phase lines provide a visual understanding of the system's behavior without solving the differential equation.
2. They clearly show which equilibrium points are stable (solutions approach them) and which are unstable (solutions move away).
3. They reveal how solutions behave as time approaches infinity.

[H*] **Harvesting a Renewable Resource**

Suppose that the population of y of a certain species of fish (for example, tuna or halibut) in a given area of the ocean is described by the logistic equation

$$\frac{dy}{dt} = r\left(1 - \frac{y}{K}\right)y \tag{5.71}$$

Although it is desirable to utilize this source of food, it is intuitively clear that if too many fish are caught, then the fish population may be reduced below a useful level and possibly even driven to extinction.

The solution to Eq. (5.71) with initial value problem $y(0) = y_0$ is

$$y(t) = \frac{y_0 K e^{rt}}{(y_0 - K)\left(\frac{y_0 e^{rt}}{y_0 - K} - 1\right)} \tag{5.72}$$

[H*] **Example:**

At a given level of effort, it is reasonable to assume that the rate at which fish are caught depends on the population y: the more fish there are, the easier it is to catch them. Thus we assume that the rate at which fish are caught is given by Ey, where E is a positive constant, with units of $1/\text{time}$, that measures the total effort made to harvest the given species of fish. To include this effect, the logistic equation is replaced by

$$\frac{dy}{dt} = r\left(1 - \frac{y}{K}\right)y - Ey \tag{5.73}$$

This equation is known as the Schaefer model after the biologist M.B. Schaefer, who applied it to fish populations.

(a) Show that if $E < r$, then there are two equilibrium points $y_1 = 0$ and $y_2 = K\left(1 - \frac{E}{r}\right) > 0$.

(b) Determine the stability for the equilibrium points.

(c) A sustainable yield Y of the fishery is a rate at which fish can be caught indefinitely. It is the product of the effort E and the asymptotically stable population y_2. Find Y as a function of the effort E; the graph of this function is known as the yield-effort curve.

(d) Determine E so as to maximize Y and thereby find the maximum sustainable yield Y_m.

Solution:

(a) To determine the equilibrium points or the critical points we need to locate the roots of $f(y) = 0$

$$\begin{aligned} f(y) &= 0 \\ r\left(1 - \frac{y}{K}\right)y - Ey &= 0 \\ -\frac{ry^2}{K} + (r - E)y &= 0 \end{aligned}$$

the roots are

$$y_{1,2} = \frac{-(r-E) \pm \sqrt{(r-E)^2 - 4\left(-\frac{r}{K}\right)(0)}}{2\left(-\frac{r}{K}\right)}$$
$$= \frac{-(r-E) \pm (r-E)}{-\frac{2r}{K}}$$

$$y_{1,2} = \begin{cases} 0 \\ \frac{-2r+2E}{-2r/K} = \frac{K(E-r)}{-r} = K\left(1-\frac{E}{r}\right) \end{cases}$$

If $E < r$ then $K\left(1-\frac{E}{r}\right) > 0$ and is a possible equilibrium population.

(b) To determine which equilibrium solution that is stable and unstable we will check with inputting y as several values in $\frac{dy}{dt} = f(y)$

$$f(y) = -\frac{ry^2}{K} + (r-E)y$$
$$= \left(y - K\left(1-\frac{E}{r}\right)\right)y$$

If $0 < y < K\left(1-\frac{E}{r}\right)$ then $\frac{dy}{dt} < 0$, so y is decreasing on these intervals.
If $y > K\left(1-\frac{E}{r}\right)$ then $\frac{dy}{dt} > 0$, so y is increasing on these intervals.

The equilibrium at $y = 0$ is a sink (asymptotically stable) and the equilibrium at $K\left(1-\frac{E}{r}\right)$ is a source (unstable).

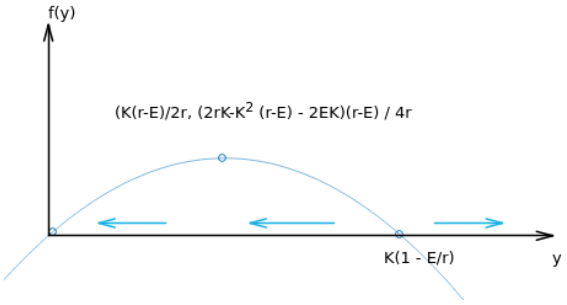

Figure 5.37: *The equilibrium points and the stability analysis for* $\frac{dy}{dt} = f(y) = \left(y - K\left(1-\frac{E}{r}\right)\right)y$ (***Hamzstplot/Examples with Makefile/Plot 2D Equilibrium Solution for Schaefer Model/main.cpp***).

To determine the vertex of the parabola you will have to compute

$$\frac{df(y)}{dy} = 0$$
$$\frac{d\left(-\frac{ry^2}{K} + (r-E)y\right)}{dt} = 0$$
$$y = \frac{K(r-E)}{2r}$$

then input y back into $f(y)$ again to obtain the vertex fully

$$\begin{aligned} f(y) &= -\frac{r\left(\frac{K(r-E)}{2r}\right)^2}{K} + (r-E)\left(\frac{K(r-E)}{2r}\right) \\ &= \frac{(2rK - K^2(r-E) - 2EK)(r-E)}{4r} \end{aligned}$$

thus the vertex is $\left(\frac{K(r-E)}{2r}, \frac{(2rK-K^2(r-E)-2EK)(r-E)}{4r}\right)$.

(c) The sustainable yield is the effort times the asymptotically stable population.

$$Y(E) = EK\left(1 - \frac{E}{r}\right)$$

To maximize the sustainable yield, take the derivative of it, set it equal to zero, and solve the resulting equation for E.

$$\begin{aligned} Y'(E) &= 0 \\ K\left(1 - \frac{E}{r}\right) + EK\left(-\frac{1}{r}\right) &= 0 \\ K\left(1 - \frac{E}{r}\right) &= \frac{EK}{r} \\ K(r-E) &= EK \\ rK - EK &= EK \\ 2EK &= rK \\ E &= \frac{r}{2} \end{aligned}$$

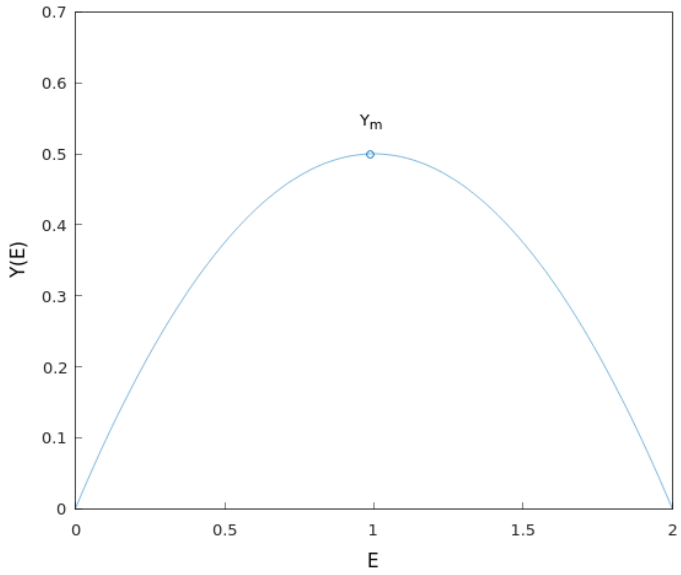

Figure 5.38: *The yield-effort curve for $Y(E) = EK\left(1 - \frac{E}{r}\right)$ with $r = 2$ and $K = 1$ (**Hamzstplot/Examples with Makefile/Plot 2D Yield-Effort Curve for Schaefer Model/main.cpp**).*

(d) The maximum sustainable yield is therefore

$$\begin{aligned} Y_m &= Y\left(E = \frac{r}{2}\right) \\ &= \left(\frac{r}{2}\right) K\left(1 - \frac{1}{2}\right) \\ &= \frac{rK}{4} \end{aligned}$$

[H*] **Example:**
We assume that fish are caught at a constant rate h independent of the size of the fish population. Then y satisfies

$$\frac{dy}{dt} = r\left(1 - \frac{y}{K}\right) y - h \tag{5.74}$$

The assumption of a constant catch rate h may be reasonable when y is large but becomes less so when y is small.

(a) If $h < rK/4$, show that Eq. (5.74) has two equilibrium points y_1 and y_2 with $y_1 < y_2$; determine these points.

(b) Show that y_1 is unstable and y_2 is asymptotically stable.

(c) From a plot of $f(y)$ versus y, show that if the initial population $y_0 > y_1$, then $y \to y_2$ as $t \to \infty$, but that if $y_0 < y_1$, then y decreases as t increases. Note that $y = 0$ is not an equilibrium point, so if $y_0 < y_1$, then extinction will be reached in a finite time.

(d) If $h > rK/4$, show that y decreases to zero as t increases regardless of the value of y_0.

(e) If $h = rK/4$, show that there is a single equilibrium point $y = K/2$ and that this point is semistable. Thus the maximum sustainable yield is $h_m = rK/4$, corresponding to the equilibrium value $y = K/2$. Observe that h_m has the same value as Y_m in the previous example. The fishery is considered to be overexploited if y is reduced to a level below $K/2$.

Solution:

(a) To determine the equilibrium points we will find the roots of $f(y) = 0$

$$\begin{aligned} f(y) &= 0 \\ r\left(1 - \frac{y}{K}\right) y - h &= 0 \\ -\frac{r}{K} y^2 + ry - h &= 0 \end{aligned}$$

with quadratic roots we will obtain

$$\begin{aligned} y_{1,2} &= \frac{-r \pm \sqrt{r^2 - 4\left(-\frac{r}{K}\right)(-h)}}{2\left(-\frac{r}{K}\right)} \\ &= \frac{-r \pm \sqrt{r^2 - \left(\frac{4rh}{K}\right)}}{-\frac{2r}{K}} \\ &= \frac{r \pm \sqrt{r^2 - \left(\frac{4rh}{K}\right)}}{\frac{2r}{K}} \end{aligned}$$

$$y_{1,2} = \begin{cases} \frac{r+\sqrt{r^2-\left(\frac{4rh}{K}\right)}}{\frac{2r}{K}} = \frac{K+\sqrt{K^2-\frac{4Kh}{r}}}{2} \\ \frac{r-\sqrt{r^2-\left(\frac{4rh}{K}\right)}}{\frac{2r}{K}} = \frac{K-\sqrt{K^2-\frac{4Kh}{r}}}{2} \end{cases}$$

These are the equilibrium solutions as long as

$$r^2 - \frac{4rh}{K} > 0$$
$$h < \frac{rK}{4}$$

(b) We will try to plot $f(y) = r\left(1 - \frac{y}{K}\right)y - h$ versus y with $r = 1, K = 1$, and $h = 1/8$.

With the chosen parameters, we will have

$$f(y) = (1-y)y - \frac{1}{8}$$

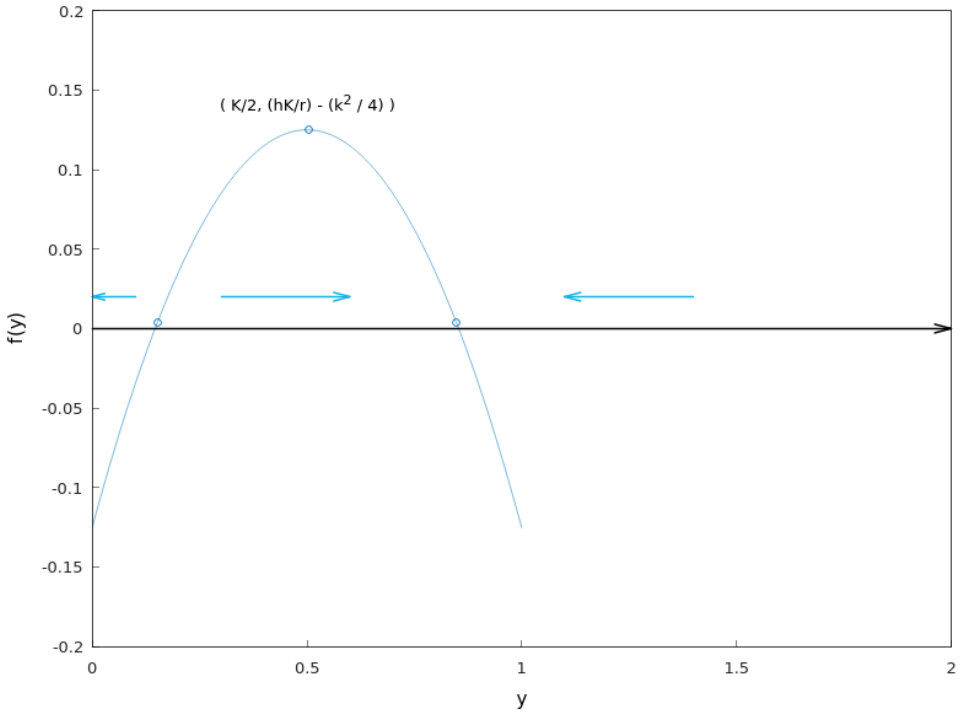

Figure 5.39: *The plot for* $f(y) = (1-y)y - \frac{1}{8}$ ***(Hamzstplot/Examples with Makefile/Plot 2D Equilibrium Solution for Schaefer Model Variation/main.cpp).***

To save more time, we do not need to compute the exact numerical equilibrium points of y_1 and y_2 here, we only need to know $f(y) = (1-y)y - \frac{1}{8}$ and then plot $f(y)$ versus y, from the plot we will be able to determine the stability by approximation. For example, it looks very clearly that $y = 0$ is less than $y = y_1$ so we can compute $f(0)$ to see $\frac{dy}{dt}$ behavior in that interval, if $y < y_1$ then $\frac{dy}{dt} < 0$ and the arrow

is pointing to the left on the horizontal line, or pointing downward in the phase line.

We see that the equilibrium point y_1 is a source (unstable) and the equilibrium point at $y = y_2$ is a sink (asymptotically stable).

(c) Suppose that the initial condition is

$$y(0) = y_0$$

As indicated by the arrows from Figure (5.39), if $y_0 > y_1$, then the population will tend to $y(t) = y_2$ as $t \to \infty$.

If $y_0 = y_1$, then the population will remain there.

If $y_0 < y_1$, then the population will tend to zero in a finite time because there are no equilibrium points to the left of $y = y_1$.

(d) Suppose that $h > \frac{rK}{4}$, for example, using $r = 1, K = 1$, and $h = 1$. Then the entire graph of $f(y) = r\left(1 - \frac{y}{K}\right) y - h$ versus y lies below the y-axis, which means all the arrows point to the left.

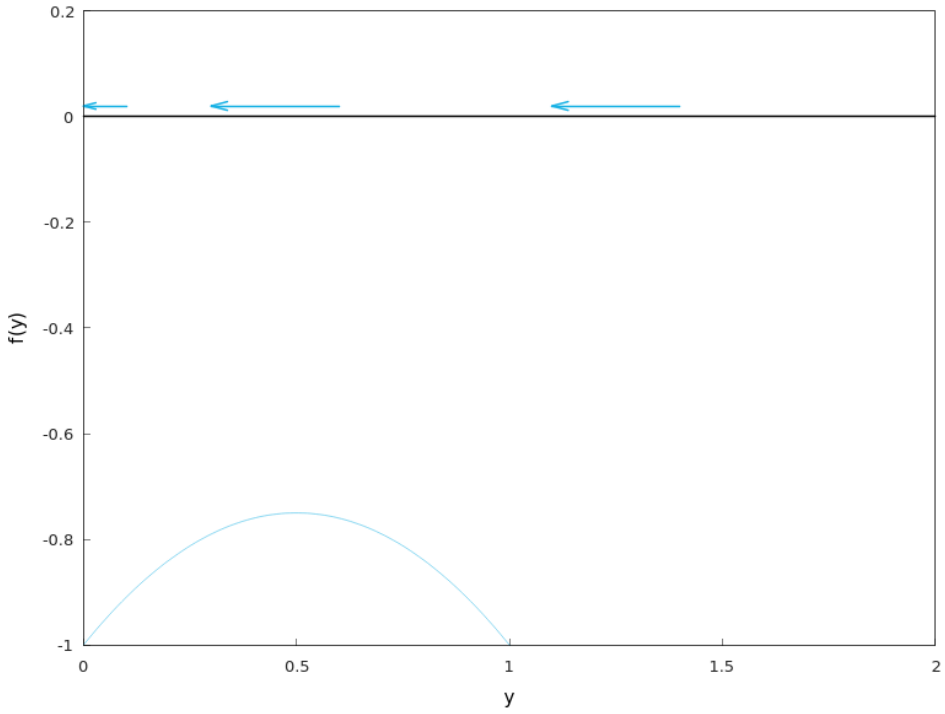

Figure 5.40: *The plot for $f(y) = (1 - y)y - 1$ (**Hamzstplot/Examples with Makefile/Plot 2D No Equilibrium Solution for Schaefer Model Variation/main.cpp**).*

This happens because $h > \frac{rK}{4}$ and this condition does not fit the requirement to have equilibrium solution from part (a).

The population tends to zero regardless of what y_0 is.

(e) Suppose that $h = \frac{rK}{4}$, then there will only be one equilibrium point

$$y = \frac{K \pm \sqrt{K^2 - \frac{4Kh}{r}}}{2} = \frac{K \pm \sqrt{0}}{2} = \frac{K}{2}$$

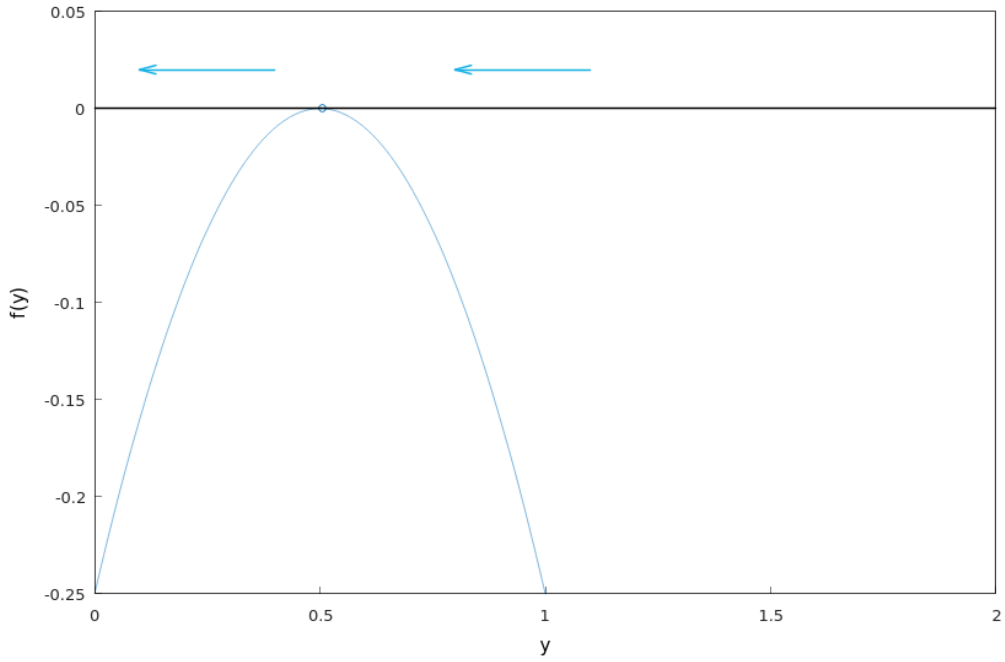

Figure 5.41: *The plot for $f(y) = (1-y)y - \frac{1}{4}$ (**Hamzstplot/Examples with Makefile/Plot 2D Equilibrium Solution Only 1 for Schaefer Model Variation/main.cpp**).*

Even though the graph lies below the y-axis, the difference between part (d) is that there's now an equilibrium point at $y = \frac{K}{2}$.

If $y_0 > \frac{K}{2}$, then the population will tend to $y(t) = \frac{K}{2}$ as $t \to \infty$.

If $y_0 = \frac{K}{2}$, then the population will remain there.

If $y_0 < \frac{K}{2}$, then the population will tend to zero in a finite time because there are no equilibria to the left of $y = \frac{K}{2}$.

Thus, the equilibrium point here at $y = \frac{K}{2}$ is semistable (stable from one side and unstable from the other side).

[H*] **Epidemics**

The use of mathematical methods to study the spread of contagious diseases goes back at least to some work by Daniel Bernoulli in 1760 on smallpox. In more recent years many mathematical models have been proposed and studied for many different diseases. Similar models have also been used to describe the spread of rumors and of

consumer products.

[H*] **Example:**
Suppose that a given population can be divided into two parts: those who have a given disease and can infect others, and those who do not have it but are susceptible. Let x be the proportion of susceptible individuals and y the proportion of infectious individuals; then $x + y = 1$. Assume that the disease spreads by contact between sick and well members of the population and that the rate of spread $\frac{dy}{dt}$ is proportional to the number of such contacts. Further, assume that members of both groups move about freely among each other, so the number of contacts is proportional to the product of x and y. Since $x = 1 - y$, we obtain the initial value problem

$$\frac{dy}{dt} = \alpha y(1 - y), \quad y(0) = y_0 \tag{5.75}$$

where α is positive proportionality factor, and y_0 is the initial proportion of infectious individuals.

(a) Find the equilibrium points for the differential equation (5.75) and determine whether each is asymptotically stable, semistable, or unstable.

(b) Solve the initial value problem (5.75) and verify that the conclusions you reached in part (a) are correct. Show that $y(t) \to 1$ as $t \to \infty$, which means that ultimately the disease spreads through the entire population.

Solution:

(a) The equilibrium points are found by setting

$$\frac{dy}{dt} = 0$$

and solving the resulting equation for y.

$$\alpha y(1 - y)$$

The equilibrium points are at $y = \phi_1(t) = 0$ and $y = \phi_2(t) = 1$. With phase line analysis we will obtain that the equilibrium solution $y = \phi_1(t) = 0$ is unstable and the equilibrium solution $y = \phi_2(t) = 1$ is asympotically stable.

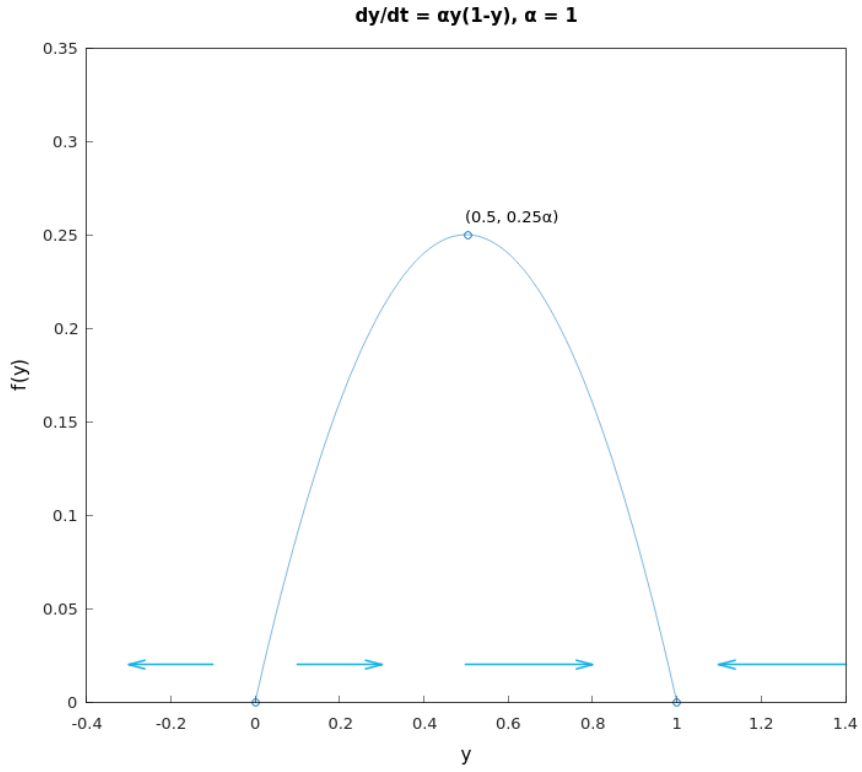

Figure 5.42: *The plot for $f(y) = \alpha y(1-y)$ with $\alpha = 1$ (**Hamzstplot/Examples with Makefile/Plot 2D Equilibrium Solution for Smallpox Epidemics Model/main.cpp**).*

What this means is that even if only one individual is infected in a population of eight billion, the entire population will eventually be infected.

(b) The ODE is separable, with

$$[y(1-y)]^{-1}dy = \alpha\, dt$$

Integrating both sides and invoking the initial condition, the solution is

$$\begin{aligned}
\frac{dy}{\alpha y(1-y)} &= dt \\
\frac{1}{\alpha y} + \frac{1/\alpha}{1-y}\, dy &= dt \\
\frac{1}{\alpha}\ln|y| + \frac{1}{\alpha y}(-\ln|y-1|) &= t + c \\
\frac{\ln|y| - \ln|y-1|}{\alpha} &= t + c \\
\ln\left|\frac{y}{y-1}\right| &= \alpha t + C
\end{aligned}$$

To determine C we will apply the initial condition with $y(0) = y_0$, thus

$$C = \ln\left|\frac{y_0}{y_0 - 1}\right|$$

As a result, the previous equation becomes

$$\ln\left|\frac{y}{y-1}\right| = \alpha t + \ln\left|\frac{y_0}{y_0-1}\right|$$

Since y and y_0 are between 0 and 1, the absolute value signs can be removec by writing the denominators as $1-y$ and $1-y_0$.

$$\begin{aligned}
\ln\frac{y}{1-y} &= \alpha t + \ln\frac{y_0}{1-y_0} \\
\frac{y}{1-y} &= e^{\alpha t + \ln\frac{y_0}{1-y_0}} \\
&= e^{\alpha t} e^{\ln\frac{y_0}{1-y_0}} \\
&= e^{\alpha t}\left(\frac{y_0}{1-y_0}\right) \\
\frac{y}{1-y}(1-y) &= (1-y)\left(e^{\alpha t}\left(\frac{y_0}{1-y_0}\right)\right) \\
y &= e^{\alpha t}\left(\frac{y_0}{1-y_0}\right) - e^{\alpha t}\left(\frac{y_0}{1-y_0}\right) y \\
y\left[1+e^{\alpha t}\left(\frac{y_0}{1-y_0}\right)\right] &= e^{\alpha t}\left(\frac{y_0}{1-y_0}\right) \\
y &= \frac{e^{\alpha t}\left(\frac{y_0}{1-y_0}\right)}{1+e^{\alpha t}\left(\frac{y_0}{1-y_0}\right)} \\
&= \frac{e^{\alpha t} y_0}{(1-y0)+e^{\alpha t} y_0} \\
y(t) &= \frac{y_0}{(1-y_0)e^{-\alpha t}+y_0}
\end{aligned}$$

It is evident that (independent of y_0)

$$\lim_{t\to-\infty} y(t) = 0$$

and

$$\lim_{t\to\infty} y(t) = 1$$

If there are no infected individuals initially ($y_0 = 0$), then $y(t) = 0$. However, if $0 < y_0 < 1$, then the exponential term decays to zero as $t \to \infty$. If at least one person is infected, then ultimately the disease spreads through the entire population.

[H*] **Bifurcation Points**

For an equation of the form

$$\frac{dy}{dt} = f(a, y) \tag{5.76}$$

where a is a real parameter, the critical points (equilibrium solutions) usually depend on the value of a. As a steadily increases or decreases, it often happens that at a certain value of a, called a bifurcation point, critical points come together, or separate, and

equilibrium solutions may either be lost or gained.

Bifurcation points are of great interest in many applications, because near them the nature of the solution of the underlying differential equation is undergoing an abrupt change. For example, in fluid mechanics a smooth (laminar) flow may break up and become turbulent. Or an axially loaded column may suddenly buckle and exhibit a large lateral displacement. Or, as the amount of one of the chemicals in a certain mixture is increased, a spiral wave patterns of varying color may suddenly emerge in an originally quiescent fluid.

[H*] **Example:**
Consider the equation

$$\frac{dy}{dt} = a - y^2 \tag{5.77}$$

(a) Find all of the critical points for Eq. (5.77). Observe that there are no critical points if $a < 0$, one critical point if $a = 0$, and two critical points if $a > 0$

(b) Draw the phase line in each case and determine whether each critical point is asymptotically stable, semistable, or unstable.

(c) In each case sketch several solutions of Eq. (5.77) in the ty-plane.

(d) If we plot the location of the critical points as a function of a in the ay-plane, we obtain the bifurcation diagram for Eq. (5.77). The bifurcation at $a = 0$ is called a saddle-node bifurcation.

Solution:

(a) The critical points are found by setting $\frac{dy}{dt} = 0$ and solving the resulting equation for y.

$$\begin{aligned} a - y^2 &= 0 \\ y^2 &= a \\ y &= \pm\sqrt{a} \end{aligned}$$

If a is negative, then there are no critical points; if a is zero, then there is one critical point; and if a is positive, then there are two critical points.

(b) The phase line for the case of $a < 0$:

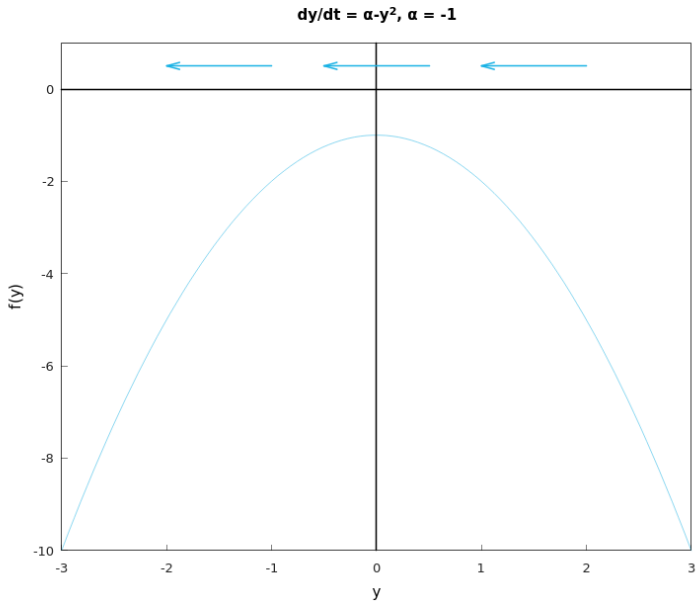

Figure 5.43: *The plot for $f(y) = \alpha - y^2$ with $\alpha = -1$, the phase line is drawn with the blue arrows (**Hamzstplot/Examples with Makefile/Plot 2D for Bifurcation Points Problem 25 No Equilibrium Solution/main.cpp**).*

There is no critical point here.

The phase line for the case of $a = 0$:

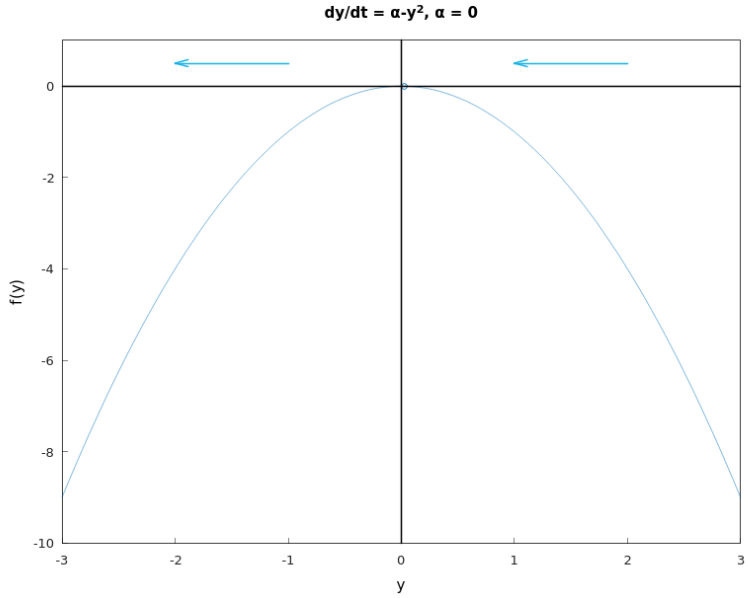

Figure 5.44: *The plot for $f(y) = \alpha - y^2$ with $\alpha = 0$, the phase line is drawn with the blue arrows (**Hamzstplot/Examples with Makefile/Plot 2D for Bifurcation Points Problem 25 Only 1 Equilibrium Solution/main.cpp**).*

There is one critical point here $y = \phi_1(t) = 0$, and it is semistable.

The phase line for the case of $a > 0$:

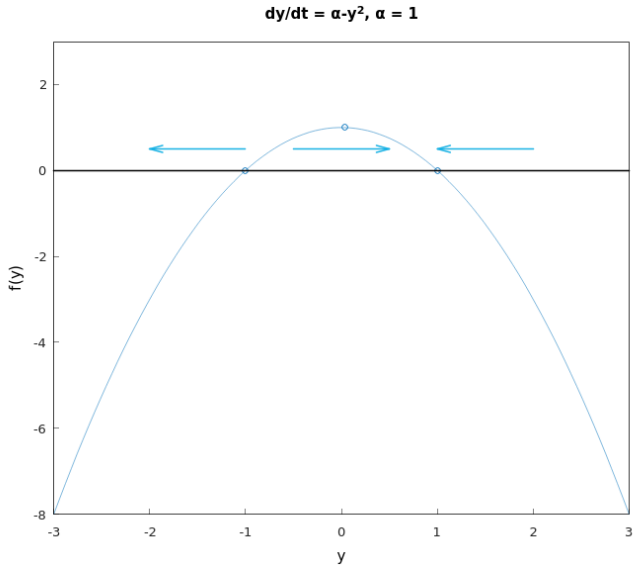

Figure 5.45: *The plot for $f(y) = \alpha - y^2$ with $\alpha = 1$, the phase line is drawn with the blue arrows (**Hamzstplot/Examples with Makefile/Plot 2D for Bifurcation Points Problem 25 2 Equilibrium Solutions/main.cpp**).*

There are two critical points here $y = \phi_1(t) = \sqrt{a} = 1$ that is asymptotically stable and $y = \phi_2(t) = -\sqrt{a} = -1$ that is unstable.

(c) The ty-plane is the direction field for $\frac{dy}{dt}$

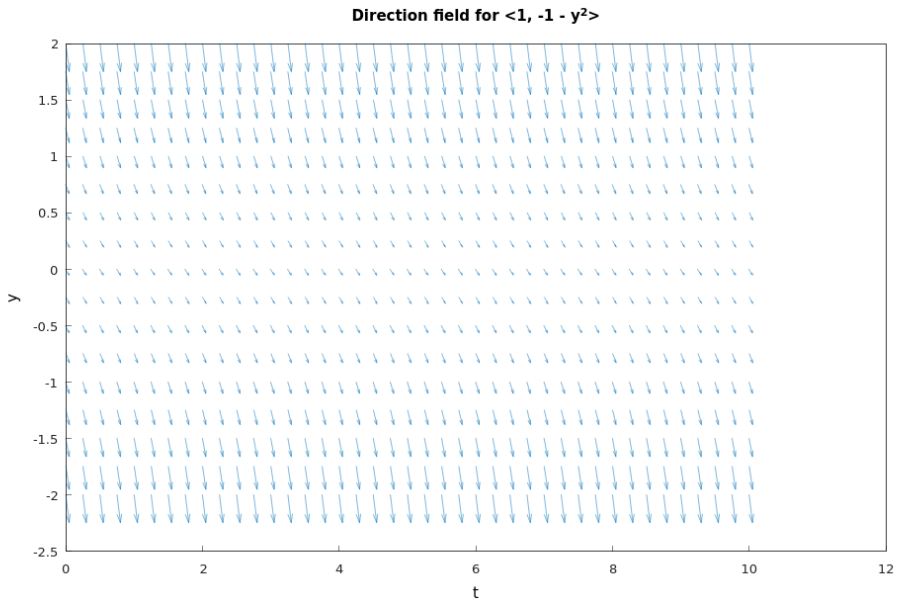

Figure 5.46: *The direction field for $\frac{dy}{dt} = -1 - y^2$. Solution curves in the ty-plane corresponding to $\alpha = -1$ are tangent to the vectors in the direction field $< 1, -1 - y^2 >$ at every point (**Hamzstplot/Examples with Makefile/Plot Direction Fields for Bifurcation Points Problem 25a/main.cpp**).*

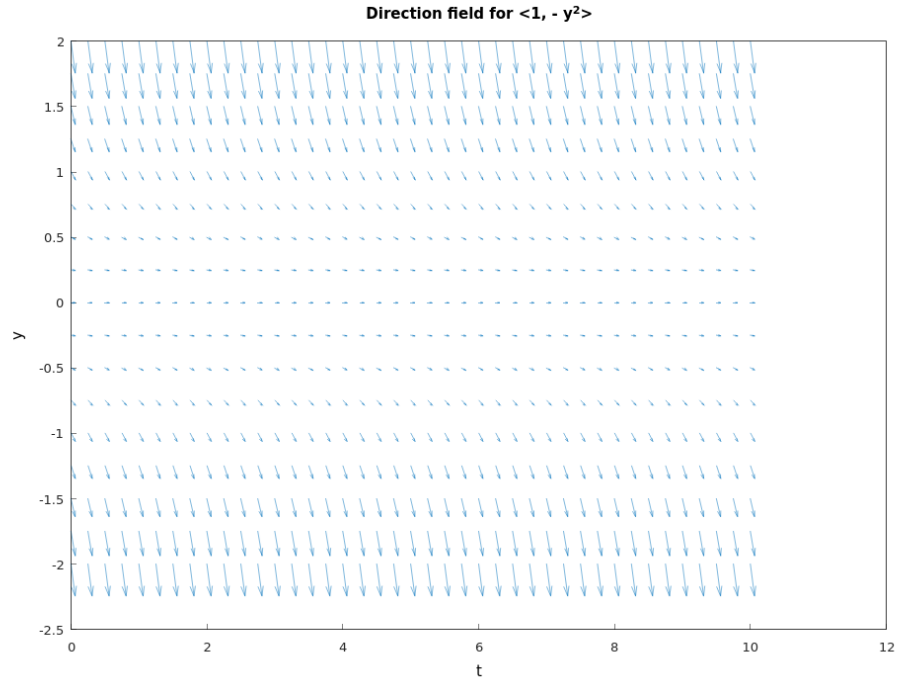

Figure 5.47: *The direction field for $\frac{dy}{dt} = -y^2$. Solution curves in the ty-plane corresponding to $\alpha = 0$ are tangent to the vectors in the direction field $< 1, -y^2 >$ at every point (**Hamzstplot/Examples with Makefile/Plot Direction Fields for Bifurcation Points Problem 25b/main.cpp**).*

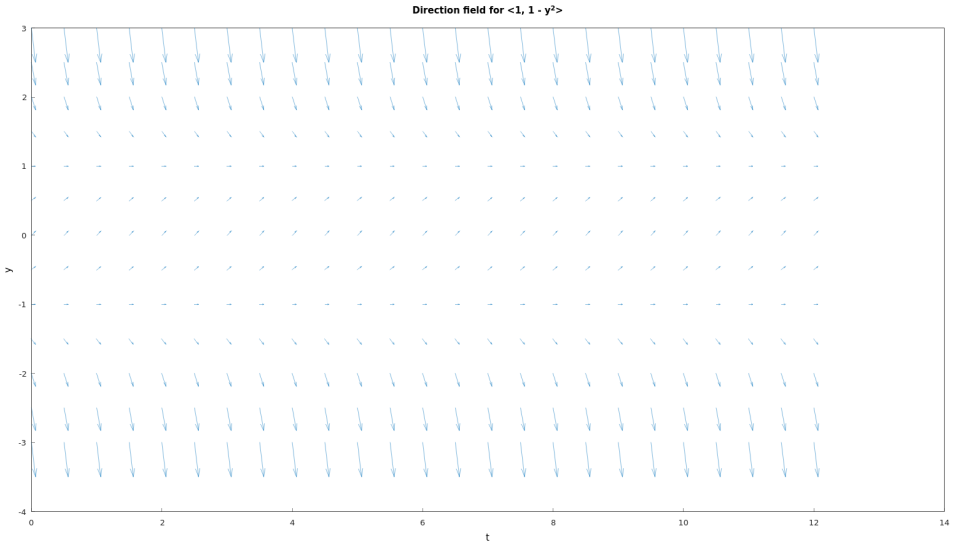

Figure 5.48: *The direction field for* $\frac{dy}{dt} = 1 - y^2$. *Solution curves in the ty-plane corresponding to* $\alpha = 1$ *are tangent to the vectors in the direction field* $< 1, 1 - y^2 >$ *at every point (**Hamzstplot/Examples with Makefile/Plot Direction Fields for Bifurcation Points Problem 25c/main.cpp**).*

(d) We are going to plot the location of the critical points as a function of a, it is

$$y(a) = \pm\sqrt{a}$$

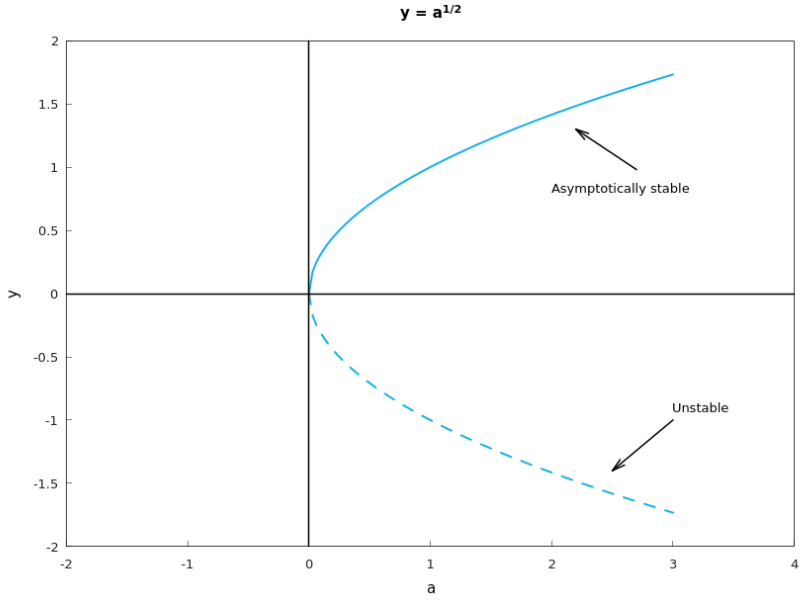

Figure 5.49: *The bifurcation diagram for* $\frac{dy}{dt} = a - y^2$, *the bifurcation point at* $a = 0$ *is called a saddle-node bifurcation where the solution then differ into two kind of solution: stable and unstable (**Hamzstplot/Examples with Makefile/Plot 2D Bifurcation Diagram Problem 25/main.cpp**).*

[H*] **Example:**
Consider the equation

$$\frac{dy}{dt} = ay - y^3 = y(a - y^2) \tag{5.78}$$

(a) Consider the cases $a < 0, a = 0$, and $a > 0$. In each case find all of the critical points for Eq. (5.78), draw the phase line, and determine whether each critical point is asympotically stable, semistable, or unstable.

(b) In each case sketch several solutions of Eq. (5.78) in the ty-plane.

(c) Draw the bifurcation diagram of Eq. (5.78), that is, plot the location of the critical points versus a. For Eq. (5.78) the bifurcation point at $a = 0$ is called a pitchfork bifurcation.

Solution:

(a) The critical points are found by setting $\frac{dy}{dt} = 0$ and solving the resulting equation for y.

$$y(a - y^2) = 0$$
$$y = \{0, \pm\sqrt{2}\}$$

If a is negative, then there is only one critical point at $y = 0$; if a is zero, then there is still only one critical point at $y = 0$; and if a is positive, then there are three critical points.

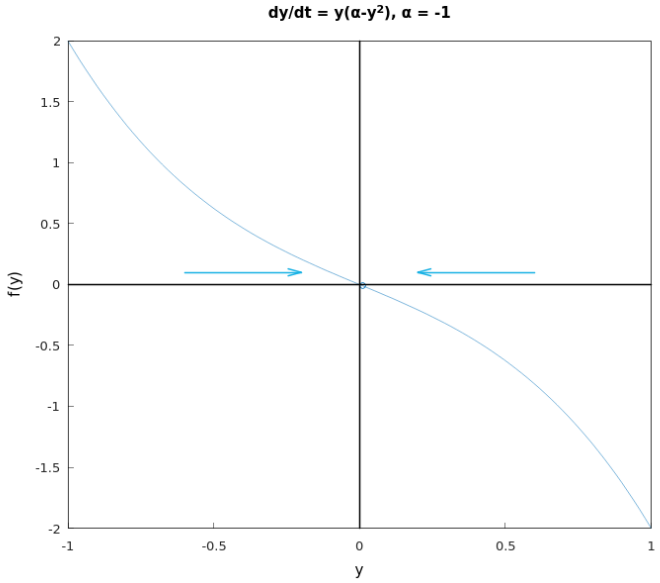

Figure 5.50: *The plot for $f(y) = y(\alpha - y^2)$ with $\alpha = -1$, the phase line is drawn with the blue arrows (**Hamzstplot/Examples with Makefile/Plot 2D for Bifurcation Points Problem 26 Only 1 Equilibrium Solution/main.cpp**).*

There is one critical point at $y = 0$, and it is a sink (stable).

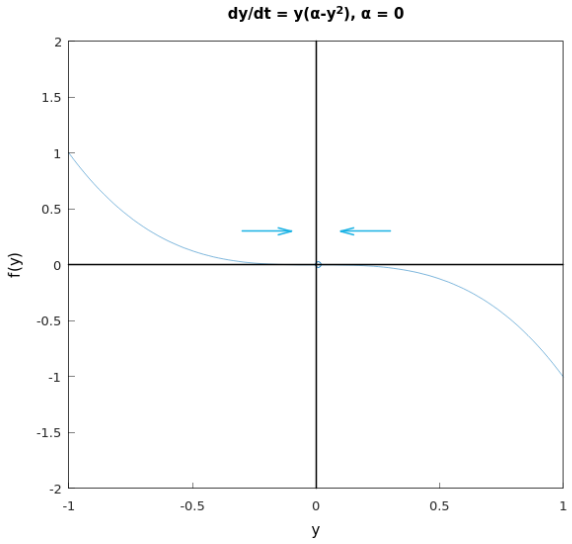

Figure 5.51: *The plot for $f(y) = y(\alpha - y^2)$ with $\alpha = 0$, the phase line is drawn with the blue arrows (**Hamzstplot/Examples with Makefile/Plot 2D for Bifurcation Points Problem 26 Only 1 Equilibrium Solution/main.cpp**).*

There is one critical point at $y = 0$, and it is a sink (stable).

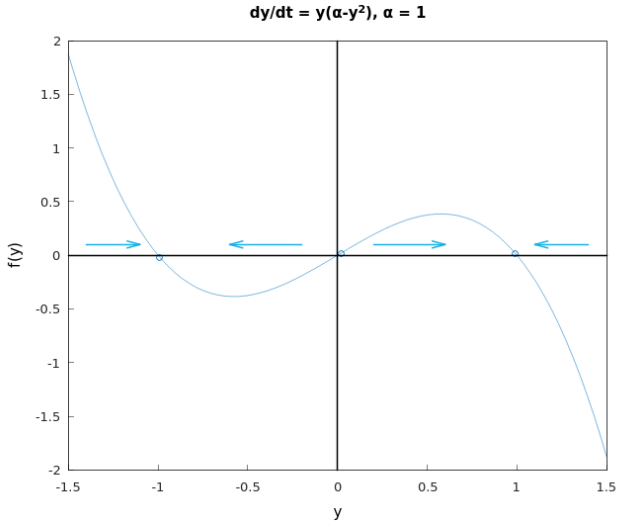

Figure 5.52: *The plot for $f(y) = y(\alpha - y^2)$ with $\alpha = 1$, the phase line is drawn with the blue arrows (**Hamzstplot/Examples with Makefile/Plot 2D for Bifurcation Points Problem 26 2 Equilibrium Solutions/main.cpp**).*

There are now three critical points at $y = -\sqrt{a}, y = 0$, and $y = \sqrt{a}$, and they are stable, unstable, and stable, respectively.

(b) We will draw the solutions in the ty-plane / the direction field.

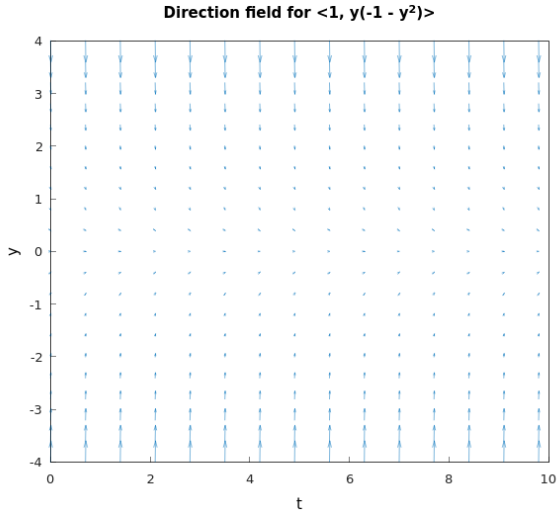

Figure 5.53: *The direction field for $\frac{dy}{dt} = y(-1 - y^2)$. Solution curves in the ty-plane corresponding to $\alpha = 1$ are tangent to the vectors in the direction field $< 1, y(-1 - y^2) >$ at every point (**Hamzstplot/Examples with Makefile/Plot Direction Fields for Bifurcation Points Problem 25c/main.cpp**).*

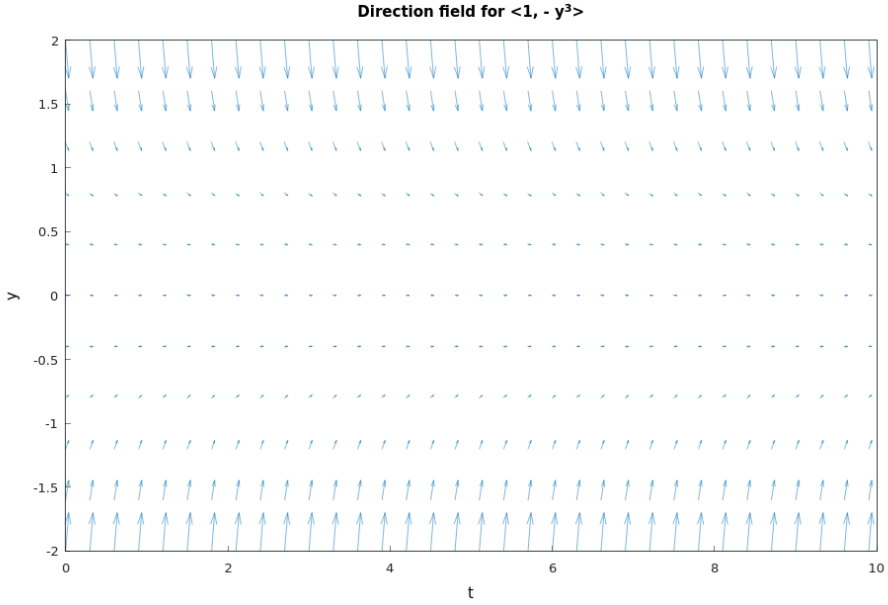

Figure 5.54: *The direction field for $\frac{dy}{dt} = -y^3$. Solution curves in the ty-plane corresponding to $\alpha = 0$ are tangent to the vectors in the direction field $< 1, -y^3 >$ at every point (**Hamzstplot/Examples with Makefile/Plot Direction Fields for Bifurcation Points Problem 25c/main.cpp**).*

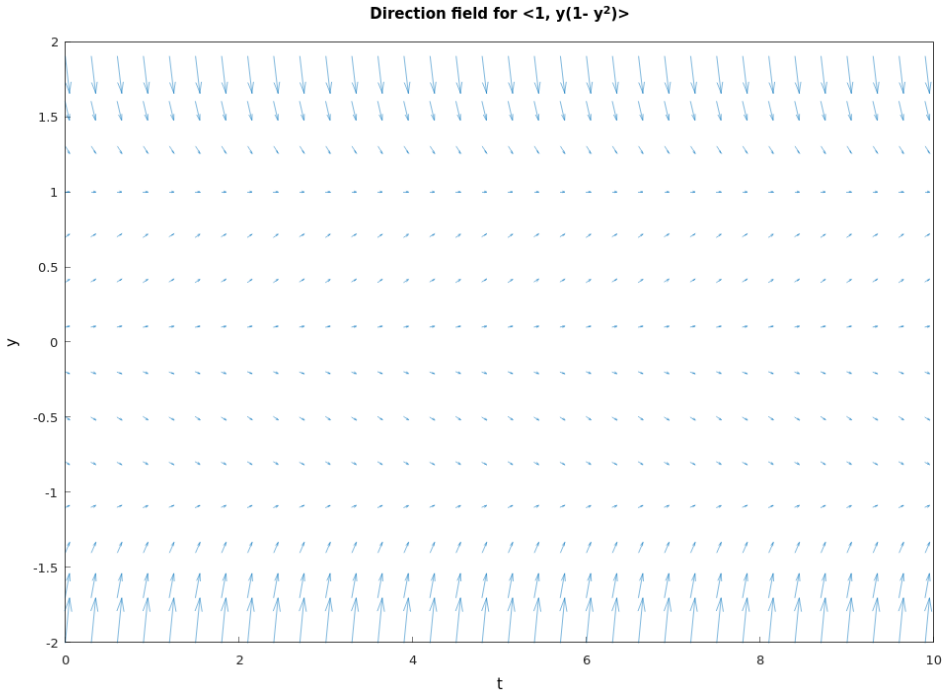

Figure 5.55: *The direction field for $\frac{dy}{dt} = y(1-y^2)$. Solution curves in the ty-plane corresponding to $\alpha = 1$ are tangent to the vectors in the direction field $< 1, y(1-y^2) >$ at every point (**Hamzstplot/Examples with Makefile/Plot Direction Fields for Bifurcation Points Problem 25c/main.cpp**).*

(c) This is the bifurcation diagram, looks like a pitchfork.

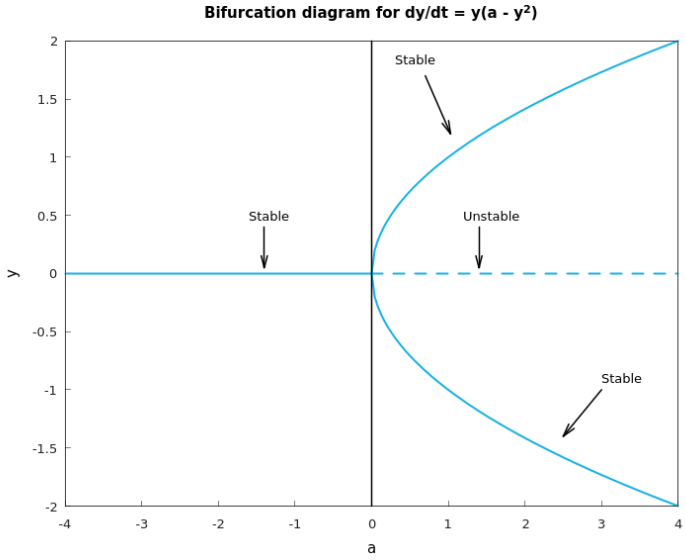

Figure 5.56: *The bifurcation diagram for $\frac{dy}{dt} = y(a-y^2)$, the bifurcation point at $a = 0$ is called a saddle-node bifurcation where the solution then differ into two kind of solution: stable and unstable (**Hamzstplot/Examples with Makefile/Plot 2D Bifurcation Diagram Problem 26/main.cpp**).*

[H*] **Example:**
Consider the equation

$$\frac{dy}{dt} = ay - y^2 = y(a - y) \tag{5.79}$$

(a) Consider the cases $a < 0, a = 0$, and $a > 0$. In each case find all of the critical points for Eq. (5.79), draw the phase line, and determine whether each critical point is asympotically stable, semistable, or unstable.

(b) In each case sketch several solutions of Eq. (5.79) in the ty-plane.

(c) Draw the bifurcation siagram of Eq. (5.79). Observe that for Eq. (5.79) there are the same number of critical points for $a < 0$ and $a > 0$ but that their stability. has changed. For $a < 0$ the equilibrium solution $y = 0$ is asympotically stable and $y = a$ is unstable, while for $a > 0$ the situation is reversed. Thus there has been an exchange of stability as a passes through the bifurcation point $a = 0$. This type of bifurcation is called a transcritical bifurcation.

Solution:

(a) .

[H*] **Chemical Reactions Example:**
A second order chemical reaction involves the interaction (collision) of one molecule of a substance P with one molecule of a substance Q to produce one molecule of a new substance X; this is denoted by

$$P + Q \to X$$

Suppose that p and q, where $p \neq q$, are the initial concentrations of P and Q, respectively, and let $x(t)$ be the concentration of X at time t. Then $p - x(t)$ and $q - x(t)$ are the concentrations of P and Q at time t, and the rate at which the reaction occurs is given by the equation

$$\frac{dx}{dt} = \alpha(p - x)(q - x) \tag{5.80}$$

where α is a positive constant.

(a) If $x(0) = 0$, determine the limiting value of $x(t)$ as $t \to \infty$ without solving the differential equation. Then solve the initial value problem and find $x(t)$ for any t.

(b) If the substances P and Q are the same, then $p = q$ and Eq. (5.80) is replaced by

$$\frac{dx}{dt} = \alpha(p - x)^2 \tag{5.81}$$

If $x(0) = 0$, determine the limiting value of $x(t)$ as $t \to \infty$ without solving the differential equation. Then solve the initial value problem and determine $x(t)$ for any t.

Solution:

(a) The chemical reaction for this problem is

$$P + Q \to X$$

The (stoichiometric) coefficients of P and Q are both 1, so the molecules of P and Q combine in a one-to-one fashion. A sample direction field

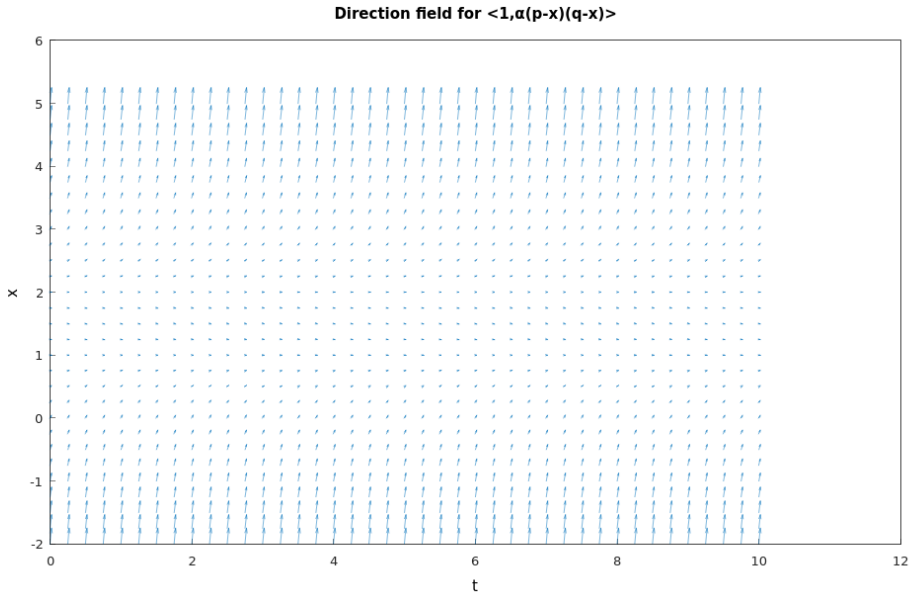

Figure 5.57: *The direction field for* $< 1, \alpha(p-x)(q-x) >$ *with* $\alpha = 1$, $p = 1$, *and* $q = 2$ (***Hamzstplot/Examples with Makefile/Plot Direction Fields for Chemical Reactions Problem 28a/main.cpp***).

We see that the solution $x(t)$ tends to 1 as $t \to \infty$ for the initial condition $x(0) = 0$. We conclude then that $x(t)$ tends to p or q, whichever of the two is smaller, as $t \to \infty$.

$$\lim_{t \to \infty} x(t) = \begin{cases} p, & \text{if } p < q \\ q, & \text{if } q < p \end{cases}$$

Solve the ODE now by separating variables

$$\begin{aligned}
\frac{dx}{dt} &= \alpha(p-x)(q-x) \\
\frac{dx}{(p-x)(q-x)} &= \alpha\, dt \\
\int \frac{dx}{(p-x)(q-x)} &= \alpha t + C \\
\int \left(\frac{1/(q-p)}{p-x} + \frac{1/(p-q)}{q-x} \right) dx &= \alpha t + C \\
\frac{1}{q-p} \int \frac{dx}{p-x} + \frac{1}{p-q} \int \frac{dx}{q-x} &= \alpha t + C \\
\frac{1}{p-q} \int \frac{dx}{x-p} + \frac{1}{q-p} \int \frac{dx}{x-q} &= \alpha t + C \\
\frac{1}{p-q} \ln|x-p| + \frac{1}{q-p} \ln|x-q| &= \alpha t + C \\
\ln|x-p|^{\frac{1}{p-q}} + \ln|x-q|^{\frac{1}{q-p}} &= \alpha t + C \\
-\ln|x-p|^{\frac{1}{q-p}} + \ln|x-q|^{\frac{1}{q-p}} &= \alpha t + C \\
\ln\left|\frac{x-q}{x-p}\right|^{\frac{1}{q-p}} &= \alpha t + C
\end{aligned}$$

Apply the initial condition $x(0) = 0$ now to determine C

$$\begin{aligned}
\ln\left|\frac{-q}{-p}\right|^{\frac{1}{q-p}} &= C \\
C &= \ln\left|\frac{q}{p}\right|^{\frac{1}{q-p}}
\end{aligned}$$

The previous equation is then

$$\begin{aligned}
\ln\left|\frac{x-q}{x-p}\right|^{\frac{1}{q-p}} &= \alpha t + \ln\left|\frac{q}{p}\right|^{\frac{1}{q-p}} \\
\frac{1}{q-p} \ln\left|\frac{x-q}{x-p}\right| &= \alpha t + \frac{1}{q-p} \ln\left|\frac{q}{p}\right| \\
\ln\left|\frac{x-q}{x-p}\right| &= \alpha t(q-p) + \ln\left|\frac{q}{p}\right| \\
\left|\frac{x-q}{x-p}\right| &= e^{\alpha t(q-p)} \left(\frac{q}{p}\right) \\
\frac{x-q}{x-p} &= \pm \frac{q}{p} e^{\alpha t(q-p)} \\
x - q &= \frac{q}{p} e^{\alpha t(q-p)} x - q e^{\alpha t(q-p)} \\
x\left[1 - \frac{q}{p} e^{\alpha t(q-p)}\right] &= q - q e^{\alpha t(q-p)} \\
x(t) &= \frac{q(1 - e^{\alpha t(q-p)})}{1 - \frac{q}{p} e^{\alpha t(q-p)}}
\end{aligned}$$

from the $\pm$ sign, we choose the plus sign so that the initial condition remains satisfied. Thus, we simplify again one time and the general solution can be written as

$$x(t) = \frac{pq(1 - e^{\alpha t(q-p)})}{p - qe^{\alpha t(q-p)}} \tag{5.82}$$

(b) The direction field below shows that the solution curves, which lie tangent to the direction field vectors at every point, show that the solution $x(t)$ tends to 1 as $t \to \infty$ with the initial value of $x(0) = 0$ and $p = 1, \alpha = 1$. We conclude that

$$\lim_{t\to\infty} x(t) = p$$

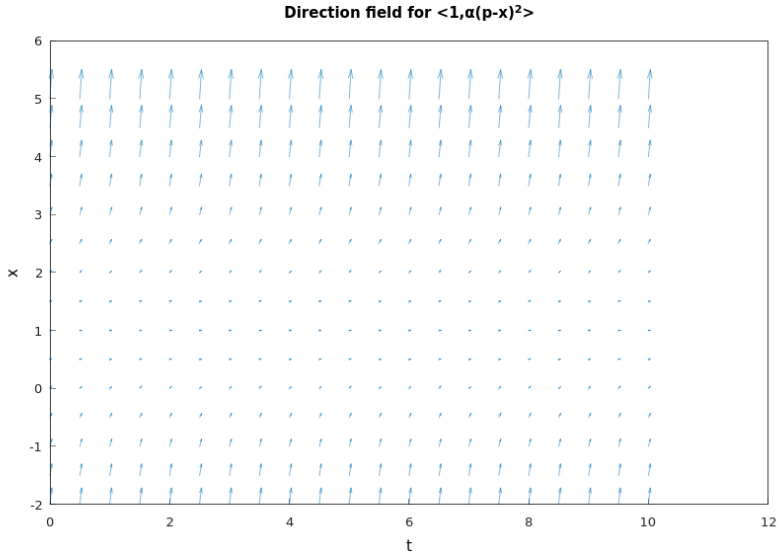

Figure 5.58: *The direction field for* $< 1, \alpha(p-x)^2 >$ *with* $\alpha = 1$ *and* $p = 1$ (***Hamzstplot/Examples with Makefile/Plot Direction Fields for Chemical Reactions Problem 28b/main.cpp***).

Solve the ODE now by separating variables

$$\begin{aligned}
\frac{dx}{dt} &= \alpha(p-x)^2 \\
\frac{dx}{(p-x)^2} &= \alpha\, dt \\
\int \frac{dx}{(p-x)^2} &= \alpha t + C_1 \\
\int \frac{dx}{(x-p)^2} &= \alpha t + C_1 \\
-\frac{1}{x-p} &= \alpha t + C_1
\end{aligned}$$

Apply the initial condition $x(0) = 0$ now to determine C_1.

$$-\frac{1}{-p} = C_1$$
$$C_1 = \frac{1}{p}$$

As a result, the previous equation becomes

$$\begin{aligned}
-\frac{1}{x-p} &= \alpha t + \frac{1}{p} \\
-(x-p) &= \frac{1}{\alpha t + \frac{1}{p}} \\
x - p &= -\frac{1}{\alpha t + \frac{1}{p}} \\
x(t) &= p - \frac{1}{\alpha t + \frac{1}{p}} \\
&= p - \frac{p}{\alpha p t + 1} \\
&= \frac{\alpha p^2 t + p - p}{\alpha p t + 1} \\
x(t) &= \frac{\alpha p^2 t}{\alpha p t + 1}
\end{aligned}$$

- Exact Equations and Integrating Factors

[H*] Exact equations are a class of equations for which there is also a well-defined method of solution.

[H*] **Example:**
Solve the differential equation

$$2x + y^2 + 2xyy' = 0 \tag{5.83}$$

The equation is neither linear nor separable, so the methods suitable for those types of equations are not applicable here. However, observe that the function $\psi(x, y) = x^2 + xy^2$ has the property that

$$\begin{aligned}
2x + y^2 &= \frac{\partial \psi}{\partial x} \\
2xy &= \frac{\partial \psi}{\partial y}
\end{aligned} \tag{5.84}$$

Therefore the differential equation can be written as

$$\frac{\partial \psi}{\partial x} + \frac{\partial \psi}{\partial y}\frac{dy}{dx} = 0 \tag{5.85}$$

Assuming that y is a function of x and calling upon the chain rule, we can write Eq. (5.85) in the equivalent form

$$\frac{\partial \psi}{\partial x} = \frac{d}{dx}(x^2 + xy^2) = 0 \tag{5.86}$$

Therefore

$$\psi(x,y) = x^2 + xy^2 = c \tag{5.87}$$

where c is an arbitrary constant, is an equation that defines solutions of Eq. (5.83) implicitly.

The key step here was the recognition that there is a function ψ that satisfies Eqs. (5.84).
[H*] More generally, let the differential equation

$$M(x,y) + N(x,y)y' = 0 \tag{5.88}$$

be given. Suppose that we can identify a function ψ such that

$$\begin{aligned} \frac{\partial \psi}{\partial x}(x,y) &= M(x,y) \\ \frac{\partial \psi}{\partial y}(x,y) &= N(x,y) \end{aligned} \tag{5.89}$$

and such that $\psi(x,y) = c$ defines $y = \phi(x)$ implicitly as a differentiable function of x. Then

$$M(x,y) + N(x,y)y' = \frac{\partial \psi}{\partial x} + \frac{\partial \psi}{\partial y}\frac{dy}{dx} = \frac{d}{dx}\psi[x,\phi(x)]$$

and the differential equation (5.88) becomes

$$\frac{d}{dx}\psi[x,\phi(x)] = 0 \tag{5.90}$$

In this case Eq. (5.88) is said to be an exact differential equation. Solutions of Eq. (5.88), or the equivalent Eq. (5.90), are given implicitly by

$$\psi(x,y) = c \tag{5.91}$$

where c is an arbitrary constant.

Theorem 5.3: Exact Differential Equation in a Region R

Let the functions M, N, M_y, and N_x, where subscripts denote the partial derivatives, be continous in the rectangular region $R : \alpha < x < \beta,\ \gamma < y < \delta$. Then Eq. (5.88)

$$M(x,y) + N(x,y)y' = 0$$

is an exact differential equation in R if and only if

$$M_y(x,y) = N_x(x,y) \tag{5.92}$$

at each point of R. That is, there exists a function ψ satisfying Eqs. (5.89),

$$\begin{aligned} \psi_x(x,y) &= M(x,y) \\ \psi_y(x,y) &= N(x,y) \end{aligned}$$

if and only if M and N satisfy Eq. (5.92).

- Numerical Approximations: Euler's Method

[H*] Numerical methods for ordinary differential equations are methods used to find numerical approximations to the solutions of ordinary differential equations. Their use is also known as "numerical integration."

Many differential equations cannot be solved exactly. For practical purposes, however - such as in engineering - a numeric approximation to the solution is often sufficient. The algorithms studied here can be used to compute such an approximation. An alternative method is to use techniques from calculus to obtain a series expansion of the solution.

Ordinary differential equations occur in many scientific disciplines, including physics, chemistry, biology, and economics. In addition, some methods in numerical partial differential equations convert the partial differential equation into an ordinary differential equation, which must then be solved.

[H*] **Euler Method**

From any point on a curve, you can find an approximation of a nearby point on the curve by moving a short distance along a line tangent to the curve.

Suppose we have a first-order differential equation with initial value problem of the form

$$y'(t) = \frac{dy}{dt} = f(t, y(t)), \quad y(t_0) = y_0 \tag{5.93}$$

Starting with the differential equation Eq. (5.93), we replace the derivative y' by the finite difference approximation

$$y'(t) \approx \frac{y(t+h) - y(t)}{h} \tag{5.94}$$

which when re-arranged yields the following formula

$$y(t+h) \approx y(t) + hy'(t)$$

and using Eq. (5.93) gives

$$y(t+h) \approx y(t) + hf(t, y(t)) \tag{5.95}$$

This formula is usually applied in the following way.
We choose a step size h, and we construct the sequence t_0, t_1

$$\begin{aligned} t_0 \\ t_1 &= t_0 + h \\ t_2 &= t_0 + 2h \\ &\vdots \end{aligned}$$

Then we denote y_n, as the numerical estimate of the exact solution $y(t_n)$.
We compute these estimates by the following recursive scheme

$$y_{n+1} = y_n + hf(t_n, y_n) \tag{5.96}$$

This is the Euler method (or forward Euler method). The method is named after Leonhard Euler who described it in 1768.

The Euler method is an example of an explicit method. This means that the new value y_{n+1} is defined in terms of things that are already known, like y_n.

[H*] Consider this initial value problem

$$\frac{dy}{dt} = f(t, y(t)), \quad y(t_0) = y_0 \tag{5.97}$$

First, if f and $\frac{\partial f}{\partial y}$ are continuous, then the initial value problem (5.97) has a unique solution $y = \phi(t)$ in some interval surrounding the initial point $t = t_0$. Second, it is usually not possible to find the solution ϕ by symbolic manipulations of the differential equation.

We have learned differential equations that are linear, separable, or exact, or that can be transformed into one of these types. Nevertheless, it remains true that solutions of the vast majority of first order initial value problems cannot be found by analytical means.

To see how the Euler method works, let us consider how we might use tangent lines to approximate the solution $y = \phi(t)$ of Eqs. (5.97) near $t = t_0$. We know that the solution passes through the initial point (t_0, y_0), and from the differential equation, we also know that its slope at this point is $f(t_0, y_0)$. Thus, we can write down an equation for the line tangent to the solution curve at (t_0, y_0), namely

$$y = y_0 + f(t_0, y_0)(t - t_0) \tag{5.98}$$

The tangent line is a good approximation to the actual solution curve on an interval short enough so that the slope of the solution oes not change appreciably from its value at the initial point.
Thus, if t_1 is close enough to t_0, we can approximate $\phi(t_1)$ by the value y_1 determined by substituting $t = t_1$ into the tangent line approximation at $t = t_0$; thus

$$y_1 = y_0 = f(t_0, y_0)(t_1 - t_0) \tag{5.99}$$

To proceed further, we can try to repeat the process. Unfortunately, we do not know the value $\phi(t_1)$ of the solution at t_1. The best we can do is to use the approximate value y_1 instead. Thus we construct the line through (t_1, y_1) with the slope $f(t_1, y_1)$,

$$y = y_1 + f(t_1, y_1)(t - t_1) \tag{5.100}$$

To approximate the value of $\phi(t)$ at a nearby point t_2, we use Eq. (5.100) instead, obtaining

$$y_2 = y_1 + f(t_1, y_1)(t_2 - t_1) \tag{5.101}$$

Continuing in this manner, we use the value of y calculated at each step to determine the slope of the approximation for the next step. The general expression for the tangent line starting at (t_n, y_n) is

$$y = y_n + f(t_n, y_n)(t - t_n) \tag{5.102}$$

hence the approximation value y_{n+1} at t_{n+1} in terms of t_n, t_{n+1}, and y_n is

$$y_{n+1} = y_n + f(t_n, y_n)(t_{n+1} - t_n), \quad n = 0, 1, 2, \ldots \tag{5.103}$$

If we introduce the notation $f_n = f(t_n, y_n)$, then we can rewrite Eq. (5.103) as

$$y_{n+1} = y_n + f_n(t_{n+1} - t_n), \quad n = 0, 1, 2, \ldots \tag{5.104}$$

Finally, if we assume that there is a uniform step size h between points $t_0, t_1, t_2, \ldots$, then $t_{n+1} = t_n + h$ for each n, and we obtain Euler's formula in the form

$$y_{n+1} = y_n + f_n h, \quad n = 0, 1, 2, \ldots \tag{5.105}$$

To use Euler's method, you simply evaluate Eq. (5.104) or Eq. (5.105) repeatedly, depending on whether or not the step size is constant, using the result of each step to execute the next step. In this way you will generate a sequence of values $y_1, y_2, y_3, \ldots$ that approximate the values of the solution $\phi(t)$ at the points $t_1, t_2, t_3, \ldots$.

If, instead of a sequence of points, you need a function to approximate the solution $\phi(t)$, then you can use the piecewise linear function constructed fom the collection of tangent line segments. That is, let y be given in $[t_0, t_1]$ by Eq. (5.102) with $n = 0$, in $[t_1, t_2]$ by Eq. (5.102) with $n = 1$, and so on.

[H*] **Backward Euler Method**

If, instead of Eq. (5.94), we use the approximation

$$y'(t) \approx \frac{y(t) - y(t-h)}{h} \tag{5.106}$$

we get the backward Euler method:

$$y_{n+1} = y_n + hf(t_{n+1}, y_{n+1}) \tag{5.107}$$

The backward Euler method is an implicit method, meaning that we have to solve an equation to find y_{n+1}. One often uses fixed-point iteration or the Newton-Raphson method to achieve this.

It costs more time to solve this equation than explicit methods; this cost must be taken into consideration when one selects the method to use.

The advantage of implicit methods are that they usually more stable for solving a stiff equation, meanin that a larger step size h can be used.

The limitation to the Euler method is that it can be prone to significant errors, especially with larger step sizes or highly nonlinear functions. Furthermore, the Euler method is often not accurate enough. In more precise terms, it only has order one. The first-order accuracy means that the error is proportional to the step size h. This caused mathematicians to look for higher-order methods.

One possibility is to use not only the previously computed value y_n to determine y_{n+1}, but to make the solution depend on more past values. This yields a so-called multistep method. Another possibility is to use more points in the interval $[t_n, t_{n+1}]$. This leads to the family of Runge-Kutta methods. One of their fourth-order methods is especially popular.

[H*] Numerical analysis is not only the design of numerical methods, but also their analysis. Three central concepts in this analysis are:

1. Convergence: whether the method approximates the solution.
   A numerical method is said to be convergent if the numerical solution approaches the exact solution as the step size h goes to 0. More precisely, we require that

for every ordinary differential equations with type of Eq. (5.93) with a Lipschitz function f and every $t^* > 0$,

$$\lim_{h \to 0^+} \max_{n=0,1,\ldots,[t^*/h]} ||y_{n,h} - y(t_n)|| = 0$$

2. Order: how well it approximates the solution.
Suppose the numerical method is

$$y_{n+k} = \psi(t_{n+k}; y_n, y_{n+1}, \ldots, y_{n+k-1}; h)$$

The local (truncation) error of the method is the error committed by one step of the method. That is, it is the difference between the result given by the method, assuming that no error was made in earlier steps, and the exact solution:

$$\delta^h_{n+k} = \psi(t_{n+k}; y_n, y_{n+1}, \ldots, y_{n+k-1}; h) - y(t_{n+k})$$

The method is said to be consistent if

$$\lim_{h \to 0} \frac{\delta^h_{n+k}}{h} = 0$$

The method has order p if

$$\delta^h_{n+k} = O(h^{p+1})$$

as $h \to 0$.
Hence a method is consistent if it has an order greater than 0. The forward Euler method and the backward Euler method both have order 1, so they are consistent. Most methods being used in practice attain higher order. Consistency is a necessary condition for convergence, but not sufficient; for a method to be convergent, it must be both consistent and zero-stable.

A related concept is the global(truncation) error, the error sustained in all the steps one needs to reach a fixed time t. Explicitly, the global error at time t is $y_N - y(t)$ where $N = \frac{t-t_0}{h}$. The global error of a pth order one-step method is $O(h^p)$; in particular, such a method is convergent. This statement is not necessarily true for multi-step methods.

3. Stability: whether errors are damped out.
For some differential equations, application of standard methods-such as the Euler method, explicit Runge-Kutta methods, multistep methods (e.g., Adams-Bashforth methods)-exhibit instability in the solutions, though other methods may produce stable solutions.

This "difficult behavior" in the equation (which may not necessarily be complex itself) is described as stiffness, and is often caused by the presence of different time scales in the underlying problem.

For example, a collision in a mechanical system like in an impact oscillator typically occurs at much smaller time scale than the time for the motion of objects; this discrepancy makes for very "sharp turns" in the curves of the state parameters.

Stiff problems are ubiquitous in chemical kinetics, control theory, solid mechanics, weather forecasting, biology, plasma physics, and electronics. One way to overcome stiffness is to extend the notion of differential equation to that of differential inclusion, which allows for and models non-smoothness.

[H*] **Example:**
Consider the initial value problem

$$\frac{dy}{dt} = 3 - 2t - \frac{1}{2}y, \quad y(0) = 1 \tag{5.108}$$

Use Euler's method with step size $h = 0.2$ to find approximate values of the solution of Eqs (5.99) at $t = 0.2, 0.4, 0.6, 0.8$ and 1. Compare them with the corresponding values of the actual solution of the initial value problem.

Solution:
Note that the differential equation in the given initial value problem is linear and can be solved using the integrating factor $e^{t/2}$.

The resulting solution of the initial value problem (5.108) is

$$y = \phi(t) = 14 - 4t - 13e^{-t/2} \tag{5.109}$$

To approximate this solution by means of Euler's method, note that in this case

$$f(t, y) = 3 - 2t - \frac{y}{2}$$

Using the initial values $t_0 = 0$ and $y_0 = 1$, we find that

$$f_0 = f(t_0, y_0) = f(0, 1) = 3 - 0 - 0.5 = 2.5$$

and then, the tangent line approximation near $t = 0$ is

$$y = 1 + 2.5(t - 0) = 1 + 2.5t \tag{5.110}$$

Setting $t = 0.2$ in Eq. (5.110), we find the approximate value y_1 of the solution at $t = 0.2$, namely,

$$y_1 = 1 + (2.5)(0.2) = 1.5$$

At the next step we have

$$\begin{aligned} f_1 &= f(0.2, 1.5) = 3 - 2(0.2) - 0.5(1.5) = 3 - 0.4 - 0.75 = 1.85 \\ y &= 1.5 + 1.85(t - 0.2) = 1.13 + 1.85t \\ y_2 &= 1.13 + 1.85(0.4) = 1.87 \end{aligned}$$

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Solve Initial Value Problem with Euler Method ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Solve Initial Value Problem with Euler Method ]# ./main
For dy/dt = 3 - 2*t -0.5*y
y(t) = -4*t+C*e^(-0.5*t)+14
ivp for y(0)=1
y(t) = -4*t-13*e^(-0.5*t)+14
Euler' Method

f(x) = -2*t-0.5*y+3

     t          Euler approximation y_{i}                    Tangent line
     0          1                                    2.5*t+1
   0.2          1.5                                  1.85*t+1.13
   0.4          1.87                                 1.265*t+1.364
   0.6          2.123                                0.7385*t+1.6799
   0.8          2.2707                               0.26465*t+2.05898
     1          2.32363                              -0.161815*t+2.48545

solution =
2.32363

Time taken by function: 253124 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Solve Initial Value Problem with Euler Method ]#
```

Figure 5.59: *The computation of initial value problem solution with Euler's method. The first column contains the t-values separated by the step size $h = 0.2$. The second column shows the corresponding y-values computed from Euler's formula (5.105). In the third column are the tangent line approximations found from Eq. (5.102) (**SymIntegration/Examples/Test SymIntegration Solve Initial Value Problem with Euler Method/main.cpp**).*

One way to achieve more accurate results is to use a smaller step size, with a corresponding increase in the number of computational steps.

Figure 5.60: *The solution curve for $\frac{dy}{dt} = 3 - 2t - \frac{1}{2}y, \quad y(0) = 1$ and sequence of points from the Euler's method approximation (**Hamzstlab-Mathematics/examples/First Order Differential Equation/main.cpp**).*

- The Existence and Uniqueness Theorem

[H*] We have discussed a theorem before, which states that under certain conditions on $f(t, y)$, the initial value problem

$$y' = f(t, y(t)), \quad y(t_0) = y_0 \tag{5.111}$$

has a unique solution in some interval containing the point t_0.

- First Order Difference Equations

[H*] YES

V. Second Order Linear Equations

- Homogeneous Equations with Constant Coefficients

[H*] The first
[H*]

- Solutions of Linear Homogeneous Equations; the Wronskian

[H*] The first
[H*]

VI. Higher Order Linear Equations

- General Theory of nth Order Linear Equations

[H*] The first
[H*]

- The Method of Variation of Parameters

[H*] The first
[H*]

VII. Series Solutions of Second Order Linear Equations

- Power Series

[H*] The first
[H*]

- Series Solutions Near an Ordinary Point

[H*] The first
[H*]

VIII. Laplace Transform

- Definition

[H*] The first
[H*]

- Solution of Initial Value Problems

[H*] The first
[H*]

IX. Systems of First Order Linear Equations

- Introduction

[H*] The first
[H*]

- Linear Algebraic Equations

[H*] The first
[H*]

X. Numerical Methods

- The Runge-Kutta Method

[H*] The first
[H*]

- Multistep Methods

[H*] The first
[H*]

XI. Nonlinear Differential Equations and Stability

- General Theory of nth Order Linear Equations

[H*] The first
[H*]

- The Method of Variation of Parameters

[H*] The first
[H*]

XII. Boundary Value Problems and Sturm-Liouville Theory

- The Occurrence of Two-Point Boundary Value Problems

  [H*] The first
  [H*]

- Nonhomogeneous Boundary Value Problems

  [H*] The first
  [H*]

Chapter 6

Partial Differential Equations

*"PDE makes my Mathematics Learning more colorful..." - **Glanz***

*"Learn from basic Ordinary Differential Equation, don't jump to PDE yet (but still help Glanz learning PDE on 2023)" - **Sentinel***

There is always a funny story behind every chapter in Lasthrim Projection, it is only 3 days before Partial Differential Equation exam, and after surgery on Glanz' left leg, miss 2 weeks of classes, she thinks a chapter dedicated for PDE(Partial Differential Equation) is better than include just a snippet in the **Differential Equation** chapter, actually Freya is the one ordered her to add this chapter. We shall see why this chapter exists, the book bought by Glanz has her drawing of Stacy (Valhalla Projection' subordinate in Coppito, L'Aquila and Todai / University of Tokyo).

There are a lot of books talking about Partial Differential Equations, a lot of people wrote in their blogs and websites, and create the video to explain it. We read and watch a lot then try to summarize it after we able to comprehend. Hopefully, after certain time if we read it again, the memory will be refreshed quickly.

I. Preliminaries

[H*] In many important physical problems there are two or more independent variables governing the process, thus the corresponding mathematical models involve partial differential equations. For solving partial differential equations we use a method separation of variables, with its essential feature to replace the partial differential equation by a set of ordinary differential equations, which then will be solved by the given initial or boundary conditions.

The desired solution of the partial differential equation is expressed as a sum, or an infinite series, formed from solutions of the ordinary differential equations.

We will call Partial Differential Equation with PDE, interchangeably.
[H*]

II. Derivation of Partial Differential Equations

[H*] Partial differential equations arise in many physical problems. Three known partial differential equations are Heat equation, wave equation, and Laplace's equation.

[H*] The key defining property of a partial differential equation (PDE) [19] is that

- There is more than one independent variable $x, y, \dots$
- There is a dependent variable that is an unknown function of these variables $u(x, y, \dots)$
- $\partial u / \partial x$ denoted as the partial derivative of the dependent variable toward the independent variable x.

[H*] When a solution function has two or more independent variables, say $u(x, y)$, then this function can make a partial differential equation by specifying a relation between its' derivatives [10], such as

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0$$

Another way of writing partial differential equation is to see it as an identity that relates the independent variables, the dependent variable u, and the derivatives of u. It can be written as

$$F(x, y, u, u_x, u_y) = 0 \tag{6.1}$$

with $u = u(x, y)$, a function in x and y terms as the independent variables.
The general second-order PDE in two independent variables is

$$F(x, y, u, u_x, u_y, u_{xx}, u_{xy}, u_{yy}) = 0 \tag{6.2}$$

A solution of a PDE is a function $u(x, y, \dots)$ that satisfies the equation identically.

[H*] For PDE, the distinction between the independent variables and the dependent variable (the unknown / the solution) is always maintained.

[H*] An example of a general linear second-order partial differential equation is like this:

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial u}{\partial t} = 0$$

The general nonlinear second-order partial differential equation:

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial u}{\partial t} + u^2 = 0$$

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial u}{\partial t} + \sin(u) = 0$$

The differences are:

1. The linear partial differential equation will contain the unknown / the solution with power of 1, the nonlinear will contain the unknown / the solution with power other than 1 (could be 2 or argument of $exp, \log, \sin$).

[H*] A linear partial differential equation is homogeneous if the unknown or the solution that satisfies the partial differential equation is 0, for example:

$$\frac{\partial^2 u}{\partial t^2} + c^2 \frac{\partial^2 u}{\partial x^2} = 0$$

and the solution of the partial differential equation above is $u(x,t) = 0$, thus the partial differential equation is homogeneous.

A homogeneous linear differential equation has an important property:

$$\alpha u + \beta v \tag{6.3}$$

are solutions for $\alpha, \beta \in \mathcal{R}$, whenever u and v are solutions of the equation.

A linear partial differential equation is non-homogeneous if the unknown or the solution that satisfies the partial differential equation is not 0, for example:

$$\frac{\partial u}{\partial t} = 2u + \sin(4\pi t)$$

is equivalent to

$$\frac{\partial u}{\partial t} - 2u = \sin(4\pi t)$$

When all terms involving the unknown / the solution on the left side of equation and the right side is not 0 then it is non-homogeneous

[H*] If u and v is any function and $\mathcal{L}$ is an operator, then $\mathcal{L}v$ is a new function. Thus the definition for linearity is

$$\mathcal{L}(u+v) = \mathcal{L}v + \mathcal{L}v\mathcal{L}(cu) = c(u) \tag{6.4}$$

Hence

$$\mathcal{L}(u) = 0 \tag{6.5}$$

is called a homogeneous linear equation. Then

$$\mathcal{L}(u) = g \tag{6.6}$$

where $g \neq 0$ is a given function of the independent variables, is called a non-homogeneous linear equation, for any functions u, v and any constant c.

[H*] Recall this Taylor series expansion of a function $f(z)$ about the point $z = a$, the expansion is given by

$$f(z) = f(a) + f'(a) + f'(a)(z-a) + f''(a)\frac{(z-a)^2}{2!} + f'''(a)\frac{(z-a)^3}{3!}$$

where $f'(a)$ means the derivative of $f(z)$ with respect to z evaluated at the point $z = a$, $f''(a)$ is the second derivative of $f(z)$ with respect to z evaluated at the point $z = a$, etc. If, in the above equation we let

$$z = a + h \quad \text{and} \quad a = x$$

we obtain

$$f(x+h) = f(x) + f'(x)h$$

Thus the value of $f(x)$ at the point $x + h$ (near to x) is given by the value of the function at the point x plus the value of its first derivative evaluated at x, multiplied by the distance $[(x+h) - x = h]$ between the two points.

[H*] The partial derivative of a function $u(x,y)$ with respect to x at the point x, is denoted by

$$\frac{\partial u}{\partial x}$$

[H*] The general solution of the partial differential equation involves as many arbitrary functions as the order of the equation (the order of the highest partial differential coefficient in the equation), for example:

$$\frac{\partial u(x,t)}{\partial t} = -D\frac{\partial^2 u(x,t)}{\partial x^2}$$

has order of 2, we can also call it second-order partial differential equation, this form of PDE is known as the Heat Equation.

[H*] If $w(x,y,z,t)$ is a function of time and the three rectangular coordinates of a point in space, then the following are partial differential equations with order of 2:

$$\frac{\partial^2 w}{\partial x^2} + \frac{\partial^2 w}{\partial y^2} + \frac{\partial^2 w}{\partial z^2} = 0$$
$$a^2\left(\frac{\partial^2 w}{\partial x^2} + \frac{\partial^2 w}{\partial y^2} + \frac{\partial^2 w}{\partial z^2}\right) = \frac{\partial w}{\partial t}$$
$$a^2\left(\frac{\partial^2 w}{\partial x^2} + \frac{\partial^2 w}{\partial y^2} + \frac{\partial^2 w}{\partial z^2}\right) = \frac{\partial^2 w}{\partial t^2}$$

The first equation is called Laplace's equation, the second is the heat equation in 3-dimension, and the third is the wave equation in 3-dimension.

[H*] The conditions of partial differential equations are classified into two categories [1]:

1. **Initial Value Problem (IVP)**
   A partial differential equation with initial conditions will have a dependent variable and a sufficient number of its derivatives are prescribed at the initial point of domain.
2. **Boundary Value Problem (BVP)**
   A partial differential equations with its dependent variable and a sufficient number o its derivatives are prescribed at the boundary of domain.

   For example consider this heat equation or a metal rod:

$$\frac{\partial u(x,t)}{\partial t} = -D\frac{\partial^2 u(x,t)}{\partial x^2}, \quad 0 \le x \le 20,\ t > 0$$

   with $u(x,t)$ denotes the temperature of the rod at length-x at time t. The length of the rod is 20 cm.
   A lot of people sometimes write u to replace $u(x,t)$, it could cause confusion to those newly learning differential equation, we prefer to write the whole as $u(x,t)$ to denote all the dependent variables that is used by the function u. Just like when we first learning calculus we write $f(x)$ not only writing f.

   Now for the initial condition, it will be written like this:

$$u(x,0) = \begin{cases} x, & 0 \le x \le 10 \\ 20 - x, & 10 \le x \le 20 \end{cases}$$

   and the boundary conditions:

$$u(0,t) = 0$$
$$u(L,t) = 0$$

both ends of the metal rod will be cooled at 0^0 perhaps attaching both ends at ice cube. From this we can learn how the temperature will behave on the rod after certain period of time.

[H*] Consider this PDE:

$$\frac{\partial^2 u(x,t)}{\partial t^2} - \frac{\partial^2 u(x,t)}{\partial x^2} = 0, \quad 0 \leq x \leq L,\ t > 0$$

There are four broad categories of boundary conditions:

1. **Dirichlet boundary conditions**
   On the boundary, the values of the dependent variable are specified.
   $$u(0,t) = 0, \quad u(L,t) = 0$$
2. **Neumann boundary conditions**
   On the boundary, the normal derivative of the dependent variable is specified.
   $$\frac{\partial u(0,t)}{\partial x} = 0, \quad \frac{\partial u(L,t)}{\partial x} = 0$$
3. **Cauchy boundary conditions**
   On the boundary, both the values of the dependent variable and its normal derivative are specified.
   $$u(0,t) = 0, \quad \frac{\partial u(L,t)}{\partial x} = 0$$
4. **Robin boundary conditions**
   On the boundary, a linear combination of the dependent variable and its normal derivatives are specified.
   $$u(0,t) = 1, \quad \frac{\partial u(L,t)}{\partial x} + u(L,t) = 0$$

[H*] A second-order partial differential equation of the form

$$Au_{xx} + Bu_{xy} + Cu_{yy} + Du_x + Eu_y + Fu < 0$$

is elliptic, while

$$Au_{xx} + Bu_{xy} + Cu_{yy} + Du_x + Eu_y + Fu = 0$$

is parabolic, and

$$Au_{xx} + Bu_{xy} + Cu_{yy} + Du_x + Eu_y + Fu > 0$$

is hyperbolic.

[H*] To have a unique, stable solution, each class of PDEs requires a different class of boundary conditions

1. Dirichlet or Neumann boundary conditions on a closed boundary surrounding the region of interest are the requirements to elliptic equations. Other boundary conditions are either insufficient to determine a unique solution, overly restrictive or lead to instabilities.
2. Cauchy boundary conditions on an open surface are the requirements to elliptic equations. Other boundary conditions are either too restrictive for a solution to exist, or insufficient to determine a unique solution.

3. Parabolic equations require Dirichlet or Neumann boundary conditions on an open surface. Other boundary conditions are too restrictive.

- Two-Point Boundary Value Problems

[H*] Consider this differential equation of second order

$$y'' + p(t)y' + q(t)y = g(t) \tag{6.7}$$

with the initial conditions

$$y(t_0) = y_0 \quad y'(t_0) = y_0' \tag{6.8}$$

[H*] Consider this independent variables: $x, y, z, \dots$
and now an unknown function of these variables can be written as below

$$u(x, y, \dots) \tag{6.9}$$

We denote the derivative with subscript, for example a derivative of u with respect to the independent variable x is

$$\frac{\partial u}{\partial x} = u_x \tag{6.10}$$

for y:

$$\frac{\partial u}{\partial y} = u_y \tag{6.11}$$

[H*] **Partial Differential Equation**
A PDE is an identity that relates the independent variables $(x, y, \dots)$ with the dependent variable u, and the partial derivatives of u. It can be written as

$$F(x, y, u(x, y), u_x(x, y), u_y(x, y)) = F(x, y, u, u_x, u_y) = 0 \tag{6.12}$$

This is the most general PDE in two independent variables x and y, of first order.

[H*] **Order**
The order of an equation is the highest derivative that appears.

For example:

* $u_t + 2u_x$ is a first order partial differential equation
* $u_{xx} - u_t$ is a second order partial differential equation

[H*] **Solution**
A solution to a PDE is a function $u(x, y, \dots)$ that satisfies the equation identically.

In PDEs, the distinction between the independent variables and the dependent variable is always maintained.

III. Fourier Series

- General Theory of nth Order Linear Equations

[H*] The first
[H*]

- The Method of Variation of Parameters

[H*] The first
[H*]

IV. Even and Odd Functions

- General Theory of nth Order Linear Equations

[H*] The first
[H*]

- The Method of Variation of Parameters

[H*] The first
[H*]

V. The Wave Equation

- General Theory of nth Order Linear Equations

[H*] The first
[H*]

- The Method of Variation of Parameters

[H*] The first
[H*]

VI. The Heat Equation

- General Theory of nth Order Linear Equations

[H*] This heat equation is one of the most famous partial differential equation that we will try to simulate, the applications are tremendous, from Black-Scholes equation derived from this, create a long lasting ice cream, to be able to create better computer components will need the comprehension of this equation as well.
[H*]

- The Method of Variation of Parameters

[H*] The first
[H*]

VII. The Laplace Equation

- General Theory of nth Order Linear Equations

[H*] The first
[H*]

- The Method of Variation of Parameters

[H*] The first
[H*]

VIII. The Navier-Stokes Equation

- Introduction

[H*] The Navier-Stokes equations are partial differential equations which describe the motion of viscous fluid substances, named after French engineer and physicist Claude-Louis Navier and Anglo-Irish physicist and mathematician Georrge Gabriel Stokes. They were developed over several decades of progressively building the theories, from 1822 (Navier) to 1842-1850 (Stokes)

[H*] The Navier-Stokes equations mathematically express momentum balance and conservation of mass for Newtonian fluids. They are sometimes accompanied by an equation of state relating pressure, temperature and density. They are arise from applying Isaac Newton's second law to fluid motion, together with the assumption that the stress in the fluid is the sum of a diffusing viscous term (proportional to the gradient of velocity) and a pressure term-hence describing viscous flow.

The difference between them and the closely related Euler equations is that Navier-Stokes equations take viscosity into account while the Euler equations model only inviscid flow. As a result, the Navier-Stokes are a parabolic equation and therefore have better analytic properties, at the expense of having less mathematical structure (e.g. they are never completely integrable).

[H*] The Navier-Stokes equations are useful because they describe the physics of many phenomena of scientific and engineering interest. They may be used to model the weather, ocean currents, water flow in a pipe and air flow around a wing. The Navier-Stokes equations, in their full and simplified forms, help with the design of aircraft and cars, the study of blood flow, the design of power stations, the analysis of pollution, and many other things. Coupled with Maxwell's equations, they can be used to model and study magnetohydrodynamics.

Despite their wide range of practical uses, it has not yet been proven whether smooth solutions always exist in three dimensions, i.e., whether they are infinitely differentiable (or even just bouned) at all points in the domain.

- Flow Velocity

[H*] The solution of the equations is a flow velocity. It is a vector field-to every point in a fluid, at any moment in a time interval, it gives a vector whose direction and magnitude are those of the velocity of the fluid at that point in space and at that moment in time.

[H*]

- Cross-Sectional Area Computation

[H*] The cross-sectional area is the area of a two-dimensional shape that results from slicing a three-dimensional object perpendicular to a given axis. Most of the time in three dimensions we will need to determine the cross-sectional area for engineering problem (electrical conductivity, strength of materials), in Navier-Stokes it will be used to compute the cross-sectional area of a fluid that is flowing through a pipe or a hose.

Example: Loaf of bread, the surface of a slice of bread is the cross-sectional area.

[H*] To take a cross-sectional area, first you decide the plane that will be the slice / the area that the water will go through, in three dimension we have 3 planes: xy-plane, yz-plane, and xz-plane.

Then cut at the axis that is not the basis of the plane / building the plane. For example, xy-plane is built by x axis and y axis, so the axis that is not the basis is z.

Now we will use a cylinder in three dimension, for better illustration.

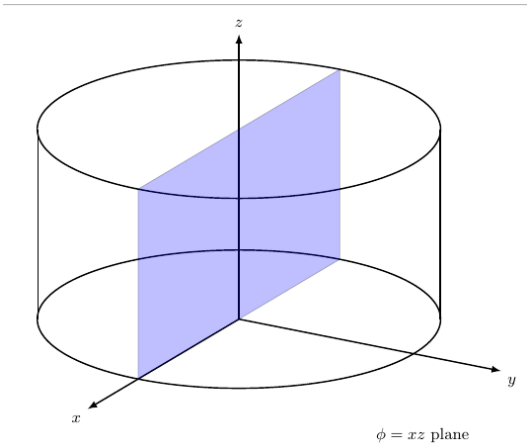

Figure 6.1: *The cross-sectional area of the cylinder in this figure is the blue plane ϕ = xz-plane, it is built by x and z axis, and we cut it at $y = 0$.*

The cross-sectional area of Figure (6.1) is

$$A = 2r * h$$

with r as the radius of the circle and h is the cylinder height. It is only a theory to choose this cross-sectional area, if we make a fluid flow through a cylinder along the y-axis, then the cylinder will have to turn into a box, to comply with the cross-sectional area. As a rule of thumb, if we have a cylinder in three dimension, the water will always flow through the cross-sectional area of a circle, which in the Figure (6.1) it is the area made by xy-plane, then cut at $z = c$,with c as constant.

Chapter 7

Probability and Statistics

*"LAM VAM RAM YAM HAM OM AH." - **7 Chakras Chanting***

D

I. Statistics and Data Analysis

Statistical methods are used to analyze data from an industrial process (manufacturing, energy sources, drug testing). Statistical methods are involved with gathering of information or **scientific data**, which then will be processed to extract the knowledge from the data and improve the **quality** of the process.

II. Variability

Life would be simple for scientists if the observed product were always the same and were always on target, there would be no need for for statistical methods. Statisticians make use of fundamental laws of probability and statistical inference to draw conclusions about scientific systems.

Information is gathered in the form of **samples**, or collections of **observations**. Samples are collected from **populations**, which are collections of all individuals or individual items of a particular type.

Research scientists gain much from scientific data. Data provide understanding of scientific phenomena.

Chapter 8

Linear and Nonlinear Programming

"My only place is by your side. If you want to carry out your wish, I will follow you. That is my Nature." -
Sarah (Suikoden III)

Linear programming is an optimization problem, a problem that is well rooted as a principle underlying the analysis of many complex decision or allocation problems. For example, if we have limited resources and we want to achieve a certain goal, we need to learn to optimize our resources with either linear programming or nonlinear programming depends on our problems at hand. Like how we decide to buy groceries depending on our needs / diet (the goal), and the money we have (the constraints).

Optimization, also known as mathematical programming, collection of mathematical principles and methods used for solving quantitative problems in many disciplines, including physics, biology, engineering, economics, and business. The subject grew from a realization that quantitative problems in manifestly different disciplines have important mathematical elements in common. Because of this commonality, many problems can be formulated and solved by using the unified set of ideas and methods that make up the field of optimization.

Optimization problems typically have three fundamental elements. The first is a single numerical quantity, or objective function, that is to be maximized or minimized. The objective may be the expected return on a stock portfolio, a company's production costs or profits, the time of arrival of a vehicle at a specified destination, or the vote share of a political candidate. The second element is a collection of variables, which are quantities whose values can be manipulated in order to optimize the objective. Examples include the quantities of stock to be bought or sold, the amounts of various resources to be allocated to different production activities, the route to be followed by a vehicle through a traffic network, or the policies to be advocated by a candidate. The third element of an optimization problem is a set of constraints, which are restrictions on the values that the variables can take. For instance, a manufacturing process cannot require more resources than are available, nor can it employ less than zero resources. Within this broad framework, optimization problems can have different mathematical properties. Problems in which the variables are continuous quantities (as in the resource allocation example) require a different approach from problems in which the variables are discrete or combinatorial quantities (as in the selection of a vehicle route from among a predefined set of possibilities).

Chapter 9

Introduction to Theory of Interest and Financial Mathematics

"In my world everything what would be it wouldn't be, and what wouldn't be it would." (Alice in Wonderland 1951) - ***Alice***

Mathematical finance, also known as quantitative finance and financial mathematics, is a field of applied mathematics, concerned with mathematical modeling of financial markets.

In general, there exist two separate branches of finance that require advanced quantitative techniques: derivatives pricing on the one hand, and risk and portfolio management on the other. Mathematical finance overlaps heavily with the fields of computational finance and financial engineering. The latter focuses on applications and modeling, often by help of stochastic asset models, while the former focuses, in addition to analysis, on building tools of implementation for the models. Also related is quantitative investing, which relies on statistical and numerical models (and lately machine learning) as opposed to traditional fundamental analysis when managing portfolios.

French mathematician Louis Bachelier's doctoral thesis, defended in 1900, is considered the first scholarly work on mathematical finance. But mathematical finance emerged as a discipline in the 1970s, following the work of Fischer Black, Myron Scholes and Robert Merton on option pricing theory. Mathematical investing originated from the research of mathematician Edward Thorp who used statistical methods to first invent card counting in blackjack and then applied its principles to modern systematic investing.

I. Bond Pricing

•

Chapter 10

Introduction to Stochastic Processes

"One side will make you taller, the other side will make you shorter" (Alice in Wonderland 1951) - ***The Caterpillar***

In probability theory and related fields, a stochastic or random process is a mathematical object usually defined as a sequence of random variables, where the index of the sequence has the interpretation of time. Stochastic processes are widely used as mathematical models of systems and phenomena that appear to vary in a random manner. Examples include the growth of a bacterial population, an electrical current fluctuating due to thermal noise, or the movement of a gas molecule. Stochastic processes have applications in many disciplines such as biology, chemistry, ecology, neuroscience, physics, image processing, signal processing, control theory, information theory, computer science, and telecommunications. Furthermore, seemingly random changes in financial markets have motivated the extensive use of stochastic processes in finance.

Applications and the study of phenomena have in turn inspired the proposal of new stochastic processes. Examples of such stochastic processes include the Wiener process or Brownian motion process, used by Louis Bachelier to study price changes on the Paris Bourse, and the Poisson process, used by A. K. Erlang to study the number of phone calls occurring in a certain period of time. These two stochastic processes are considered the most important and central in the theory of stochastic processes, and were discovered repeatedly and independently, both before and after Bachelier and Erlang, in different settings and countries.

Chapter 11

Numerical Analysis

Je n'avais pas peur, je suis né pour faire ça - Jeanne d'Arc

Numerical analysis is the study of algorithms that use numerical approximation (as opposed to symbolic manipulations) for the problems of mathematical analysis (as distinguished from discrete mathematics). It is the study of numerical methods that attempt at finding approximate solutions of problems rather than the exact ones. Numerical analysis finds application in all fields of engineering and the physical sciences, and in the 21st century also the life and social sciences, medicine, business and even the arts.

Modern numerical analysis begins by categorizing equations as belonging to different types and then devises methods of solution for generic problems of each type.

I. Solutions of Equations in One Variable

- Root Finding Methods
  In this section, we consider one of the most basic problems of numerical approximation, the root-finding problem. This process involves finding a root, or solution of an equation of the form

$$f(x) = 0$$

  for a given function f.

Definition 11.1: Bisection Method

The Bisection Method is based on the Intermediate Value Theorem. Suppose f is a continuous function defined on the interval $[a, b]$ with $f(a)$ and $f(b)$ of opposite sign. By the Intermediate Value Theorem, there exists a number p in (a, b) with

$$f(p) = 0$$

We assume for simplicity that the root in the interval $[a, b]$ is unique. This method calls for a repeated halving of subintervals of $[a, b]$.

[H*] The bisection method is slow to converge, N may become quite large before $|p - p_N|$ is sufficiently small, and a good intermediate approximation can be inadvertently discarded.

[H*] The bisectionmethod has the important property that it always converges to a solution.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Bisection Method with Function ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Bisection Method with Function ]# ./main
f = -cos(θ0)-2*θ0+2*sin(θ0)+1
iteration            a                    b                    p                    f(p)
     1               1                    2                    1.5                  -0.0757472

     2               1                    1.5                  1.25                 0.0826469

     3               1.25                 1.5                  1.375                0.0172384

     4               1.375                1.5                  1.4375               -0.0256436

     5               1.375                1.4375               1.40625              -0.00331926

     6               1.375                1.40625              1.39062              0.00717788

     7               1.39062              1.40625              1.39844              0.00198421

     8               1.39844              1.40625              1.40234              -0.000653763

     9               1.39844              1.40234              1.40039              0.000668658

    10               1.40039              1.40234              1.40137              8.30682e-06

    11               1.40137              1.40234              1.40186              -0.000322513

    12               1.40137              1.40186              1.40161              -0.00015705

    13               1.40137              1.40161              1.40149              -7.43579e-05
Procedure completed successfully
solution = 1.40149

Time taken by function: 30471 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Bisection Method with Function ]#
```

Figure 11.1: *The computation for bisection method for* $f(\theta_0) = 2\theta_0 - 2\sin\ \theta_0 + \cos\ \ theta_0 - 1$ *with function (**SymIntegration/Examples/Test SymIntegration Bisection Method with Function for Brachistochrone/main.cpp**).*

The computation is using SymIntegration, with this function: **bisectionmethod(**f**,** x**,** a**,** b**,**N**)** that can be called to compute the the solution of a function with one variable, this method doesn't need to compute the derivative so it can be used to almost all functions.

As a comparison we try to measure the time if we apply the bisection method directly in the main source code, turns out it is faster than using the function.

```
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Bisection Method ]# ./main
f = -cos(00)-2*00+2*sin(00)+1

a = 1
b = 2
p = 1.5
f(p) = -0.0757472
f(b) = -0.765258
f(a) = 0.14264
f(a)*f(p) =-0.0108046
iteration               a                       b                       p                       f(p)
     1                  1                       2                       1.5                     -0.0757472

     2                  1                       1.5                     1.25                    0.0826469

     3                  1.25                    1.5                     1.375                   0.0172384

     4                  1.375                   1.5                     1.4375                  -0.0256436

     5                  1.375                   1.4375                  1.40625                 -0.00331926

     6                  1.375                   1.40625                 1.39062                 0.00717788

     7                  1.39062                 1.40625                 1.39844                 0.00198421

     8                  1.39844                 1.40625                 1.40234                 -0.000653763

     9                  1.39844                 1.40234                 1.40039                 0.000668658

    10                  1.40039                 1.40234                 1.40137                 8.30682e-06

    11                  1.40137                 1.40234                 1.40186                 -0.000322513

    12                  1.40137                 1.40186                 1.40161                 -0.00015705

    13                  1.40137                 1.40161                 1.40149                 -7.43579e-05
Procedure completed successfully

Time taken by function: 28906 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Bisection Method ]#
```

Figure 11.2: *The computation for bisection method for* $f(\theta_0) = 2\theta_0 - 2\sin\ \theta_0 + \cos\ \ theta_0 - 1$ *without function (**SymIntegration/Examples/Test SymIntegration Bisection Method Manual for Brachistochrone/-main.cpp**).*

Definition 11.2: Newton-Raphson Method

The Newton-Raphson Method is one of the most powerful and well known numerical methods for solving a root-finding problem.
Suppose that $f \in \mathbb{C}^2[a,b]$. Let $\bar{x} \in [a,b]$ be an approximate to p such that $f'(\bar{x}) \neq 0$ and $|p - \bar{x}|$ is "small." Consider the first Taylor polynomial for $f(x)$ expanded about $(\bar{x})$,

$$f(x) = f(\bar{x}) + (x - \bar{x})f'(\bar{x}) + \frac{(x-\bar{x})^2}{2}f''(\xi(x))$$

where $\xi(x)$ lies between x and $\bar{x}$. Since $f(p) = 0$, this equation with $x = p$ gives

$$0 = f(\bar{x}) + (p - \bar{x})f'(\bar{x}) + \frac{(p-\bar{x})^2}{2}f''(\xi(p))$$

Newton's method is derived by assuming that since $|p - \bar{x}|$ is small, the term involving $(p - \bar{x})^2$ is much smaller, so

$$0 \approx f(\bar{x}) + (p - \bar{x})f'(\bar{x})$$

Solving for p gives

$$p \approx \bar{x} - \frac{f(\bar{x})}{f'(\bar{x})}$$

Thus, the Newton-Raphson method, which starts with an initial approximation p_0 and generate the sequence $\{p_n\}_{n=0}^{\infty}$, is

$$p_n = p_{n-1} - \frac{f(p_{n-1})}{f'(p_{n-1})}, \quad \text{for } n \geq 1 \tag{11.1}$$

Starting with the initial approximation p_0, the approximation p_1 is the x-intercept of the tangent line to the graph of f at $(p_0, f(p_0))$. The approximation p_2 is the x-intercept of the tangent line to the graph of f at $(p_1, f(p_1))$ and so on.

[H*] Newton-Raphson method is a functional iteration technique of the form

$$p_n = g(p_{n-1})$$

for which

$$g(p_{n-1}) = p_{n-1} - \frac{f(p_{n-1})}{f'(p_{n-1})}, \quad \text{for } n \geq 1 \tag{11.2}$$

this is the functional iteration technique that was used to give the rapid convergence.

[H*] It is clear that Newton-Raphson method cannot be continued if $f'(p_{n-1}) = 0$ for some n. This method is the most effective when f' is bounded away from zero near p.

Figure 11.3: *The plot for root finding with bisection method (above) for* $f(x) = x^3 + 4x^2 - 10$ *and the Newton-Raphson method (below) for* $f(x) = \cos(x) - x$. *(**Hamzstlab-Mathematics/examples/Root Finding/-main.cpp**).*

The computation is using SymIntegration, with this function: **newtonmethod(**f**,**x**,**x_0**,**N**)** that can be called to compute the the solution of a function with one variable, given that the derivative is continuous and exist.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Newton's Method with function ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Newton's Method with function ]# ./main
Newton method :
f(x) = cos(x)-x
f'(x) = -sin(x)-1

     n          p_{n}
     0          0.78539816339742
     1          0.73953613351524
     2          0.73908517810601
     3          0.73908513321516
The procedure was successful.
solution =
0.73908513321516
Time taken by function: 5645 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Newton's Method with function ]#
```

Figure 11.4: *The computation for root finding Newton-Raphson method for* $f(x) = \cos(x) - x$. *with function (**SymIntegration/Examples/Test SymIntegration Newton's Method with function/main.cpp**).*

As a comparison we try to implement Newton-Raphson manually, in the main source code, it turns out calling the function resulting in faster speed.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Newton's Method Manual ]# make
g++     -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Newton's Method Manual ]# ./main
f(x) = cos(x)-x
f(x)[x==pi/4] = -0.0782914

f'(x) = -sin(x)-1
f'(x)[x==pi/4] = -1.70711

     n          p_{n}                          p_{n} in Degree
     0          0.78539816339742               44.999999999999
     1          0.73953613351524               42.372299247846
     2          0.73908517810601               42.346461406149
     3          0.73908513321516               42.346458834093
The procedure was successful.
Time taken by function: 6779 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Newton's Method Manual ]#
```

Figure 11.5: *The manual computation for root finding Newton-Raphson method for $f(x) = \cos(x) - x$. (**SymIntegration/Examples/Test SymIntegration Newton's Method Manual/main.cpp**).*

II. Interpolation and Polynomial Approximation

- Interpolation and the Lagrange Polynomial

  Taylor polynomials are not appropriate for interpolation, because the approximations are needed only at points close to x_0. For ordinary computational purposes it is more efficient to use methods that include information at various points. To generalize the concept of linear interpolation, consider construction of a polynomial of degree at most n that passes through $n+1$ points

  $$(x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n)$$

  to approximate that $n+1$ points, we will use the well-established Lagrange Interpolating Polynomial.

Theorem 11.1: nth Lagrange Interpolating Polynomial

If $x_0, x_1, \ldots, x_n$ are $n+1$ distinct numbers and f is a funciton whose values are given at these numbers, then a unique polynomial $P(x)$ of degreen at most n exists with

$$f(x_k) = P(x_k)$$

for each $k = 0, 1, \ldots, n$ This polynomial is given by

$$P(x) = f(x_0)L_{n,0}(x) + \cdots + f(x_n)L_{n,n}(x) = \sum_{k=0}^{n} f(x_k)L_{n,k}(x) \tag{11.3}$$

where, for each $k = 0, 1, \ldots, n$,

$$\begin{aligned} L_{n,k}(x) &= \frac{(x-x_0)(x-x_1)\ldots(x-x_{k-1})(x-x_{k+1})\ldots(x-x_n)}{(x_k-x_0)(x_k-x_1)\ldots(x_k-x_{k-1})(x_k-x_{k+1})(x_k-x_n)} \\ &= \prod_{i=0,\ i\neq k}^{n} \frac{(x-x_i)}{(x_k-x_i)} \end{aligned} \tag{11.4}$$

- **Example:**

  The example is taken from example 1, chapter 3.1 of [3].

  Use the numbers (or nodes) $x_0 = 2, x_1 = 2.5$, and $x_2 = 4$ to find the second interpolating

polynomial for $f(x) = \frac{1}{x}$.

Solution:
We need to determine the coefficient polynomials $L_0(x), L_1(x)$, and $L_2(x)$. In nested form they are

$$\begin{aligned} L_0(x) &= \frac{(x-2.5)(x-4)}{(2-2.5)(2-4)} = (x-6.5)x+10 \\ L_1(x) &= \frac{(x-2)(x-4)}{(2.5-2)(2-4)} = \frac{(-4x+24)x-32}{3} \\ L_2(x) &= \frac{(x-2)(x-2.5}{(4-2)(4-2.5)} = \frac{(x-4.5)x+5}{3} \end{aligned}$$

Since

$$\begin{aligned} f(x_0) &= f(2) = 0.5 \\ f(x_1) &= f(2.5) = 0.4 \\ f(x_2) &= f(4) = 0.25 \end{aligned}$$

we will have

$$\begin{aligned} P(x) &= \sum_{k=0}^{2} f(x_k) L_k(x) \\ &= 0.5\left((x-6.5)x+10\right) + 0.4\left(\frac{(-4x+24)x-32}{3}\right) + 0.25\left(\frac{(x-4.5)x+5}{3}\right) \\ &= (0.05x-0.425)x+1.15 \end{aligned}$$

We can now check with $x = 3$

$$\begin{aligned} f(3) &= \frac{1}{3} \approx 0.33333 \\ P(3) &= 0.325 \end{aligned}$$

it is quite a good approximation.

The C++ code is using SymIntegration and Armadillo to help us compute the polynomial Lagrange $P(x)$, we can input the initial nodes $x_0, x_1, \ldots, x_N$ from textfile **vectorX.txt**, and then the function $f(x)$ can be inputted in the main source code symbolically.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Lagrange Interpolating Polynomial with Armadillo ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -larmadillo -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Lagrange Interpolating Polynomial with Armadillo ]# ./main
Vector x:
   2.0000
   2.5000
   4.0000

f(x) = x^(-1)

Vector f:
   0.5000
   0.4000
   0.2500

L :
[       x^(2)-6.5*x+10       ]
[ -1.33333*x^(2)+8*x-10.6667 ]
[0.333333*x^(2)-1.5*x+1.66667]


P(x) = 0.05*x^(2)-0.425*x+1.15
P(3) = 0.325
f(3) = 1/3

Time taken by function: 39599 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Lagrange Interpolating Polynomial with Armadillo ]#
```

Figure 11.6: *The computation to approximate $f(x) = \frac{1}{x}$ with nodes $x_0 = 2, x_1 = 2.5$, and $x_2 = 4$ using Lagrange interpolating polynomial with SymIntegration and Armadillo (**SymIntegration/Examples/Test SymIntegration Lagrange Interpolating Polynomial with Armadillo/main.cpp**).*

Figure 11.7: *The plot for Lagrange interpolating polynomial with Hamzstlab, SymIntegration and Armadillo for $f(x) = \frac{1}{x}$ with nodes $x_0 = 2, x_1 = 2.5$, and $x_2 = 4$. (**Hamzstlab-Mathematics/examples/Interpolation/main.cpp**).*

III. Numerical Methods for Optimization

- Minimizing a single-variable function

[H*] The numerical methods for minimizing a single-variable function can be broadly categorized into interval reduction methods and descent methods.

Interval reduction methods, like Golden Section search, iteratively narrow down an interval known to contain the minimum, while descent methods, such as gradient

descent, use derivative information to iteratively move towards the minimum.

[H*] The interval reduction methods:

1. **Golden Section Search**
   This method is a bracketing method that works by iteratively narrowing down the interval containing the minimum. It uses the golden ratio to determine two points within the interval, evaluates the function at these points, and then discards a portion of the interval based on the function values. This process is repeated until the interval is sufficiently small.
2. **Fibonacci Search**
   Similar to Golden Section Search, Fibonacci Search uses the Fibonacci sequence to determine the points at which to evaluate the function. It offers optimal performance for a fixed number of function evaluations.

[H*] The descent methods:

1. **Gradient Descent**
   This method uses the gradient (derivative) of the function to determine the direction of steepest descent and iteratively moves towards the minimum. At each step, the algorithm moves in the opposite direction of the gradient.
2. **Newton's Method**
   This method uses both the first and second derivatives (gradient and Hessian) to find the minimum. It offers faster convergence than gradient descent but requires the calculation of the Hessian.

   Many algorithms for geometry optimization are based on some variant of the Newton-Raphson (NR) scheme. In order to find the minima, the first derivative $f'(x) = 0$ (must be zero) and the second derivative $f''(x) > 0$ (must be positive). To find the maxima, the first derivative $f'(x) = 0$ (must be zero) and the second derivative $f''(x) < 0$ (must be negative).

[H*] The other methods:

1. **Brent's Method**
   A hybrid method that combines the bracketing approach of Golden Section Search with the local quadratic approximation of Newton's method, offering a good balance of robustness and efficiency.
2. **Downhill Simplex Method (Nelder-Mead)**
   A direct search method that does not require derivative information. It works by iteratively moving and transforming a simplex (a geometric shape with $n + 1$ vertices in n-dimensional space) in the search space to find the minimum.

[H*] The best method for minimizing a single-variable function depends on the specific characteristics of the function and the desired level of accuracy. For well-behaved functions, gradient-based methods like Newton's method can offer fast convergence. For functions whose derivatives are difficult or impossible to compute, direct search methods like Brent's method or the downhill simplex method may be more suitable.

- Gradient Descent
  Gradient Descent (GD) is an iterative first-order optimisation algorithm, used to find a local maximum/minimum of a given function. This method is commonly used in machine learning (ML) and deep learning (DL) to minimize a cost/loss function (e.g. in a linear regression). Due to its importance and ease of implementation, this algorithm is usually

taught at the beginning of almost all machine learning courses. It is also widely used in areas like control engineering (robotics, chemical), computer games, mechanical engineering.

[H*] This method was proposed long before the era of modern computers by Augustin-Louis Cauchy in 1847.

[H*] Gradient descent algorithm does not work for all functions. There are two specific requirements. A function has to be:

1. Differentiable
   Differentiable means a function has a derivative for each point in its domain. Typical non-differentiable functions have a step or a cusp or a discontinuity, such as $f(x) = \frac{1}{x}, f(x) = \sqrt{|x|}$
2. Convex
   For a univariate function, this means that the line segment connecting two function's points lays on or above its curve (it does not cross it). If it does, then it means that it has a local minimum which is not a global one.
   In mathematics notation, for two points x_1 and x_2 laying on the function's curve this condition is expressed as:

$$f(\lambda x_1 + (1-\lambda)x_2) \leq \lambda f(x_1) + (1-\lambda) f(x_2) \tag{11.5}$$

   where λ denotes a point's location on a section line and its value has to be between 0 (left point) and 1 (right point), e.g. $\lambda = 0.5$ means a location in the middle.

   Another way to check mathematically if a univariate function is convex is to calculate the second derivative and check if itsvalue is always bigger than 0.

$$\frac{d^2 f(x)}{dx^2} > 0$$

[H*] A gradient fo an n-dimensional function $f(x)$ at a given point p is defined as

$$\nabla f(p) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(p) \\ \frac{\partial f}{\partial x_2}(p) \\ \vdots \\ \frac{\partial f}{\partial x_n}(p) \end{bmatrix} \tag{11.6}$$

The upside-down triangle is a notation called nabla, or you can also call it del operator.

[H*] **Gradient Descent Algorithm**
Gradient Descent Algorithm iteratively calculates the next point using gradient at the current position, scales it (by a learning rate) and subtracts obtained value from the current position (makes a step). It subtracts the value because we want to minimize the function (to maximize it would be adding). This process can be written as:

$$p_{n+1} = p_n - \gamma \nabla f(p_n) \tag{11.7}$$

There's an important parameter γ (read: gamma) which scales the gradient and thus controls the step size. In machine learning, gamma is called the learning rate and have a strong influence on performance.

The smaller the learning rate, the longer the GD converges, or may reach maximum iteration before reaching the optimum point.

If the learning rate is too big the algorithm may not converge to the optimal point (jump around) or even diverge completely.

The Gradient Descent method's steps are:

1. Choose a starting point (initialisation).
2. Calculate gradient at this point.
3. Make a scaled step in the opposite direction to the gradient (objective: minimize).
4. Repeat steps number 2 and 3 until one of the criteria is met:
   - Maximum number of iterations reached.
   - Step size is smaller than the tolerance (due to scaling or a small gradient)

[H*] **Example:**
Find the global minimum of a function that is given by

$$f(x) = x^2 - x + 3$$

Solution:
We will compute the first and second derivative of $f(x)$.

$$\frac{df(x)}{dx} = 2x - 1$$
$$\frac{d^2f(x)}{dx^2} = 2$$

Because the second derivative is always bigger than 0, $f(x)$ is strictly convex.

Now we examine the first derivative and equate it to zero.

$$\frac{df(x)}{dx} = 0$$
$$2x - 1 = 0$$
$$x = 0.5$$

$x = 0.5$ is the critical point.

Now, we will check with the gradient descent method, first we will choose the initial value and the parameter γ (the learning rate):

$$x_{old} = 2, \quad \gamma = 0.1$$

then we will have

$$x_{new} = x_{old} - \gamma \nabla f(x_{old})$$
$$x_{new} = 2 - 0.1(2(2) - 1) = 1.7 \quad (i = 1)$$
$$x_{new} = 1.7 - 0.1(2(1.7) - 1) = 1.46 \quad (i = 2)$$
$$x_{new} = 1.46 - 0.1(2(1.46) - 1) = 1.268 \quad (i = 3)$$
$$\vdots$$
$$x_{new} = 0.500027 - 0.1(2(0.500027) - 1) = 0.500021 \quad (i = 50)$$

It is proven then with the gradient descent method that the critical point is at $x = 0.5$. The computation can be done with **SymIntegration**, as long as the function $f(x)$ is differentiable and the derivative can be computed then we can use gradient descent method. Polynomials without the combination of trigonometry and transcendentals functions can be differentiated easily, even the fraction polynomial.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Gradient Descent
 ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Gradient Descent
 ]# ./main
F(x) = x^(2)-x+3
i = 1
xnew  = 1.7
f(xnew)  = 4.19
i = 2
xnew  = 1.46
f(xnew)  = 3.6716
i = 3
xnew  = 1.268
f(xnew)  = 3.33982
i = 4
xnew  = 1.1144
f(xnew)  = 3.12749
i = 5
xnew  = 0.99152
f(xnew)  = 2.99159
i = 6
xnew  = 0.893216
f(xnew)  = 2.90462
i = 7
xnew  = 0.814573
f(xnew)  = 2.84896
```

Figure 11.8: *The computation for minimization problem for function $f(x) = x^2 - x + 3$ with gradient descent method. (**SymIntegration/Examples/Test SymIntegration Gradient Descent/main.cpp**).*

```
i = 45
xnew  = 0.500065
f(xnew)  = 2.75
i = 46
xnew  = 0.500052
f(xnew)  = 2.75
i = 47
xnew  = 0.500042
f(xnew)  = 2.75
i = 48
xnew  = 0.500033
f(xnew)  = 2.75
i = 49
xnew  = 0.500027
f(xnew)  = 2.75
i = 50
xnew  = 0.500021
f(xnew)  = 2.75
The gradient descent converges.
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Gradient Descent ]#
```

Figure 11.9: *The gradient descent method took 50 iteration to converges with $\gamma = 0.1$. (**SymIntegration/Examples/Test SymIntegration Gradient Descent/main.cpp**).*

[H*] **Example:**
Find the global minimum of a function that is given by

$$f(x, y) = \frac{1}{2}x^2 + 2x + y^2 + y + 3$$

Solution:
In the case of a multivariate function, we need to compute the vector of derivatives in each main direction (along variable axes). Because we are interested only in a slope along one axis and we don't care about others, these derivatives are called partial derivatives.

We will have

$$z = f(x,y) = \frac{1}{2}x^2 + 2x + y^2 + y + 3$$
$$\frac{\partial f}{\partial x} = x + 2$$
$$\frac{\partial f}{\partial y} = 2y + 1$$

then the nabla is

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix} = \begin{bmatrix} x+2 \\ 2y+1 \end{bmatrix}$$

at initialization we will input the initial value and the parameter γ

$$x_{old} = -5$$
$$y_{old} = 21$$
$$\gamma = 0.1$$

thus the gradient descent method

$$\begin{aligned}
x_{new} &= x_{old} - \gamma \nabla f(x_{old}) \\
x_{new} &= -5 - 0.1((-5)+2) = -4.7 \quad (i=1) \\
x_{new} &= -4.7 - 0.1((-4.7)+2) = -4.43 \quad (i=2) \\
x_{new} &= -4.43 - 0.1((-4.43)+2) = -4.187 \quad (i=3) \\
&\vdots \\
x_{new} &= -2.01718 - 0.1((-2.01718)+2) = -2.01546 \quad (i=50)
\end{aligned}$$

$$\begin{aligned}
y_{new} &= y_{old} - \gamma \nabla f(y_{old}) \\
y_{new} &= 21 - 0.1(2(21)+1) = 16.7 \quad (i=1) \\
y_{new} &= 16.7 - 0.1(2(16.7)+1) = 13.26 \quad (i=2) \\
y_{new} &= 13.26 - 0.1(2(13.26)+1) = 10.508 \quad (i=3) \\
&\vdots \\
y_{new} &= -0.499616 - 0.1(2(-0.499616)+1) = -0.499693 \quad (i=50)
\end{aligned}$$

and the value for function $f(x,y)$ at every iteration

$$\begin{aligned}
f(x_{new}, y_{new}) &= 300.235 \quad (i=1) \\
f(x_{new}, y_{new}) &= 193.04 \quad (i=2) \\
f(x_{new}, y_{new}) &= 124.318 \quad (i=3) \\
&\vdots \\
f(x_{new}, y_{new}) &= 0.75012 \quad (i=50)
\end{aligned}$$

so the function $f(x,y) = \frac{1}{2}x^2 + 2x + y^2 + y + 3$ reaches the minimum of 0.75012 at $(-2.01546, -0.499693)$.

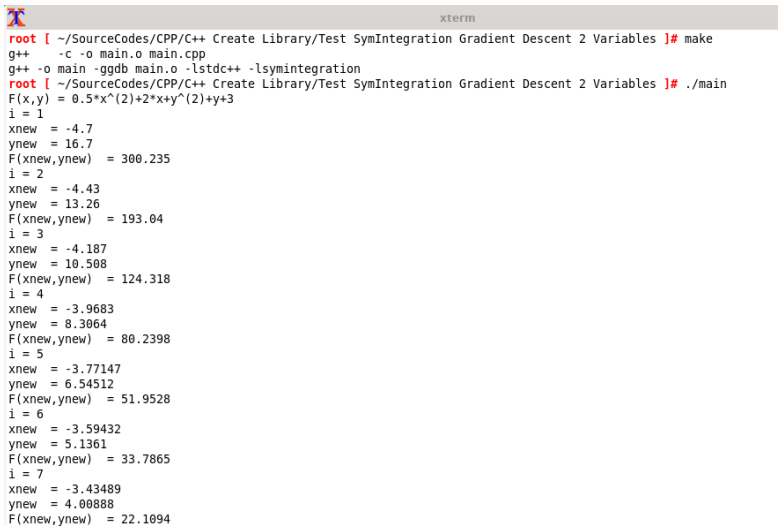

Figure 11.10: *The computation for minimization problem for function $f(x,y) = \frac{1}{2}x^2 + 2x + y^2 + y + 3$ with gradient descent method. (**SymIntegration/Examples/Test SymIntegration Gradient Descent 2 Variables/-main.cpp**).*

Figure 11.11: *The gradient descent method took 50 iteration to converges with $\gamma = 0.1$. (**SymIntegration/Examples/Test SymIntegration Gradient Descent 2 Variables/main.cpp**).*

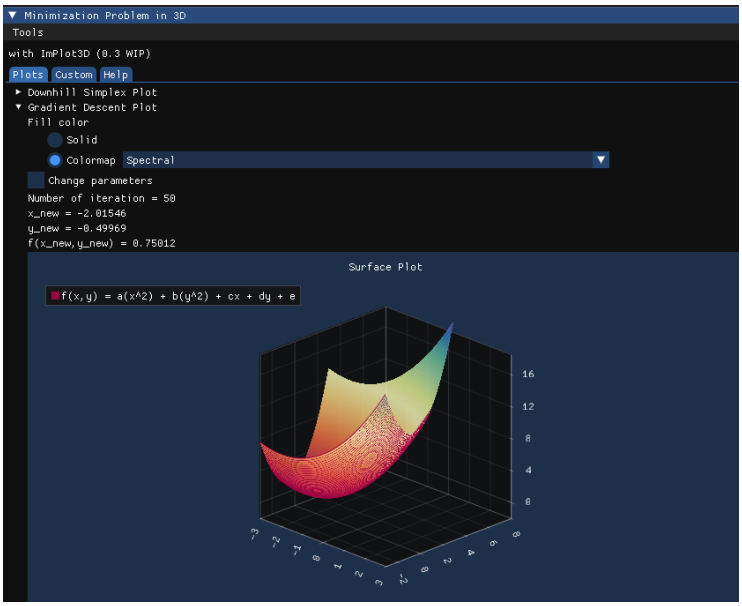

Figure 11.12: *The interactive surface plot for $f(x,y) = \frac{1}{2}x^2 + 2x + y^2 + y + 3$ with Hamzstlab Mathematics and SymIntegration combined so you can see the optimum / minimum value there. (**Hamzstlab-Mathematics/examples/3D Minimization/main.cpp**).*

- Downhill Simplex

  The downhill simplex, introduced by Nelder and Mead, also known as the Nelder-Mead' method [12], it is a method for solving multidimensional, unconstrained, numerical optimization problems. This type of problem is one of the most common in engineering and science, and this method is one of the simplest and fairly effective to solve the problem.

  [H*] We define a minimization problem as

$$\arg\ \min f(\boldsymbol{x}) \tag{11.8}$$

$$\boldsymbol{x} \in \mathbb{R}^N,\ f : \mathbb{R}^N \rightarrowtail \mathbb{R}^1$$

  [H*] **Method and simplex transforms**

  A simplex is a collection of connected points in $\mathbb{R}^N$. We call it a k-simplex where k is the degree that indicates how many points it contains. A k-simplex contains $k+1$ points, so a triangle which has 3 points is a 2-simplex,a tetrahedron which has 4 points is a 3-simplex and so on.

  When optimizing a function with N parameters we will use a simplex with $N+1$ points. Meaning the degree k of the simplex is the same as the dimension of the parameter space. So for a problem with 2 parameters, $N = 2$, we use a 2-simplex, a triangle, that has three points. A 1 parameter problem would use a line segment in 1D and a 3 parameter problem would use a tetrahedron in 3D.

  Each point on the simplex, denoted by $\boldsymbol{x}_i, 0 \leq i \leq N$, corresponds to a complete set of parameters and therefore to a value of the objective function $f_i = f(\boldsymbol{x}_i)$.

The core of the method is to keep the points sorted so that $f(\boldsymbol{x}_0)$ is the current lowest value, i.e. the best point, and $\boldsymbol{x}_N$ is the worst. We then calculate the center of mass of all points except $\boldsymbol{x}_N$:

$$\boldsymbol{c} = \frac{1}{N}\sum_{i=0}^{N-1} \boldsymbol{x}_i \tag{11.9}$$

Afterwards we attempt to replace the point $\boldsymbol{x}_N$ with one along the line formed by $\overrightarrow{\boldsymbol{x}_N \boldsymbol{c}}$ according to a set of transformations called reflection, expansion, and contraction. A shrink transform that moves all points closer to the current best can also be performed. We associate each transform with a parameter, α for reflection, β for contraction, γ for expansion and δ for shrink, that should satisfy the following constraints:

- $\alpha > 0$
- $0 < \beta < 1$
- $\gamma > \alpha$, $\gamma > 1$
- $0 < \delta < 1$

[H*] **Reflection**

The following illustrations are a 2D problem with a triangle simplex (can be used for function with two independent variable such as $f(x, y)$). The point $\boldsymbol{x}_0$ with blue color is the current best, the point $\boldsymbol{x}_1$ with green color is the second best, and the point $\boldsymbol{x}_2$ with light red color is the third best / the one being transformed except for when the simplex performs a shrink and all but the best are changed. The blue lines indicate the current simplex and the light red lines are the new one.

For every iteration the reflected point $\boldsymbol{x}_r$ is the one tried first

$$\boldsymbol{x}_r = \boldsymbol{c} + \alpha(\boldsymbol{c} - \boldsymbol{x}_2) \tag{11.10}$$

If this point is better than the second best but not better than the current best such that

$$f(\boldsymbol{x}_0) \leq f(\boldsymbol{x}_r) < f(\boldsymbol{x}_1)$$

we replace $\boldsymbol{x}_2$ with $\boldsymbol{x}_r$, otherwise we do either expansion, contraction or shrink.

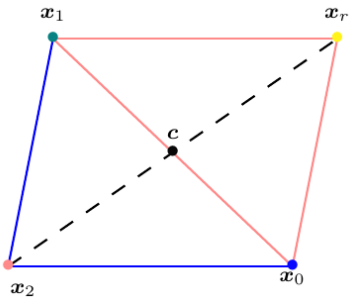

Figure 11.13: *$\boldsymbol{x}_2$ is mirrored in $\boldsymbol{c}$ and creates a reflected point $\boldsymbol{x}_r$.*

[H*] **Expansion**

If $\boldsymbol{x}_r$ is better than the current best,

$$f(\boldsymbol{x}_r) < f(\boldsymbol{x}_0)$$

we continue to explore and compute the expansion point $\boldsymbol{x}_e$

$$\boldsymbol{x}_e = \boldsymbol{c} + \gamma(\boldsymbol{x}_r - \boldsymbol{c}) \tag{11.11}$$

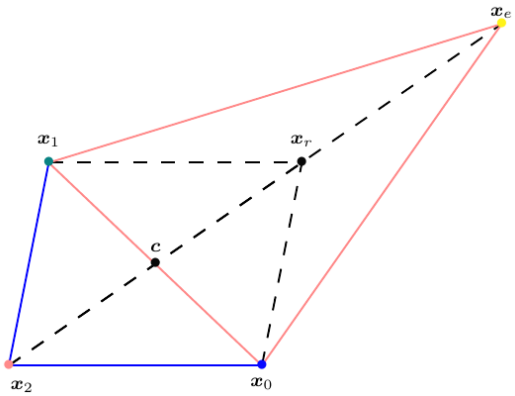

Figure 11.14: *The expansion point $\boldsymbol{x}_e$ is computed by moving further away from $\boldsymbol{c}$.*

Then one of two approaches can be used:

- Greedy minimization
  in which the reflected and expanded points are compared and the better of the two is picked such that if
  $$f(\boldsymbol{x}_e) < f(\boldsymbol{x}_r) < f(\boldsymbol{x}_0)$$
  we accept $\boldsymbol{x}_e$ ($\boldsymbol{x}_2 = \boldsymbol{x}_e$) and terminate the iteration.

  Otherwise, if
  $$f(\boldsymbol{x}_r) \leq f(\boldsymbol{x}_e)$$
  then $\boldsymbol{x}_r$ is accepted ($\boldsymbol{x}_2 = \boldsymbol{x}_r$).
- Greedy expansion
  we will accept $\boldsymbol{x}_e$ if
  $$f(\boldsymbol{x}_e) < f(\boldsymbol{x}_1)$$
  and
  $$f(\boldsymbol{x}_r) < f(\boldsymbol{x}_0)$$
  and skip comparing $f(\boldsymbol{x}_e)$ to $f(\boldsymbol{x}_r)$. This will keep the simplex as large as possible which can have some advantages for non-smooth functions.

[H*] **Contraction**

If $\boldsymbol{x}_r$ is worse than the current second worst point, in 2D case it is $\boldsymbol{x}_1$, which means

$$f(\boldsymbol{x}_r) \geq f(\boldsymbol{x}_1)$$

we compute a contraction point using $\boldsymbol{x}_2$ and $\boldsymbol{x}_r$.

It will be

- Outside
  if
  $$f(\boldsymbol{x}_1) \leq f(\boldsymbol{x}_r) < f(\boldsymbol{x}_2)$$
  We compute the contraction point as
  $$\boldsymbol{x}_c^{out} = \boldsymbol{c} + \beta(\boldsymbol{x}_r - \boldsymbol{c}) \tag{11.12}$$
  and if
  $$f(\boldsymbol{x}_c^{out}) \leq f(\boldsymbol{x}_r)$$
  then $\boldsymbol{x}_c^{out}$ is accepted ($\boldsymbol{x}_2 = \boldsymbol{x}_c^{out}$). Otherwise, a shrink of the simplex is performed.

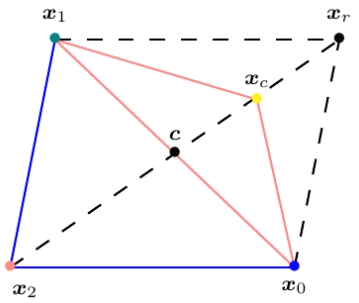

Figure 11.15: *The contraction point $\boldsymbol{x}_c^{out}$ is computed outside the old simplex.*

- Inside
  if
  $$f(\boldsymbol{x}_r) \geq f(\boldsymbol{x}_2)$$
  We compute the contraction point as
  $$\boldsymbol{x}_c^{in} = \boldsymbol{c} + \beta(\boldsymbol{x}_2 - \boldsymbol{c}) \tag{11.13}$$
  and if
  $$f(\boldsymbol{x}_c^{in}) \leq f(\boldsymbol{x}_2)$$
  then $\boldsymbol{x}_c^{in}$ is accepted ($\boldsymbol{x}_2 = \boldsymbol{x}_c^{in}$). Otherwise, a shrink of the simplex is performed.

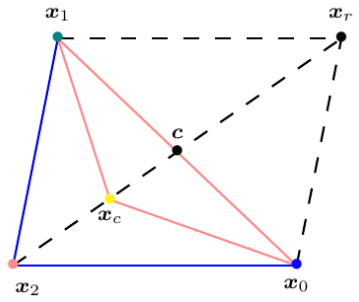

Figure 11.16: *The contraction point $\boldsymbol{x}_c^{in}$ is computed inside the old simplex.*

[H*] **Shrink**
the last resort that will be performed if none of the criteria so far have been met is to shrink the entire simplex by moving all points closer to the current best points.

$$\boldsymbol{s}_{i-1} = \boldsymbol{x}_0 + \delta(\boldsymbol{x}_i - \boldsymbol{x}_0), \quad i = \{1, 2, \ldots, N\} \tag{11.14}$$

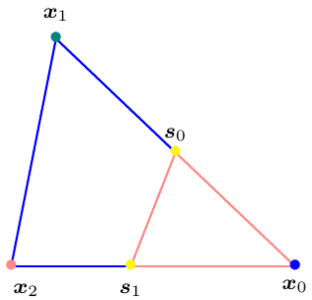

Figure 11.17: *Shrink the simplex by moving every point closer to the current best.*

[H*] **Initialization and termination**
When initializing the simplex, usually one point $\boldsymbol{x}_0$ is to be given by the user (as the initial value / input), then the other points are created by placing them at certain distance μ from the initial point along the coordinate axes in the parameter space $\mathbb{R}^N$ of the objective function.

$$\boldsymbol{x}_i = \boldsymbol{x}_0 + \mu \hat{\boldsymbol{e}}_i, \quad i = \{1, 2, \ldots, N\} \tag{11.15}$$

The algorithm terminates in one of three different ways. Either by having the simplex become sufficiently small, such that the active search space is thought to not have any considerable variation in function value. Another criteria is for all the function values of the points of the simplex to have a similar enough value. And finally the algorithm must be set to run for a maximum number of iterations or function evaluations.

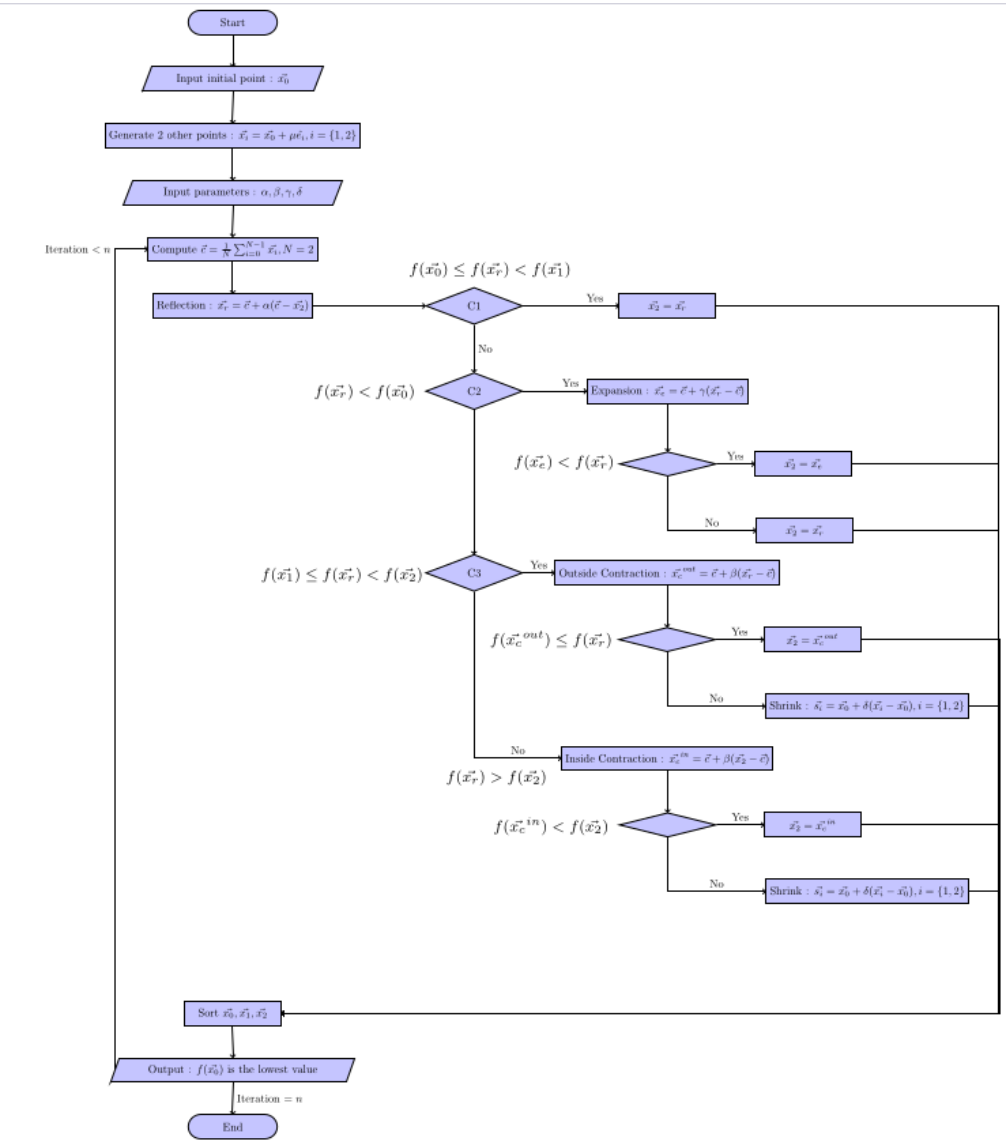

Figure 11.18: *The flowchart for downhill simplex algorithm that is used to make the C++ code for computing the solution.*

[H*] The downhill simplex method has some pros and cons. Its major strength is that it does not require the gradient / derivative computation of f. the gradient of the objective function is often not known as an analytical expression and can therefore be very expensive to evaluate. If so it has to be done using some form of finite difference scheme. This computational cost increases with the dimensionality of the problem and is also subject to potential numerical stability prolems if some care is not taken. Since downhill simplex does not require to compute the gradient of f, this makes this method applicable to non-smooth functions where the gradient may not exist on the whole domain.

This method has a memory footprint on the order of about $(N+2) \times N$ since we always

have to store $N+2$ sets of N parameters. Since we don't have to compute any gradient, then this method converges slowly compared to gradient based methods, generally because it requires more function evaluations.

[H*] **Example:**
Find the value x and y that minimizes

$$f(\boldsymbol{x}) = f(x,y) = e^{-\frac{x^2+y^2}{2s^2}} - e^{-\frac{x^2+y^2}{2t^2}} + \frac{1}{10}x^2 + \frac{1}{10}y^2, \quad s = 0.75,\ t = 1 \tag{11.16}$$

Solution:
First, we choose the initial point:

$$\boldsymbol{x}_0 = \begin{bmatrix} 0.1 \\ 0.5 \end{bmatrix}$$

The other two points can be generated from $\boldsymbol{x}_0$ by placing a certain distance $\mu = 0.05$ that we choose, thus

$$\boldsymbol{x}_1 = \boldsymbol{x}_0 + \mu\hat{e}_1 = \begin{bmatrix} 0.1 \\ 0.5 \end{bmatrix} + 0.05 \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0.15 \\ 0.5 \end{bmatrix}$$

$$\boldsymbol{x}_2 = \boldsymbol{x}_0 + \mu\hat{e}_2 = \begin{bmatrix} 0.1 \\ 0.5 \end{bmatrix} + 0.05 \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.1 \\ 0.55 \end{bmatrix}$$

We will have

$$\begin{aligned} f(\boldsymbol{x}_0) &= -0.058442 \\ f(\boldsymbol{x}_1) &= -0.0604927 \\ f(\boldsymbol{x}_2) &= -0.066630 \end{aligned}$$

(It is unsorted, but we try to compute with unsorted vector first) From here we can compute $\boldsymbol{c}$

$$\boldsymbol{c} = \frac{1}{2}\sum_{i=0}^{1} \boldsymbol{x}_i = \begin{bmatrix} 0.125 \\ 0.5 \end{bmatrix}$$

Now, we need to input the parameters as

$$\alpha = 1, \quad \beta = 0.5, \quad \gamma = 2, \quad \delta = 0.5$$

Then we will perform the reflection operation first,

$$\boldsymbol{x}_r = \boldsymbol{c} + \alpha(\boldsymbol{c} - \boldsymbol{x}_2) = \begin{bmatrix} 0.15 \\ 0.45 \end{bmatrix}$$

From here we will compare the value of $f(\boldsymbol{x}_0), f(\boldsymbol{x}_1), f(\boldsymbol{x}_2)$ and $f(\boldsymbol{x}_r)$.

If $f(\boldsymbol{x}_0) < f(\boldsymbol{x}_r) < f(\boldsymbol{x}_1)$ then

$$\boldsymbol{x}_2 = \boldsymbol{x}_r$$

and we go to iteration 2.

If $f(\boldsymbol{x}_r) < f(\boldsymbol{x}_0)$, then go to the expansion step.

If $f(\boldsymbol{x}_r) \geq f(\boldsymbol{x}_1)$, then go to the contraction step.

In this case we have

$$\begin{aligned} f(\boldsymbol{x}_0) &= -0.058442 \\ f(\boldsymbol{x}_1) &= -0.0604927 \\ f(\boldsymbol{x}_2) &= -0.0666302 \\ f(\boldsymbol{x}_r) &= -0.0523666 \end{aligned}$$

$f(\boldsymbol{x}_r) \geq f(\boldsymbol{x}_1)$ so we go and compute the contraction point, since $f(\boldsymbol{x}_r) \geq f(\boldsymbol{x}_2)$ we compute the inside contraction point

$$\boldsymbol{x}_c^{in} = \boldsymbol{c} + \beta(\boldsymbol{x}_0 - \boldsymbol{c}) = \begin{bmatrix} 0.1125 \\ 0.5 \end{bmatrix}$$

then we will have

$$\begin{aligned} f(\boldsymbol{x}_0) &= -0.058442 \\ f(\boldsymbol{x}_1) &= -0.0604927 \\ f(\boldsymbol{x}_2) &= -0.0666302 \\ f(\boldsymbol{x}_c) &= -0.0588848 \end{aligned}$$

since $f(\boldsymbol{x}_c) > f(\boldsymbol{x}_2)$ then we perform shrink that will replace $\boldsymbol{x}_1$ with $\boldsymbol{s}_0$ and $\boldsymbol{x}_2$ with $\boldsymbol{s}_1$

$$\boldsymbol{s}_0 = \boldsymbol{x}_0 + \delta(\boldsymbol{x}_1 - \boldsymbol{x}_0) = \begin{bmatrix} 0.1250 \\ 0.5250 \end{bmatrix}$$

$$\boldsymbol{s}_1 = \boldsymbol{x}_0 + \delta(\boldsymbol{x}_2 - \boldsymbol{x}_0) = \begin{bmatrix} 0.1375 \\ 0.5625 \end{bmatrix}$$

So we have reached the end of the first iteration, which is using the shrink and we get these values at the end of this iteration:

$$\begin{aligned} f(\boldsymbol{x}_0) &= -0.058442 \\ f(\boldsymbol{x}_1) &= -0.0604927 \\ f(\boldsymbol{x}_2) &= -0.0666302 \end{aligned}$$

Continue till the last number of iteration. We use 6 iterations and we use C++ to help us in computing, thus we obtain the best vector $\boldsymbol{x}_0$ that minimizes the function $f(x, y)$ is

$$\boldsymbol{x}_0 = \begin{bmatrix} 0.1916 \\ 0.5 \end{bmatrix}$$

with $f(\boldsymbol{x}_0) = -0.063016$.

Since it is only using 6 iterations, thus the real optimal solution can be different if we add more iterations, we also sort the vectors $\boldsymbol{x}_0, \boldsymbol{x}_1, \boldsymbol{x}_2$ at the end of iteration 1, if it is moved before iteration 1 then we will get faster convergence. We try the code with sorted vector before iteration 1 and obtain $f(\boldsymbol{x}_0) = -0.086404$ at the end of iteration 6.

We create the computation with **SymIntegration** and **Armadillo** and another one is with **SymIntegration** and **Eigen** for comparison, both produce the same answer but the one with **Eigen** has faster compiling time, we use 6 iterations with unsorted initial vectors then we sort the initial vectors and use 37 iterations, and then for the plotting in 3D we use Hamzstlab Mathematics with backend **ImGui** and **ImPlot3D** so we can change the value of s and t and the surface plot will change interactively as well.

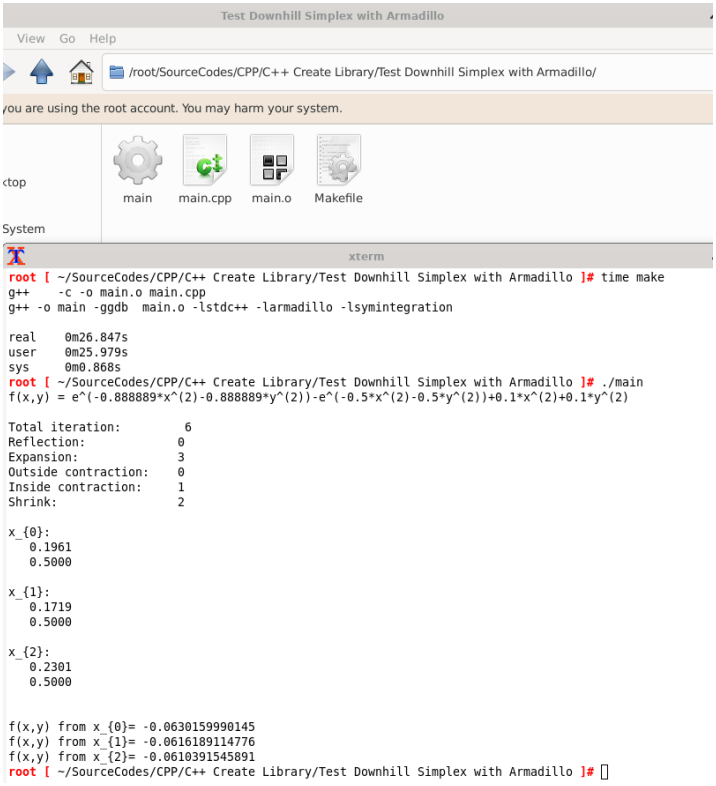

Figure 11.19: *The computation of downhill simplex for* $f(x,y) = e^{-\frac{x^2+y^2}{2s^2}} - e^{-\frac{x^2+y^2}{2t^2}} + \frac{1}{10}x^2 + \frac{1}{10}y^2$, $s = 0.75$, $t = 1$ *with SymIntegration and Armadillo (**SymIntegration/Examples/Test Downhill Simplex with Armadillo/main.cpp**).*

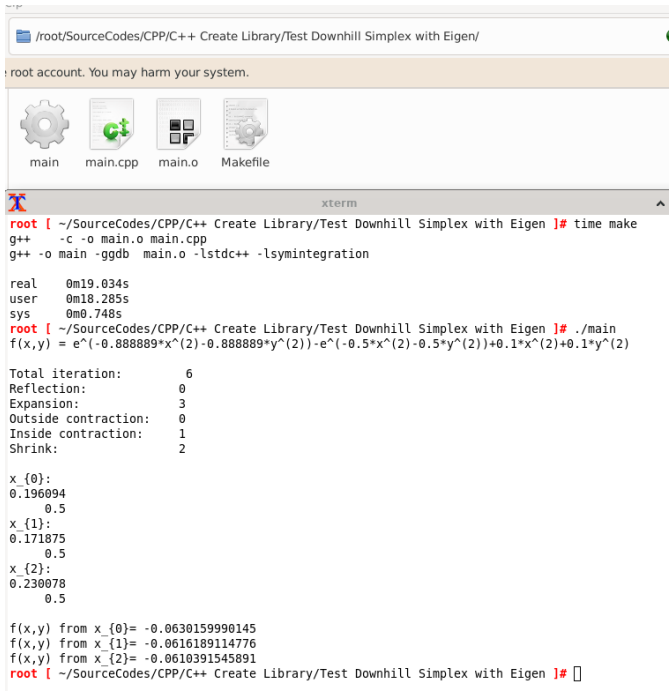

Figure 11.20: *The computation of downhill simplex for* $f(x,y) = e^{-\frac{x^2+y^2}{2s^2}} - e^{-\frac{x^2+y^2}{2t^2}} + \frac{1}{10}x^2 + \frac{1}{10}y^2$, $s = 0.75$, $t = 1$ *with SymIntegration and Eigen (**SymIntegration/Examples/Test Downhill Simplex with Eigen/-main.cpp**).*

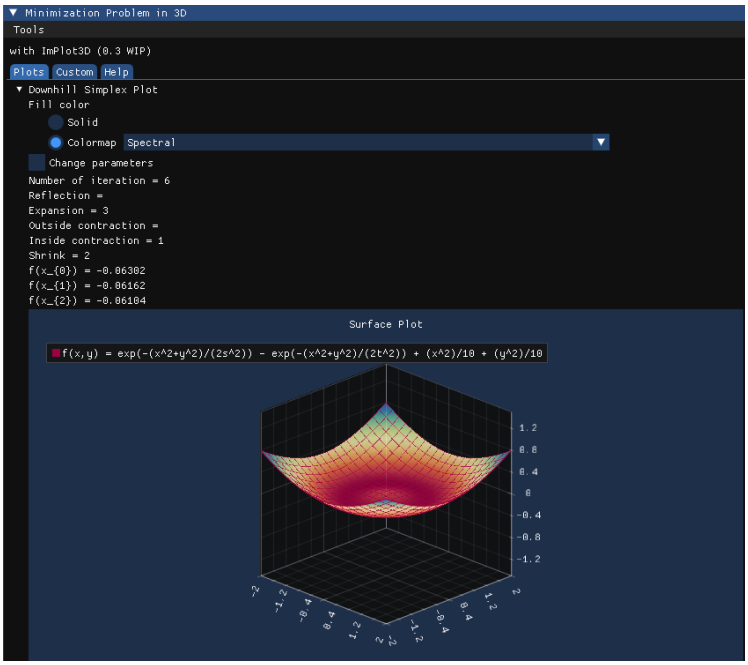

Figure 11.21: *The interactive surface plot for* $f(x,y) = e^{-\frac{x^2+y^2}{2s^2}} - e^{-\frac{x^2+y^2}{2t^2}} + \frac{1}{10}x^2 + \frac{1}{10}y^2$, $s = 0.75$, $t = 1$ *with Hamzstlab Mathematics and SymIntegration combined so you can see the optimum / minimum value there. (**Hamzstlab-Mathematics/examples/3D Minimization/main.cpp**).*

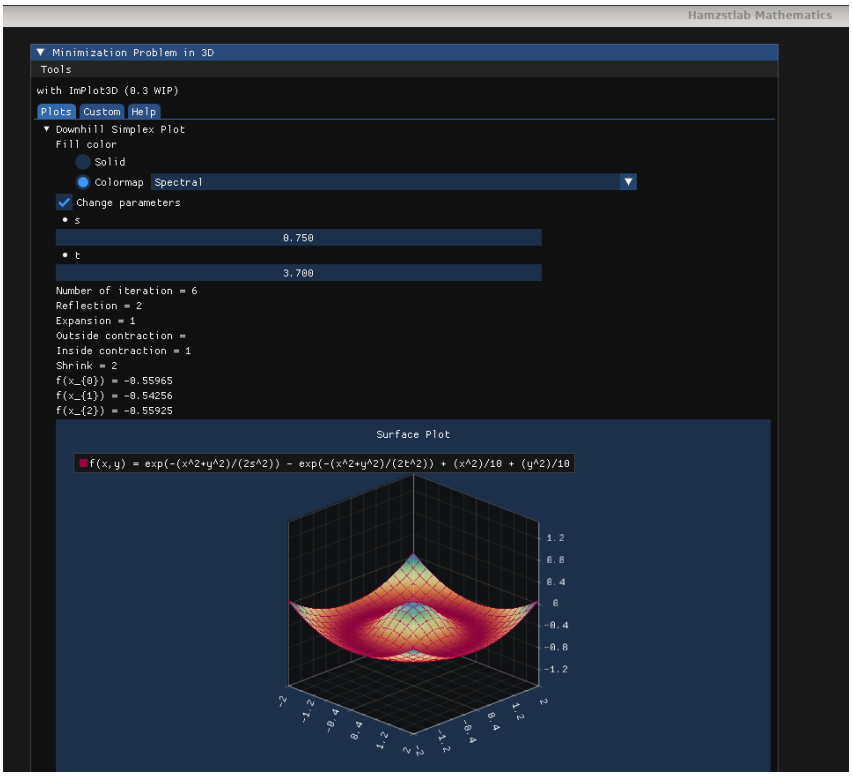

Figure 11.22: *The interactive surface plot for* $f(x,y) = e^{-\frac{x^2+y^2}{2s^2}} - e^{-\frac{x^2+y^2}{2t^2}} + \frac{1}{10}x^2 + \frac{1}{10}y^2$, $s = 0.75$ *when you change the value of* t *the computation and the plot change in an instant adapting to the parameter that has been changed. (**Hamzstlab-Mathematics/examples/3D Minimization/main.cpp**).*

```
g++ -o main -ggdb  main.o -lstdc++ -larmadillo -lsymintegration

real    0m29.108s
user    0m28.112s
sys     0m0.893s
root [ ~/SourceCodes/CPP/C++ Create Library/Test Downhill Simplex with Armadillo ]# ./main
Vector x_{0}:
   0.1000
   0.5000

Vector x_{1}:
   0.1500
   0.5000

Vector x_{2}:
   0.1000
   0.5500

After sorting,
x_{0}:
   0.1000
   0.5500

x_{1}:
   0.1500
   0.5000

x_{2}:
   0.1000
   0.5000

f(x,y) = e^(-0.888889*x^(2)-0.888889*y^(2))-e^(-0.5*x^(2)-0.5*y^(2))+0.1*x^(2)+0.1*y^(2)

Total iteration:         37
Reflection:              5
Expansion:               14
Outside contraction:     0
Inside contraction:      18
Shrink:                  0

x_{0}:
   0.0973
   0.8914

x_{1}:
   0.0852
   0.8891

x_{2}:
   0.0629
   0.8977


f(x,y) from x_{0}= -0.0992255745363
f(x,y) from x_{1}= -0.0992180473087
f(x,y) from x_{2}= -0.0992086382243
root [ ~/SourceCodes/CPP/C++ Create Library/Test Downhill Simplex with Armadillo ]#
```

Figure 11.23: *The computation of downhill simplex for* $f(x,y) = e^{-\frac{x^2+y^2}{2s^2}} - e^{-\frac{x^2+y^2}{2t^2}} + \frac{1}{10}x^2 + \frac{1}{10}y^2$, $s = 0.75$, $t = 1$ *with 37 iterations (**SymIntegration/Examples/Test Downhill Simplex with Armadillo/main.cpp**).*

```
troot [ ~/SourceCodes/CPP/C++ Create Library/Test Downhill Simplex with Eigen ]#
time make
g++    -c -o main.o main.cpp
g++ -o main -ggdb  main.o -lstdc++ -lsymintegration

real    0m19.921s
user    0m19.108s
sys     0m0.760s
root [ ~/SourceCodes/CPP/C++ Create Library/Test Downhill Simplex with Eigen ]# ./main
Vector x_{0}:
0.1
0.5
Vector x_{1}:
0.15
 0.5
Vector x_{2}:
 0.1
0.55
After sorting,
x_{0}:
 0.1
0.55
x_{1}:
0.15
 0.5
x_{2}:
0.1
0.5
f(x,y) = e^(-0.888889*x^(2)-0.888889*y^(2))-e^(-0.5*x^(2)-0.5*y^(2))+0.1*x^(2)+0.1*y^(2)

Total iteration:         37
Reflection:              5
Expansion:               14
Outside contraction:     0
Inside contraction:      18
Shrink:                  0

x_{0}:
0.0972656
 0.891406
x_{1}:
0.0851562
 0.889063
x_{2}:
0.0628906
 0.897656

f(x,y) from x_{0}= -0.0992255745363
f(x,y) from x_{1}= -0.0992180473087
f(x,y) from x_{2}= -0.0992086382243
root [ ~/SourceCodes/CPP/C++ Create Library/Test Downhill Simplex with Eigen ]#
```

Figure 11.24: *The computation of downhill simplex for* $f(x,y) = e^{-\frac{x^2+y^2}{2s^2}} - e^{-\frac{x^2+y^2}{2t^2}} + \frac{1}{10}x^2 + \frac{1}{10}y^2$, $s = 0.75$, $t = 1$ *with 37 iterations (**SymIntegration/Examples/Test Downhill Simplex with Eigen/main.cpp**).*

* the C++ codes are already updated so the initial vectors will be sorted before the **for** looping.

IV. Numerical Differentiation

V. Numerical Integration

VI. Boundary-Value Problems for Ordinary Differential Equations

VII. Numerical Solutions to Partial Differential Equations

Chapter 12

Complex Numbers

"Tobikata Wo Wasureta Tori No You Ni (A Little bird who forgot how to fly)." - ***Star Ocean 3 OST - MISIA***

I. Preliminaries

[H*] Complex analysis, traditionally known as the theory of functions of a complex variable, is the branch of mathematical analysis that investigates functions of complex numbers. It is helpful in many branches of mathematics, including algebraic geometry, number theory, analytic combinatorics, applied mathematics; as well as in physics, including the branches of hydrodynamics, thermodynamics, and particularly quantum mechanics. By extension, use of complex analysis also has applications in engineering fields such as nuclear, aerospace, mechanical and electrical engineering.

As a differentiable function of a complex variable is equal to its Taylor series (that is, it is analytic), complex analysis is particularly concerned with analytic functions of a complex variable (that is, holomorphic functions).

[H*] Complex number arises when no one on this planet up until 2024 A.D. able to compute this:

$$\sqrt{-1}$$

That number is called imaginary number, the notation is $i = \sqrt{-1}$.

[H*] Let $x, y \in \mathbb{R}$, the complex number $z \in \mathbb{C}$ can be defined as

$$z = x + iy \tag{12.1}$$

Some important properties of complex numbers:

- $\mathrm{Re}(z) = x$ and $\mathrm{Im}(z) = y$ are called the real part of z and the imaginary part of z, respectively.
- $|z| = \sqrt{x^2 + y^2}$ is called the modulus (or absolute value) of z.
- $\bar{z} = x - yi$ is called the complex conjugate of z.
- $z\bar{z} = x^2 + y^2 = |z|^2$
- The angle θ is called an argument of z.
- $\mathrm{Re}(z) = |z| \cos\ \theta$
- $\mathrm{Im}(z) = |z| \sin\ \theta$

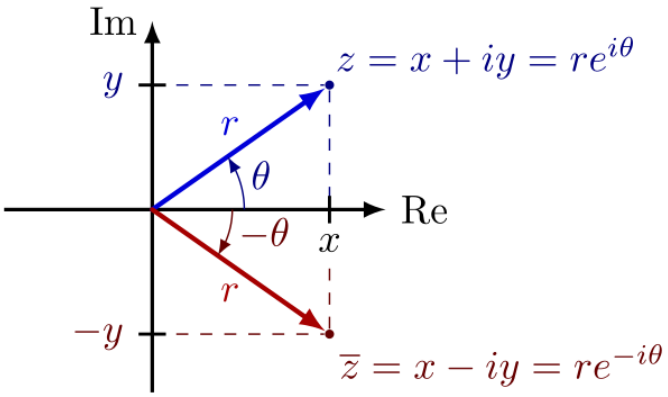

Figure 12.1: *The complex number can be represented as $z = x + iy = re^{i\theta}$, with $r = |z|$ and the angle θ is the argument of z.*

- $z = |z|(\cos\ \theta + i\ \sin\ \theta)$ is called the polar form of z.

[H*] We try to apply Newton-Raphson method to compute the root of $x^2 + 1 = 0$ and it cannot even compute i.

- General Theory of nth Order Linear Equations

[H*] The first
[H*]

- The Method of Variation of Parameters

[H*] The first
[H*]

i. C++ Plot: 2D Plot of Complex Function

Suppose we have a complex function

$$e^{ix}$$

and we want to plot it.

Note that

$$e^{ix} = \cos\ x + i\ \sin\ x$$

With **gnuplot** we can plot the function as separate functions, the first function is the real part, the second function is the imaginary part.

Thus, the real part of e^{ix} is $\cos\ x$ and the imaginary part of e^{ix} is $\sin\ x$.

```
#include <vector>
#include <cmath>
#include <utility>
#include <boost/tuple/tuple.hpp>
```

```
f(x) = 1+x^2

f'(x) = 2*x

p_{0} = 0.7853982
```

n	p_{n}	p_{n} in Degree
0	0.7853982	45.0
1	-0.24392074	-13.975629
2	1.9278858	110.45972
3	0.7045915	40.37012
4	-0.35733533	-20.473806
5	1.2205782	69.93398
6	0.20064712	11.496233
7	-2.3916135	-137.02936
8	-0.98674273	-56.536194
9	0.013346314	0.7646875
10	-37.456852	-2146.1196
11	-18.715078	-1072.295
12	-9.330823	-534.61676
13	-4.6118255	-264.23813
14	-2.1974957	-125.907234
15	-0.8712162	-49.91701
16	0.13830233	7.9241395
17	-3.5461168	-203.17754
18	-1.6320591	-93.5101
19	-0.509668	-29.201826
20	0.72619677	41.60801
21	-0.32542026	-18.645208
22	1.3737646	78.710915
23	0.3229189	18.50189
24	-1.3869171	-79.4645
25	-0.33294678	-19.076445
26	1.3352681	76.505226
27	0.2931775	16.797832
28	-1.5588628	-89.31626
29	-0.4586848	-26.280704
30	0.8607309	49.316246
31	-0.15053618	-8.625088
32	3.2461925	185.99313
33	1.4690697	84.17149
34	0.3941834	22.585045
35	-1.0713533	-61.384026
36	-0.06897712	-3.9520977
37	7.2142925	413.3485
38	3.5378394	202.70326
39	1.6275904	93.25407
40	0.50659263	29.02562
41	-0.73369	-42.037342

Figure 12.2: *The Newton-Raphson method to compute the root of $x^2 + 1 = 0$ cannot converge.*

```
#include "gnuplot-iostream.h"

int main() {
       Gnuplot gp;

       // Don't forget to put "\n" at the end of each line!
       gp << "set xrange [-10:5]\nset yrange [-3:3]\n";
       // '-' means read from stdin. The send1d() function sends data to
           gnuplot's stdin.
       gp << "i = {0,1}\n";
       gp << "plot real(exp(i*x)), imag(exp(i*x))\n";

       // To do parametric plot uncomment below and comment the plot line
           above
       //gp << "set parametric\n";
       //gp << "plot real(exp(i*t)), imag(exp(i*t))\n";

       return 0;
}
```

Code 25: *main.cpp "2D plot of complex function"*

To compile it, type:
g++ -o main main.cpp -lboost_iostreams
./main

or with Makefile, type:
make
./main
s

Explanation for the codes:

- In **gnuplot** complex constants are expressed as $\{<\text{real}>,<\text{imag}>\}$, where $<\text{real}>$ and $<\text{imag}>$ must be numerical constants. For instance:

$$a+bi \to \{a,b\}$$
$$i \to \{0,1\}$$
$$5-2i \to \{5,-2\}$$

  Gnuplot can plot only real quantities. Thus we can plot the real part of e^{ix} which is cos x and the imaginary part of e^{ix} which is sin x, but first we need to define the variable i as $\{0,1\}$.

```
gp << "i = {0,1}\n";
gp << "plot real(exp(i*x)), imag(exp(i*x))\n";
```

- If you want to do parametric plot with

$$(\text{real}(e^{ix}), \text{imag}(e^{ix}))$$

  you can uncomment the two lines at the bottom and comment the plot line above it

```
            //gp << "plot real(exp(i*x)), imag(exp(i*x))\n";

            // To do parametric plot uncomment below and comment the plot
                line above
            gp << "set parametric\n";
            gp << "plot real(exp(i*t)), imag(exp(i*t))\n";
```

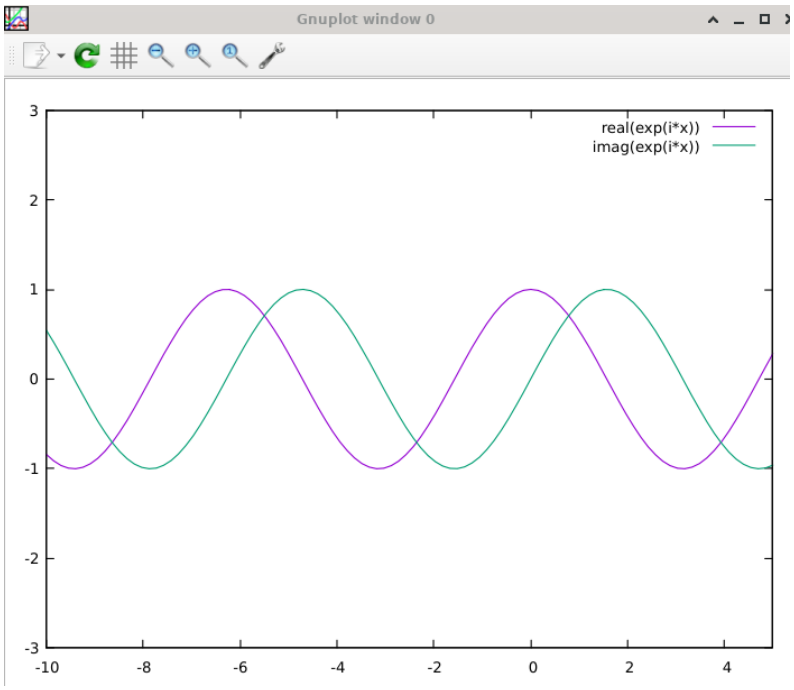

Figure 12.3: *The 2D plot of the real part of e^{ix} and the imaginary part of e^{xi} (**DFSimulatorC/Source Codes/C++/C+++ Gnuplot SymbolicC++/ch23-Numerical Linear Algebra/2D Plot Complex Function/main.cpp**).*

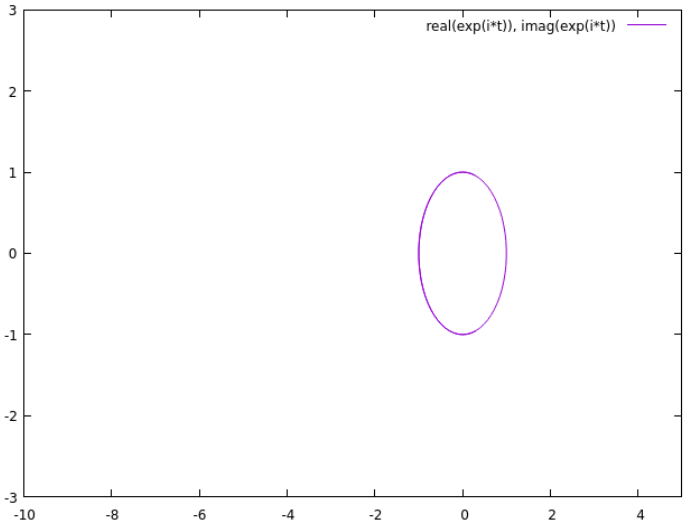

Figure 12.4: *The 2D parametric plot of* $(real(e^{ix}), imag(e^{ix}))$ *(**DFSimulatorC/Source Codes/C++/C+++ Gnuplot SymbolicC++/ch23-Numerical Linear Algebra/2D Plot Complex Function/main.cpp**).*

Chapter 13

Dynamical System

*"Chaos would suit me just fine." - **Yuber (Suikoden III)***

I. Double Pendulum

[H*] In physics and mathematics, a double pendulum, also known as a chaotic pendulum, is a pendulum with another pendulum attached to its end, forming a complex physical system that exhibits rich dynamics behavior with a strong sensitivity to initial conditions. The motion of a double pendulum is governed by a pair of coupled ordinary differential equations and is chaotic.

[H*] There are several variants of the double pendulum; the two limbs may be of equal or unequal length and masses, they may be simple pendulums or compound pendulums (complex pendulums) and the motion may be in three dimensions or restricted to one vertical plane.

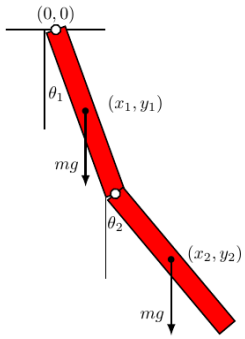

Figure 13.1: *The illustration of double compound pendulum .*

In the following analysis the limbs are taken to be identical compound pendulums of length l and mass m, and the motion is restricted to two dimensions.

In a compound pendulum, the mass is distributed along its length. If the double pendulum mass is evenly distributed, then the center of mass of each limb is at its midpoint, and the limb has a moment of inertia of

$$I = \frac{1}{12} m l^2$$

about that point.

Although it is possible to derive the equations of a double pendulum with Newtonian mechanics, it is considered to be cumbersome to work with as it would require resolving vectors with respect to constraint forces. So it is more convenient to use the angles between each limb and the vertical as the generalized coordinates defining the configuration of the system. These angles are denoted θ_1 and θ_2. The position of the center of mass of each rod may be written in terms of these two coordinates. If the origin of the Cartesian coordinate system is taken to be at the point of suspension of the first pendulum, then the center of mass of this pendulum is at:

$$x_1 = \frac{1}{2} l \sin\theta_1$$
$$y_1 = -\frac{1}{2} l \cos\theta_1$$

and the center of mass of the second pendulum is at

$$x_2 = l\left(\sin\theta_1 + \frac{1}{2}\sin\theta_2\right)$$
$$y_2 = -l\left(\cos\theta_1 + \frac{1}{2}\cos\theta_2\right)$$

This is enough information to write out the Lagrangian.

[H*] **Lagrangian for Double Compound Pendulum**

The Lagrangian is given by

$$\begin{aligned} L &= \text{kinetic energy} - \text{potential energy} \\ &= \frac{1}{2} m(v_1^2 + v_2^2) + \frac{1}{2} I(\dot{\theta}_1^2 + \dot{\theta}_2^2) - mg(y_1 + y_2) \\ &= \frac{1}{2} m(\dot{x}_1^2 + \dot{y}_1^2 + \dot{x}_2^2 + \dot{y}_2^2) + \frac{1}{2} I(\dot{\theta}_1^2 + \dot{\theta}_2^2) - mg(y_1 + y_2) \end{aligned} \tag{13.1}$$

The first term is the linear kinetic energy of the center of mass of the bodies and the second term is the rotational kinetic energy around the center of mass of each rod. The last term is the potential energy of the bodies in a uniform gravitational field. The dot-notation indicates the time derivative of the variable in question.

Using the values of x_1 and y_1 defined above, we have

$$\dot{x}_1 = \dot{\theta}_1\left(\frac{1}{2} l \cos\theta_1\right)$$
$$\dot{y}_1 = \dot{\theta}_1\left(\frac{1}{2} l \sin\theta_1\right)$$

which leads to

$$v_1^2 = \dot{x}_1^2 + \dot{y}_1^2 = \frac{1}{4}\dot{\theta}_1^2 l^2\left(\cos^2\theta_1 + \sin^2\theta_1\right) = \frac{1}{4} l^2 \dot{\theta}_1^2$$

Similarly, for x_2 and y_2 we have

$$\dot{x}_2 = l\left(\dot{\theta}_1\cos\theta_1 + \frac{1}{2}\dot{\theta}_2\cos\theta_2\right)$$
$$\dot{y}_2 = l\left(\dot{\theta}_1\sin\theta_1 + \frac{1}{2}\dot{\theta}_2\sin\theta_2\right)$$

and therefore

$$\begin{aligned} v_2^2 &= \dot{x}_2^2 + \dot{y}_2^2 \\ &= l^2 \left(\dot{\theta}_1^2 \cos^2\theta_1 + \dot{\theta}_1^2 \sin^2\theta_1 + \frac{1}{4}\dot{\theta}_2^2 \cos^2\theta_2 + \frac{1}{4}\dot{\theta}_2^2 \sin^2\theta_2 + \dot{\theta}_1\dot{\theta}_2 \cos\theta_1 \cos\theta_2 + \dot{\theta}_1\dot{\theta}_2 \sin\theta_1 \sin\theta_2 \right) \\ &= l^2 \left(\dot{\theta}_1^2 + \frac{1}{4}\dot{\theta}_2^2 + \dot{\theta}_1\dot{\theta}_2 \cos(\theta_1 - \theta_2) \right) \end{aligned}$$

Substituting the coordinates above into the definition of the Lagrangian, and rearranging the equation, gives

$$\begin{aligned} L &= \frac{1}{2}m(v_1^2 + v_2^2) + \frac{1}{2}I(\dot{\theta}_1^2 + \dot{\theta}_2^2) - mg(y_1 + y_2) \\ &= \frac{1}{2}ml^2 \left(\frac{1}{4}\dot{\theta}_1^2 + \dot{\theta}_1^2 + \frac{1}{4}\dot{\theta}_2^2 + \dot{\theta}_1\dot{\theta}_2 \cos(\theta_1 - \theta_2) \right) + \frac{1}{24}ml^2 \left(\dot{\theta}_1^2 + \dot{\theta}_2^2 \right) \\ &- mgl \left(-\frac{1}{2}\cos\theta_1 - \cos\theta_1 - \frac{1}{2}\cos\theta_2 \right) \\ &= \frac{1}{6}ml^2 \left(\dot{\theta}_2^2 + 4\dot{\theta}_1^2 + 3\dot{\theta}_1\dot{\theta}_2 \cos(\theta_1 - \theta_2) \right) + \frac{1}{2}mgl(3\cos\theta_1 + \cos\theta_2) \end{aligned}$$

The equations of motion can now be derived using the Euler-Lagrange equations, which are given by

$$\frac{d}{dt}\frac{\partial L}{\partial \dot{\theta}_i} - \frac{\partial L}{\partial \theta_i} = 0, \quad i = 1, 2. \tag{13.2}$$

Equations of motion describe how an object's position, velocity, and acceleration change over time. We begin with the equation of motion for θ_1. The derivatives of the Lagrangian are given by

$$\frac{\partial L}{\partial \theta_1} = -\frac{1}{2}ml^2\dot{\theta}_1\dot{\theta}_2 \sin(\theta_1 - \theta_2) - \frac{3}{2}mgl \sin\theta_1$$

and

$$\frac{\partial L}{\partial \dot{\theta}_1} = \frac{4}{3}ml^2\dot{\theta}_1 + \frac{1}{2}ml^2\dot{\theta}_2 \cos(\theta_1 - \theta_2)$$

Thus

$$\frac{d}{dt}\frac{\partial L}{\partial \dot{\theta}_i} = \frac{4}{3}ml^2\ddot{\theta}_1 + \frac{1}{2}ml^2\ddot{\theta}_2 \cos(\theta_1 - \theta_2) - \frac{1}{2}ml^2\dot{\theta}_2(\dot{\theta}_1 - \dot{\theta}_2) \sin(\theta_1 - \theta_2)$$

Combining these results and simplifying yields the first equation of motion,

$$\frac{4}{3}l\ddot{\theta}_1 + \frac{1}{2}l\ddot{\theta}_2 \cos(\theta_1 - \theta_2) + \frac{1}{2}l\dot{\theta}_2^2 \sin(\theta_1 - \theta_2) + \frac{3}{2}g \sin\theta_1 = 0 \tag{13.3}$$

Similarly, the derivatives of the Lagrangian with respect to θ_2 and $\dot{\theta}_2$ are given by

$$\frac{\partial L}{\partial \theta_2} = -\frac{1}{2}ml^2\dot{\theta}_1\dot{\theta}_2 \sin(\theta_1 - \theta_2) - \frac{1}{2}mgl \sin\theta_2$$

and

$$\frac{\partial L}{\partial \dot{\theta}_2} = \frac{1}{3}ml^2\dot{\theta}_2 + \frac{1}{2}ml^2\dot{\theta}_1 \cos(\theta_1 - \theta_2)$$

Thus

$$\frac{d}{dt}\frac{\partial L}{\partial \dot{\theta}_2} = \frac{1}{3}ml^2\ddot{\theta}_2 + \frac{1}{2}ml^2\ddot{\theta}_1 \cos(\theta_1 - \theta_2) - \frac{1}{2}ml^2\dot{\theta}_1(\dot{\theta}_1 - \dot{\theta}_2) \sin(\theta_1 - \theta_2)$$

Plugging these results into the Euler-Lagrange equation and simplifying yields the second equation of motion,

$$\frac{1}{3}l\ddot{\theta}_2 + \frac{1}{2}l\ddot{\theta}_1\cos(\theta_1 - \theta_2) - \frac{1}{2}l\dot{\theta}_1^2\sin(\theta_1 - \theta_2) + \frac{1}{2}g\sin\theta_2 = 0 \tag{13.4}$$

No closed form solutions for θ_1 and θ_2 as functions of time are known, therefore the system can only be solved numerically, using the Runge Kutta method or similar techniques.

[H*] **Lagrangian for Double Simple Pendulum**

The knowledge of this writing is taken from Wolfram Mathematica webpage written by Eric W. Weisstein.

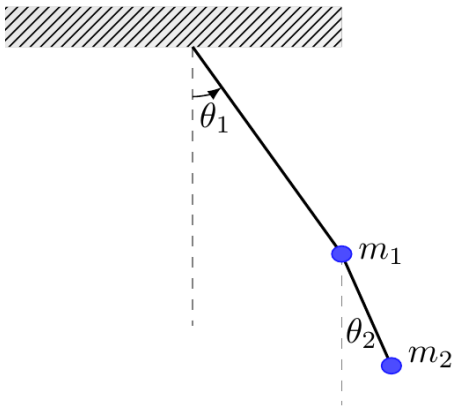

Figure 13.2: *The illustration of double simple pendulum .*

Consider a double bob pendulum with masses m_1 and m_2 attached by rigid massless wires of lengths l_1 and l_2. Further, let the angles of the two wires with the vertical be denoted by θ_1 and θ_2. Finally, let gravity be given by g. Then the positions of the bobs are given by

$$\begin{aligned} x_1 &= l_1\sin\theta_1 \\ y_1 &= -l_1\cos\theta_1 \\ x_2 &= l_1\sin\theta_1 + l_2\sin\theta_2 \\ y_2 &= -l_1\cos\theta_1 - l_2\cos\theta_2 \end{aligned}$$

the potential energy of the system is given by

$$\begin{aligned} V &= m_1gy_1 + m_2gy_2 \\ &= -(m_1 + m_2)gl_1\cos\theta_1 - m_2gl_2\cos\theta_2 \end{aligned}$$

the potential energy is negative because it is going downward, and the first potential energy for m_1 is the sum of $m_1 + m_2$ because the first bob is carrying the mass of the second bob pendulum.

The kinetic energy is given by

$$\begin{aligned} T &= \frac{1}{2}m_1v_1^2 + \frac{1}{2}m_2v_2^2 \\ &= \frac{1}{2}m_1l_1^2\dot{\theta}_1^2 + \frac{1}{2}m_2\left[l_1^2\dot{\theta}_1^2 + l_2^2\dot{\theta}_2^2 + 2l_1l_2\dot{\theta}_1\dot{\theta}_2\cos(\theta_1 - \theta_2)\right] \end{aligned}$$

The Lagrangian is then

$$\begin{aligned}L &= T - V \\ &= \frac{1}{2}(m_1+m_2)l_1^2\dot{\theta}_1^2 + \frac{1}{2}m_2l_2^2\dot{\theta}_2^2 + m_2l_1l_2\dot{\theta}_1\dot{\theta}_2\cos(\theta_1-\theta_2) - (-(m_1+m_2)gl_1\cos\theta_1 - m_2gl_2\cos\theta_2) \\ &= \frac{1}{2}(m_1+m_2)l_1^2\dot{\theta}_1^2 + \frac{1}{2}m_2l_2^2\dot{\theta}_2^2 + m_2l_1l_2\dot{\theta}_1\dot{\theta}_2\cos(\theta_1-\theta_2) + ((m_1+m_2)gl_1\cos\theta_1 + m_2gl_2\cos\theta_2)\end{aligned}$$

Therefore, the equations of motion can now be derived using the Euler-Lagrange equations, for $\dot{\theta}_1$

$$\begin{aligned}\frac{\partial L}{\partial \dot{\theta}_1} &= m_1l_1^2\dot{\theta}_1 + m_2l_1^2\dot{\theta}_1 + m_2l_1l_2\dot{\theta}_2\cos(\theta_1-\theta_2) \\ \frac{d}{dt}\left(\frac{\partial L}{\partial \dot{\theta}_1}\right) &= (m_1+m_2)l_1^2\ddot{\theta}_1 + m_2l_1l_2\ddot{\theta}_2\cos(\theta_1-\theta_2) - m_2l_1l_2\dot{\theta}_2\sin(\theta_1-\theta_2)(\dot{\theta}_1-\dot{\theta}_2)\end{aligned}$$

and for θ_1

$$\frac{\partial L}{\partial \theta_1} = -l_1g(m_1+m_2)\sin\theta_1 - m_2l_1l_2\dot{\theta}_1\dot{\theta}_2\sin(\theta_1-\theta_2)$$

so the Euler-Lagrange differential equation for θ_1 / the first equation of motion becomes

$$\begin{aligned}\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{\theta}_1}\right) - \frac{\partial L}{\partial \theta_1} &= 0 \\ (m_1+m_2)l_1^2\ddot{\theta}_1 + m_2l_1l_2\ddot{\theta}_2\cos(\theta_1-\theta_2) + m_2l_1l_2\dot{\theta}_2^2\sin(\theta_1-\theta_2) + l_1g(m_1+m_2)\sin\theta_1 &= 0\end{aligned} \tag{13.5}$$

Similarly, the equations of motion for $\dot{\theta}_2$

$$\begin{aligned}\frac{\partial L}{\partial \dot{\theta}_2} &= m_2l_2^2\dot{\theta}_2 + m_2l_1l_2\dot{\theta}_1\cos(\theta_1-\theta_2) \\ \frac{d}{dt}\left(\frac{\partial L}{\partial \dot{\theta}_2}\right) &= m_2l_2^2\ddot{\theta}_2 + m_2l_1l_2\ddot{\theta}_1\cos(\theta_1-\theta_2) - m_2l_1l_2\dot{\theta}_1\sin(\theta_1-\theta_2)(\dot{\theta}_1-\dot{\theta}_2)\end{aligned}$$

and for θ_2

$$\frac{\partial L}{\partial \theta_2} = m_2l_1l_2\dot{\theta}_1\dot{\theta}_2\sin(\theta_1-\theta_2) - l_2m_2g\sin\theta_2$$

so the Euler-Lagrange differential equation for θ_2 / the second equation of motion becomes

$$\begin{aligned}\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{\theta}_2}\right) - \frac{\partial L}{\partial \theta_2} &= 0 \\ m_2l_2^2\ddot{\theta}_2 + m_2l_1l_2\ddot{\theta}_1\cos(\theta_1-\theta_2) - m_2l_1l_2\dot{\theta}_1^2\sin(\theta_1-\theta_2) + l_2m_2g\sin\theta_2 &= 0\end{aligned} \tag{13.6}$$

[H*] **Chaotic Motion**
The double pendulum undergoes chaotic motion, and clearly shows a sensitive dependence on initial conditions.

i. Double Simple Pendulum Computation and Simulation

In SymIntegration there is a function to compute the lagrangian and the equations of motion, **lagrangian(**$x_1, y_1, x_2, y_2, \theta_1, \dot{\theta}_1, \ddot{\theta}_1, \theta_2, \dot{\theta}_2, \ddot{\theta}_2$**)**

We input them all as symbolic to obtain the equations of motion formula symbolically, and then if we want to plot or compute numerically at certain time t we can substitute the symbolic, e.g. θ_1 into numeric, including defining / substituting the mass for both bob pendulum (m_1, m_2), the length of the wire from the ceiling to the first bob pendulum (l_1), the length of the wire between the first and the second bob pendulum (l_2) and the gravity (g).

```
#include <iostream>
#include "symintegrationc++.h"
#include <bits/stdc++.h>
#include <cmath>

#include <chrono>

using namespace std::chrono;
using namespace std;
using namespace SymbolicConstant;

double division(double x, double y)
{
	return x/y;
}

int main(void)
{
	// Get starting timepoint
	auto start = high_resolution_clock::now();

	Symbolic l1("l1"), l2("l2"), t1("t1"), t2("t2"), dt1("t1'"), dt2("t2'"),
	    ddt1("t1''"),ddt2("t2''");
	Symbolic x1, y1, x2, y2;

	x1 = l1*sin(t1);
	y1 = -l1*cos(t1);
	x2 = l1*sin(t1) + l2*sin(t2);
	y2 = -l1*cos(t1) - l2*cos(t2);

	cout << "x1 = " << x1 << endl;
	cout << "y1 = " << y1 << endl;
	cout << "x2 = " << x2 << endl;
	cout << "y2 = " << y2 << endl;

	cout << "Here we go..." << lagrangian(x1,y1,x2,y2,t1, dt1, dt1, t2, dt2,
	    ddt2) << endl;
```

```
        // Get ending timepoint
        auto stop = high_resolution_clock::now();
        auto duration = duration_cast<microseconds>(stop - start);

        cout << "\nTime taken by function: " << duration.count() << " microseconds"
             << endl;

        return 0;
}
```

Code 26: *Examples/Test SymIntegration Double Pendulum Problem with Lagrangian Function/main.cpp*

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Double Pendulum Problem with Lagrangian Function ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Double Pendulum Problem with Lagrangian Function ]# ./main
x1 = l1*sin(θ1)
y1 = -l1*cos(θ1)
x2 = l1*sin(θ1)+l2*sin(θ2)
y2 = -l1*cos(θ1)-l2*cos(θ2)
Here we go...
Potential energy = -m1*g*l1*cos(θ1)-m2*g*l1*cos(θ1)-m2*g*l2*cos(θ2)
Kinetic energy = 0.5*m1*l1^(2)*θ1'^(2)+0.5*m2*l1^(2)*θ1'^(2)+m2*l1*cos(θ1)*θ1'*l2*cos(θ2)*θ2'+0.5*m2*l2^(2)*θ2'^(2)+m2*l1*sin(θ1)*θ1'*l2*sin(θ2)*θ2'
Lagrangian = 0.5*m1*l1^(2)*θ1'^(2)+0.5*m2*l1^(2)*θ1'^(2)+m2*l1*cos(θ1)*θ1'*l2*cos(θ2)*θ2'+0.5*m2*l2^(2)*θ2'^(2)+m2*l1*sin(θ1)*θ1'*l2*sin(θ2)*θ2'+m1*g*l1*cos(θ1)+m2
*g*l1*cos(θ1)+m2*g*l2*cos(θ2)

dL / dθ1 = -m2*l1*sin(θ1)*θ1'*l2*cos(θ2)*θ2'+m2*l1*cos(θ1)*θ1'*l2*sin(θ2)*θ2'-m1*g*l1*sin(θ1)-m2*g*l1*sin(θ1)

dL / dθ1' = m1*l1^(2)*θ1'+m2*l1^(2)*θ1'+m2*l1*cos(θ1)*l2*cos(θ2)*θ2'+m2*l1*sin(θ1)*l2*sin(θ2)*θ2'

d/dt (dL / dθ1') = -m2*l1*sin(θ1)*l2*cos(θ2)*θ2'*θ1'+m2*l1*cos(θ1)*l2*sin(θ2)*θ2''*θ1'-m2*l1*cos(θ1)*l2*sin(θ2)*θ2'^(2)+m2*l1*sin(θ1)*l2*cos(θ2)*θ2'^(2)+m1*l1^(2)*θ
1''+m2*l1^(2)*θ1''+m2*l1*cos(θ1)*l2*cos(θ2)*θ2''+m2*l1*sin(θ1)*l2*sin(θ2)*θ2''

dL / dθ2 = -m2*l1*cos(θ1)*θ1'*l2*sin(θ2)*θ2'+m2*l1*sin(θ1)*θ1'*l2*cos(θ2)*θ2'-m2*g*l2*sin(θ2)

dL / dθ2' = m2*l1*cos(θ1)*θ1'*l2*cos(θ2)+m2*l2^(2)*θ2'+m2*l1*sin(θ1)*θ1'*l2*sin(θ2)

d/dt (dL / dθ2') = -m2*l1*sin(θ1)*θ1'^(2)*l2*cos(θ2)+m2*l1*cos(θ1)*θ1'^(2)*l2*sin(θ2)-m2*l1*cos(θ1)*θ1'*l2*sin(θ2)*θ2'+m2*l1*sin(θ1)*θ1'*l2*cos(θ2)*θ2'+m2*l1*cos(θ
1)*l2*cos(θ2)*θ1''+m2*l1*sin(θ1)*l2*sin(θ2)*θ1''+m2*l2^(2)*θ2''


First Equation of motion =
-m2*l1*cos(θ1)*l2*sin(θ2)*θ2'^(2)+m2*l1*sin(θ1)*l2*cos(θ2)*θ2'^(2)+m1*l1^(2)*θ1''+m2*l1^(2)*θ1''+m2*l1*cos(θ1)*l2*cos(θ2)*θ2''+m2*l1*sin(θ1)*l2*sin(θ2)*θ2''+m1*g*l
1*sin(θ1)+m2*g*l1*sin(θ1)

Second Equation of motion =
-m2*l1*sin(θ1)*θ1'^(2)*l2*cos(θ2)+m2*l1*cos(θ1)*θ1'^(2)*l2*sin(θ2)+m2*l1*cos(θ1)*l2*cos(θ2)*θ1''+m2*l1*sin(θ1)*l2*sin(θ2)*θ1''+m2*l2^(2)*θ2''+m2*g*l2*sin(θ2)

Time taken by function: 9332191 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Double Pendulum Problem with Lagrangian Function ]#
```

Figure 13.3: *The computation to obtain the Lagrangian and the equations of motion for double simple pendulum (**SymIntegration/Examples/Test SymIntegration Double Pendulum Problem with Lagrangian Function/main.cpp**).*

We are using Hamzstlab Physics 2D to create a double simple pendulum simulation, we can vary the length from the wall above to the first body and the length from the first body to the second body.

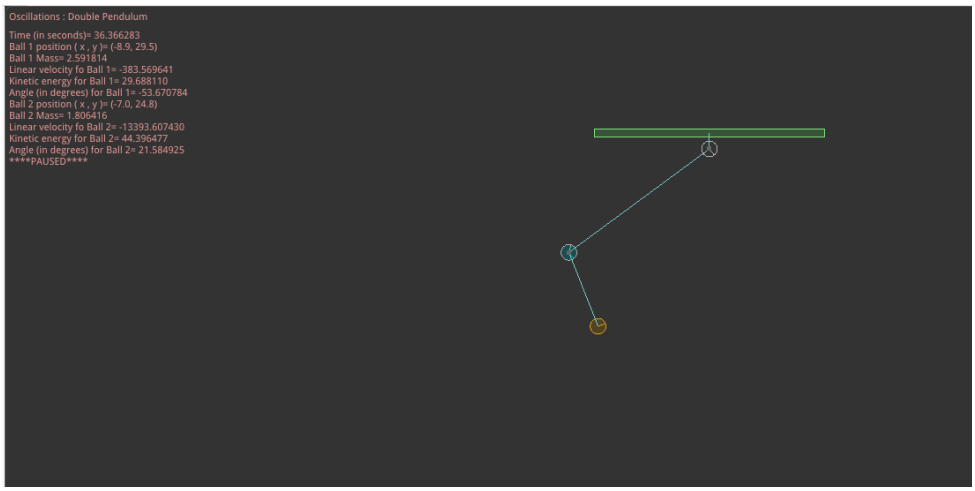

Figure 13.4: *The 2D simulation of a double simple pendulum (**Hamzstlab-Physics2D/tests/double_pendulum.cpp**).*

Bibliography

[1] Banerjee, Sandi. (2022) Mathematical Modeling 2nd Edition: Models, Analysis, and Applications, CRC Press, Boca Raton, Florida, United States.

[2] Boyce, William E., DiPrima, Richard C. (2010) Elementary Differential Equations and Boundary Value Problems 9th Edition, John Wiley & Sons, Hoboken, New Jersey, United States.

[3] Burden, Richard L., Faires, J. Douglas (2001) Numerical Analysis 7th Edition, Brooks/Cole, Pacific Grove, California, United States.

[4] Colley, Susan Jane (2012) Vector Calculus 4th Edition, Pearson, Boston, MA, United States.

[5] Dorf, Richard C., Bishop, Robert H. (2010) Modern Control Systems 12th Edition, Pearson, New Jersey, United States.

[6] Durran, Dale R. (2010) Numerical Methods for Fluid Dynamics, Springer, New York, United States.

[7] Ferziger, Joel H., Peric, Milovan, Street, Robert L. (2020) Computational Methods for Fluid Dynamics 4th Edition, Springer Nature, Switzerland.

[8] Freya, Hamzst, Glanzsche, DS, DianFreya Math Physics Simulator with C++,Berlin-Sentinel Academy of Science, AliceGard.

[9] Gezerlis, Alex (2020) Numerical Methods in Physics with Python, Cambridge University Press, Cambridge, United Kingdom.

[10] Gockenbach, Mark S. (2002) Partial Differential Equations Analytical and Numerical Methods, Society for Industrial and Applied Mathematics, 3600 University City Science Center, Philadelphia, United States.

[11] Lasthrim, Glanzsche, DS, Freya (2025) Lasthrim Projection 1st Edition, Berlin-Sentinel Academy of Science, AliceGard.

[12] Nelder J.A, Mead R. (1965). A simplex method for function minimization. The computer journal, 7(4), 308-313.

[13] Press, W.H., Teukolsky, S.A., Vetterling, W.T. Flannery, B.P. (2007) Numerical Recipes 3rd Edition, Cambridge University Press, Cambridge, United Kingdom.

[14] Purcell, Rigdon, Varberg (2007) Calculus 9th Edition, Pearson, Upper Saddle River, New Jersey, United States.

[15] Kenneth H. Rosen (2006) Discrete Mathematics and its Applications 6th Edition, McGraw-Hill, Boston, United States.

[16] Anton, Rorres (2006) Elementary Linear Algebra with Supplemental Applications 10th Edition, Wiley, Boston, United States.

[17] Anton, Howard, Rorres, Chris, Elementary Linear Algebra with Supplemental Applications 12th edition, Wiley, Hoboken, New Jersey, United States.

[18] Simmons, George F. (2017) Differential Equations with Applications and Historical Notes 3rd Edition, CRC Press, Boca Raton, Florida, United States.

[19] Strauss, Walter A. (2008) Partial Differential Equations: An Introduction 2nd Edition, John Wiley & Sons, New Jersey, United States.